Intelligent Vehicle Verification, Surveillance & Public Safety Platform using AI



Project ID: F25-37

Session: BSSE Spring 2023 to 2027

Project Advisor: Muhammad Jehanzeb

Submitted By

Abdullah Saleem	**70135356**
Areebah Mushtaq	**70143646**
Muhammad Faizan	**70141751**

Department of Software Engineering

The University of Lahore

Lahore, Pakistan

Declaration

We have read the project guidelines, and we understand the meaning of academic dishonesty, in particular plagiarism and collusion. We hereby declare that the work we submitted for our Final Year Project, entitled **"Intelligent Vehicle Verification, Surveillance & Public Safety Platform using AI"**, is original work and has not been printed, published, or submitted before as a Final Year Project, research work, publication, or any other documentation.

Group Member 1 Name: Abdullah Saleem

SAP No: 70135356

Signature: ……

Group Member 2 Name: Areebah Mushtaq

SAP No: 70143646

Signature: ……

Group Member 3 Name: Muhammad Faizan

SAP No: 70141751

Signature: ……

Statement of Submission

This is to certify that **Abdullah Saleem** (SAP No. **70135356**), **Areebah Mushtaq** (SAP No. **70143646**), and **Muhammad Faizan** (SAP No. **70141751**) have successfully submitted the Final Year Project named: **"Intelligent Vehicle Verification, Surveillance & Public Safety Platform using AI"**, at the Department of Software Engineering, Faculty of Information Technology, The University of Lahore, Lahore, Pakistan, to fulfil the partial requirements of the degree of BS in Software Engineering.

Supervisor Name: Muhammad Jehanzeb

Signature: ……

Date: ……

Dedication

This project is dedicated to our parents and families, whose unwavering love, prayers, and sacrifices have been the foundation of every achievement we have achieved. Their encouragement has carried us through every challenge.

We also dedicate this work to our respected supervisor, **Muhammad Jehanzeb**, whose guidance, patience, and expertise have shaped this project into what it is today.

Finally, this project is dedicated to all those working toward building safer, smarter, and more just communities through the thoughtful use of technology.

Acknowledgement

We truly acknowledge the cooperation, guidance, and continuous support provided by our supervisor, **Sir Jehanzeb**, **Department of Software Engineering**, **The University of Lahore**. His constant mentorship, constructive feedback, and commitment throughout the development of this project have been invaluable. We are deeply grateful for the time he dedicated to reviewing our work and helping us refine our ideas.

We would also like to extend our sincere gratitude to **Dr. Sundas Shahzeen** and the **Department of Software Engineering** for their administrative support, clear guidelines, and coordination throughout the Final Year Project process.

We are also thankful to our families and friends whose constant encouragement, patience, and moral support enabled us to remain focused and motivated throughout this demanding project. Their silent strength has been a source of inspiration at every stage of our work.

Date: _____

Abstract

Urban environments in Pakistan face escalating challenges related to vehicle-related crimes, traffic management inefficiencies, and the limitations of conventional Automatic Number Plate Recognition (ANPR) systems. Existing solutions rely primarily on number plate reading, which frequently fails under adverse conditions such as poor lighting, motion blur, physical damage, or deliberate tampering. These shortcomings highlight the urgent need for a more robust and intelligent vehicle verification and public safety system.

This project proposes and develops the Intelligent Vehicle Verification, Surveillance & Public Safety Platform using AI, an advanced, multi-layered platform designed to address these limitations. The system enhances vehicle identification by combining number plate recognition with vehicle attribute analysis (color, type) and Vehicle Re-Identification (Re-ID) techniques, enabling accurate tracking even when plates are obscured or manipulated. Suspicious activity detection, including duplicate and cloned plate identification, is achieved through cross location pattern analysis.

The platform consists of a web-based administrative dashboard built with React.js for authorities, and a mobile application developed using React Native for citizen interaction. Backend services are implemented using Node.js and Express.js, with MongoDB for data storage. AI components leverage OpenCV and TensorFlow for image processing and model development. The system incorporates voice-based complaint submission, edge AI capabilities for offline functionality, role-based access control, audit logging, watchlist management, and structured case management workflows.

The proposed platform aims to significantly improve public safety, enable data-driven traffic management, and support practical deployment within smart city infrastructure in Pakistan.

Area of Project

This project covers the following areas of Computer Science and Software Engineering:

- Artificial Intelligence (AI) and Machine Learning
- Computer Vision and Image Processing
- Web Application Development (Full-Stack)
- Mobile Application Development
- Intelligent Transportation Systems (ITS)
- Edge Computing and Offline AI

Technologies Used

Category	**Technologies**
Frontend (Web)	React.js, HTML5, CSS3, JavaScript
Frontend (Mobile)	React Native
Backend	Node.js, Express.js
Database	MongoDB
AI / ML	TensorFlow, OpenCV, Python
Real-time	Socket.io
Version Control	Git, GitHub
Development Tools	VS Code, Postman, Android Studio

Table of Content

Declaration ii
Statement of Submission iii
Dedication iv
Acknowledgement v
Abstract vi
Area of Project vii
Technologies Used vii
List of Figures xi
List of Tables xii
Chapter 1: Introduction to the Problem 2
1.1 Introduction 2
1.2 Background 3
1.3 Purpose of the System 4
1.4 Objectives 5
1.5 Existing Solutions 6
1.5.1 Punjab Safe Cities Authority (PSCA) Safe City Platform 6
1.5.2 PIPS (Pakistan Integrated Public Safety) System 6
1.5.3 UK National ANPR Infrastructure (NAIRI) 7
1.5.4 Chinese Skynet and Sharp Eyes Systems 7
1.5.5 OpenALPR / Rekor Scout 7
1.6 Competitive Analysis 8
1.7 Proposed Solution 8
1.7.1 AI and Machine Learning Layer 9
1.7.2 Backend Services Layer 9
1.7.3 Web Administrative Dashboard 9
1.7.4 Citizen Mobile Application 10
1.7.5 Edge AI and Offline Functionality 10
1.8 Gap Analysis 10
1.9 Chapter Summary 12
Chapter 2: Software Requirements Specification 14

2.1 Introduction 14
2.1.1 Purpose 14
2.1.2 Scope 14
2.1.3 Definitions, Acronyms, and Abbreviations 15
2.2 Overall Description 17
2.2.1 Product Perspective 17
2.2.2 Product Functions 17
2.2.3 User Classes and Characteristics 17
2.2.4 Operating Environment 18
2.2.5 Design and Implementation Constraints 18
2.2.6 Assumptions and Dependencies 19
2.3 System Requirements 19
2.3.1 Functional Requirements 19
2.3.2 Non-Functional Requirements 39
2.4 Chapter Summary 43
Chapter 3: Use Case Analysis 45
3.1 Introduction 45
3.2 Actors 45
3.3 Aggregated System Use Case Diagram 45
3.4 Individual Use Case Diagrams and Description Tables 46
3.4.1 UC-01: User Registration & Authentication 46
3.4.2 UC-02: Monitor Vehicle Detections in Real Time 48
3.4.3 UC-03: Configure System Zones & Connected Devices 50
3.4.4 UC-04: Manage Alerts Generated by the System 52
3.4.5 UC-05: Handle Case Workflows (Creation to Resolution) 54
3.4.6 UC-06: Link Reports & Evidence with System Data 56
3.4.7 UC-07: Define and Manage Privacy Settings 58
3.4.8 UC-08: Maintain Activity Logs for Accountability 60
3.4.9 UC-09: Manage Watchlists for Flagged Vehicles 62
3.4.10 UC-10: Check Vehicle Status Using Number Plate 64
3.4.11 UC-11: View Limited Safety-Related Information 66

3.4.12 UC-12: Report Suspicious Vehicles with Evidence 68
3.4.13 UC-13: Report Lost or Stolen Number Plates 70
3.4.14 UC-14: Register Personal Vehicle for Monitoring 72
3.4.15 UC-15: Submit Public Complaints (Traffic Issues) 74
3.4.16 UC-16: Detect and Read Number Plates (ANPR Engine) 76
3.4.17 UC-17: Identify Vehicle Attributes (Color & Type) 78
3.4.18 UC-18: Match Vehicles via Re-Identification (Re-ID Engine) 80
3.4.19 UC-19: Offline Data Storage & Synchronization 82
3.5 Chapter Summary 84
Chapter 4: System Design 86
4.1 Introduction 86
4.2 System Architecture Diagram 86
4.3 Entity Relationship Diagram (ERD) 88
4.4 Data Flow Diagrams 96
4.4.1 DFD Level 0 — Context Diagram 96
4.4.2 DFD Level 1 — Process Decomposition 97
4.5 Class Diagram 98
4.6 Activity Diagram 100
4.7 Sequence Diagram 102
4.8 Collaboration Diagram 104
4.9 State Transition Diagram 106
4.10 Component Diagram 108
4.11 Deployment Diagram 110
4.12 Chapter Summary 112

List of Figures

Figure 1: Aggregated System Use Case Diagram — IVVSP 46
Figure 2: Use Case Diagram — User Registration & Authentication 47
Figure 3: Use Case Diagram — Monitor Vehicle Detections in Real Time 49
Figure 4: Use Case Diagram — Configure System Zones & Connected Devices 51
Figure 5: Use Case Diagram — Manage Alerts Generated by the System 53
Figure 6: Use Case Diagram — Handle Case Workflows (Creation to Resolution) 55
Figure 7: Use Case Diagram — Link Reports & Evidence with System Data 57
Figure 8: Use Case Diagram — Define and Manage Privacy Settings 59
Figure 9: Use Case Diagram — Maintain Activity Logs for Accountability 61
Figure 10: Use Case Diagram — Manage Watchlists for Flagged Vehicles 63
Figure 11: Use Case Diagram — Check Vehicle Status Using Number Plate 65
Figure 12: Use Case Diagram — View Limited Safety-Related Information 67
Figure 13: Use Case Diagram — Report Suspicious Vehicles with Evidence 69
Figure 14: Use Case Diagram — Report Lost or Stolen Number Plates 71
Figure 15: Use Case Diagram — Register Personal Vehicle for Monitoring 73
Figure 16: Use Case Diagram — Submit Public Complaints (Traffic Issues) 75
Figure 17: Use Case Diagram — Detect and Read Number Plates (ANPR Engine) 77
Figure 18: Use Case Diagram — Identify Vehicle Attributes (Color & Type) 79
Figure 19: Use Case Diagram — Match Vehicles via Re-Identification (Re-ID Engine) 81
Figure 20: Use Case Diagram — Offline Data Storage & Synchronization 83
Figure 21: IVVSP System Architecture Diagram 87
Figure 22: Entity Relationship Diagram — IVVSP MongoDB Data Model 89
Figure 23: DFD Level 0 — IVVSP Context Diagram 97
Figure 24: DFD Level 1 — IVVSP Process Decomposition 98
Figure 25: Class Diagram — IVVSP Platform 99
Figure 26: Activity Diagram — IVVSP Main System Operational Flow 101
Figure 27: Sequence Diagram — Suspicious Vehicle Report, Alert & Case Creation Flow 103
Figure 28: Collaboration Diagram — Vehicle Detection & Alert Workflow 105
Figure 29: State Transition Diagram — Case & Alert Lifecycles 107
Figure 30: Component Diagram — IVVSP Platform 109
Figure 31: Deployment Diagram — IVVSP Physical Infrastructure 111

List of Tables

Table 1: Competitive Analysis of Existing Solutions vs. IVVSP 8
Table 2: Gap Analysis — Existing Solutions vs. IVVSP Resolutions 11
Table 3: Definitions, Acronyms, and Abbreviations 15
Table 4: FR-01 — User Registration & Authentication 20
Table 5: FR-02 — Monitor Vehicle Detections in Real Time 21
Table 6: FR-03 — Configure System Zones & Connected Devices 22
Table 7: FR-04 — Manage Alerts Generated by the System 23
Table 8: FR-05 — Handle Case Workflows (Creation to Resolution) 24
Table 9: FR-06 — Link Reports & Evidence with System Data 25
Table 10: FR-07 — Define and Manage Privacy Settings 26
Table 11: FR-08 — Maintain Activity Logs for Accountability 27
Table 12: FR-09 — Manage Watchlists for Flagged Vehicles 28
Table 13: FR-10 — Check Vehicle Status Using Number Plate 29
Table 14: FR-11 — View Limited Safety-Related Information 30
Table 15: FR-12 — Report Suspicious Vehicles with Evidence 31
Table 16: FR-13 — Report Lost or Stolen Number Plates 32
Table 17: FR-14 — Register Personal Vehicle for Monitoring 33
Table 18: FR-15 — Submit Public Complaints (Traffic Issues) 34
Table 19: FR-16 — Detect and Read Number Plates (ANPR Engine) 35
Table 20: FR-17 — Identify Vehicle Attributes (Color & Type) 36
Table 21: FR-18 — Match Vehicles via Re-Identification (Re-ID Engine) 37
Table 22: FR-19 — Offline Data Storage & Synchronization 38
Table 23: Non-Functional Requirements — IVVSP 39
Table 24: Use Case Description — User Registration & Authentication 48
Table 25: Use Case Description — Monitor Vehicle Detections in Real Time 50
Table 26: Use Case Description — Configure System Zones & Connected Devices 52
Table 27: Use Case Description — Manage Alerts Generated by the System 54
Table 28: Use Case Description — Handle Case Workflows (Creation to Resolution) 56
Table 29: Use Case Description — Link Reports & Evidence with System Data 58
Table 30: Use Case Description — Define and Manage Privacy Settings 60
Table 31: Use Case Description — Maintain Activity Logs for Accountability 62
Table 32: Use Case Description — Manage Watchlists for Flagged Vehicles 64
Table 33: Use Case Description — Check Vehicle Status Using Number Plate 66
Table 34: Use Case Description — View Limited Safety-Related Information 68
Table 35: Use Case Description — Report Suspicious Vehicles with Evidence 70
Table 36: Use Case Description — Report Lost or Stolen Number Plates 72
Table 37: Use Case Description — Register Personal Vehicle for Monitoring 74
Table 38: Use Case Description — Submit Public Complaints (Traffic Issues) 76
Table 39: Use Case Description — Detect and Read Number Plates (ANPR Engine) 78
Table 40: Use Case Description — Identify Vehicle Attributes (Color & Type) 80
Table 41: Use Case Description — Match Vehicles via Re-Identification (Re-ID Engine) 82

Table 42: Use Case Description — Offline Data Storage & Synchronization 84
Table 43: Data Dictionary — Users Collection 90
Table 44: Data Dictionary — Detections Collection 91
Table 45: Data Dictionary — Alerts Collection 92
Table 46: Data Dictionary — Cases Collection 93
Table 47: Data Dictionary — Watchlist Collection 94
Table 48: Data Dictionary — Reports Collection 94
Table 49: Logical Schema — All IVVSP Collections 95

CHAPTER # 1

Chapter 1: Introduction to the Problem

1.1 Introduction

The rapid growth of urbanization and motorization in developing nations has introduced a spectrum of complex challenges that traditional law enforcement and traffic management infrastructures are ill-equipped to address. In Pakistan, where vehicle registration statistics from the Pakistan Bureau of Statistics report over 30 million registered vehicles as of 2023, incidents of vehicle theft, number plate cloning, hit-and-run offences, and traffic violations have escalated at an alarming rate. Conventional surveillance systems that rely upon closed-circuit television (CCTV) footage and manual monitoring are not only resource-intensive but are also inherently prone to human error and fatigue, rendering them insufficient as standalone safety mechanisms.

Automatic Number Plate Recognition (ANPR) technology, which has been deployed in several countries as a primary vehicle identification tool, presents significant shortcomings when applied to the Pakistani road environment. The efficacy of ANPR systems is critically dependent upon the legibility of number plates; however, adverse environmental conditions such as inadequate lighting, high vehicle speeds, motion blur, physical plate damage, and deliberate tampering or obscuring of plates render these systems unreliable. The absence of a multi-modal identification fallback mechanism means that once a number plate cannot be read, the vehicle becomes effectively untrack able within the surveillance network.

Concurrently, the civic dimension of public safety remains underserved. Citizens who witness suspicious vehicular activity, stolen vehicles, or traffic violations currently possess no structured, technology-mediated channel through which to report such incidents to the relevant authorities in an efficient and verifiable manner. Similarly, authoritative bodies such as the Punjab Safe Cities Authority and provincial traffic police lack a centralized, AI-powered platform that consolidates surveillance feeds, citizen reports, watchlist management, alert generation, and case resolution into a single operational interface.

Artificial Intelligence and Computer Vision technologies have matured substantially over the past decade, offering powerful tools for real-time object detection, attribute recognition, and re-identification. Convolutional Neural Networks (CNNs), object detection architectures such as YOLO (You Only Look Once), and deep learning-based Re-Identification (Re-ID) models now enable systems to identify, track, and verify vehicles through visual characteristics beyond the number plate alone including vehicle color, make, body type, and unique physical features.

These capabilities, when integrated into a cohesive platform, present an unprecedented opportunity to overcome the limitations of existing ANPR-only systems.

This project, the Intelligent Vehicle Verification, Surveillance & Public Safety Platform using AI (hereinafter referred to as IVVSP), is conceived in direct response to these challenges. IVVSP is an advanced, multi-layered software platform that combines AI-driven vehicle identification, real-time surveillance, citizen engagement, and administrative case management to deliver a comprehensive public safety solution tailored to the urban environment of Pakistan.

1.2 Background

Pakistan's urban centers, including Lahore, Karachi, Islamabad, and Rawalpindi, have experienced sustained population growth and a corresponding increase in vehicular density over the past two decades. According to the National Transport Research Centre (NTRC), the annual growth rate of registered vehicles in Pakistan has averaged approximately 8% over the period from 2015 to 2023. This proliferation has placed considerable strain on existing traffic management and law enforcement infrastructure, both of which remain largely reliant upon manual oversight and rudimentary technological tools.

Previous government-led initiatives, such as the Punjab Safe Cities Authority (PSCA) Safe City Project in Lahore, have deployed CCTV networks and rudimentary ANPR cameras at key intersections. While these systems represent a meaningful step toward technology-assisted surveillance, they are characterized by significant operational gaps: the AI capabilities deployed are limited primarily to basic plate reading, and there exists no mechanism for multi-modal vehicle re-identification, citizen-authority interaction, or intelligent case management. Moreover, such systems lack offline processing capabilities, rendering them ineffective in areas with intermittent or absent network connectivity.

The global landscape of intelligent transportation systems (ITS) offers instructive precedents. In the United Kingdom, the National Automatic Number Plate Recognition Infrastructure (NAIRI) has been operational since the early 2000s and is complemented by machine learning-based anomaly detection systems. In the People's Republic of China, extensive AI-powered city surveillance networks incorporate vehicle attribute recognition, facial recognition, and cross-camera tracking. These international examples validate the technical feasibility of the IVVSP approach and inform its architectural design.

The present project builds upon these international precedents while addressing the specific contextual requirements of the Pakistani urban environment, including challenges related to non-standard plate formats, heterogeneous vehicle types, low-light conditions, and the need for a citizen-facing mobile interface in a society with high smartphone penetration but limited digital literacy among certain demographics.

1.3 Purpose of the System

The primary purpose of the Intelligent Vehicle Verification, Surveillance & Public Safety Platform using AI is to establish a robust, scalable, and intelligent technological infrastructure that enhances vehicle-related public safety in Pakistani urban environments. The system is designed to serve two principal user communities: (1) law enforcement and administrative authorities, who require a powerful operational dashboard for real-time monitoring, alert management, watchlist administration, and case resolution; and (2) citizens, who require a transparent, accessible, and secure channel for vehicle status verification and incident reporting.

More specifically, the purpose of IVVSP encompasses the following dimensions:

- **Operational Enhancement:** To provide law enforcement agencies with a technically superior alternative to existing ANPR-only systems by incorporating multi-modal vehicle identification, Re-ID-based cross-camera tracking, and duplicate plate detection.
- **Civic Engagement:** To empower citizens with a structured, verifiable mechanism for reporting suspicious vehicles, lost or stolen number plates, and general traffic complaints through an accessible mobile application.
- **Administrative Efficiency:** To streamline the workflow of case management from initial alert generation through investigation to final resolution, reducing administrative overhead and improving response times.
- **Data-Driven Intelligence:** To generate actionable insights from accumulated surveillance data, enabling authorities to identify patterns of vehicle crime and make evidence-based decisions regarding resource deployment and infrastructure planning.
- **Resilient Operation:** To ensure continuity of core surveillance functions in environments with intermittent network connectivity through edge AI processing and offline data synchronization capabilities.

1.4 Objectives

The following eight objectives define the scope and success criteria of the IVVSP project. Each objective is measurable and directly aligned with the functional and non-functional requirements elaborated in Chapter 2.

1. **Objective 1 – Multi-Modal Vehicle Identification:** To design and implement an AI-powered vehicle identification subsystem that combines Automatic Number Plate Recognition (ANPR) with vehicle attribute recognition (color, type, make) and Vehicle Re-Identification (Re-ID) techniques, thereby enabling accurate vehicle tracking even when number plates are obscured, damaged, or deliberately tampered with.
2. **Objective 2 – Real-Time Surveillance Dashboard:** To develop a comprehensive, role-based web administrative dashboard using React.js that provides law enforcement personnel with real-time vehicle detection feeds, alert notifications, watchlist management, and statistical analytical reporting, ensuring that authorities can monitor and respond to vehicular incidents with minimal latency.
3. **Objective 3 – Citizen Mobile Application:** To build a user-friendly mobile application using React Native that enables citizens to verify vehicle registration status, report suspicious vehicles with multimedia evidence, report lost or stolen number plates, and submit traffic complaints, thereby establishing a structured channel for community participation in public safety.
4. **Objective 4 – Suspicious Activity and Clone Detection:** To implement an intelligent rule-based and pattern-analysis system capable of identifying suspicious vehicular activity, including the detection of duplicate and cloned number plates through cross-location temporal analysis, and the automatic generation of alerts for administrative review.
5. **Objective 5 – Structured Case Management Workflow:** To design and implement a complete case lifecycle management module within the administrative dashboard, enabling the creation, assignment, investigation, and formal resolution of vehicle-related incidents, with full audit trail logging for accountability and legal compliance.
6. **Objective 6 – Offline and Edge AI Capability:** To integrate edge AI processing capabilities that allow core vehicle detection and identification functions to operate locally on surveillance hardware in environments with unreliable or absent network connectivity, with automatic data synchronization upon network restoration.

7. **Objective 7 – Secure and Scalable System Architecture:** To architect the system backend using Node.js and Express.js with MongoDB as the data store, implementing role-based access control (RBAC), encrypted communications, audit logging, and privacy-preserving data management practices, ensuring the platform meets security and scalability standards appropriate for deployment within government infrastructure.
8. **Objective 8 – Voice-Based Complaint Submission:** To incorporate a voice-based complaint submission feature within the mobile application that allows citizens, including those with limited digital literacy, to submit reports through spoken narration, which is subsequently transcribed and categorized by the system for administrative processing.

1.5 Existing Solutions

A review of existing vehicle surveillance and public safety platforms reveals a landscape characterized by technological fragmentation, limited AI integration, and a general absence of citizen-authority interaction features. The following subsections examine representative existing solutions across international and local contexts.

1.5.1 Punjab Safe Cities Authority (PSCA) Safe City Platform

The PSCA Safe City Platform, operational in Lahore since 2016, represents Pakistan's most significant government investment in urban surveillance technology. The platform incorporates an estimated 8,000 CCTV cameras across Lahore and employs basic ANPR capabilities at selected checkpoints. The system is connected to the National Database and Registration Authority (NADRA) database for vehicle owner verification. However, the platform AI capabilities are confined to rudimentary number plate reading; it does not support vehicle attribute recognition, Re-ID-based tracking, citizen reporting features, or intelligent case management workflows. Furthermore, the system is entirely dependent upon network connectivity, with no edge computing fallback mechanism.

1.5.2 PIPS (Pakistan Integrated Public Safety) System

The PIPS system, deployed under the supervision of the Punjab Information Technology Board (PITB), provides a web-based complaint management interface for law enforcement. Citizens may submit complaints through a web portal, and these are routed to the relevant police division. However, PIPS is not integrated with any vehicle surveillance infrastructure; complaints are processed manually, and there is no AI-based verification or cross-referencing of citizen reports

with surveillance data. The system lacks a dedicated mobile application and does not support multimedia evidence of submission or voice-based reporting.

1.5.3 UK National ANPR Infrastructure (NAIRI)

The UK's NAIRI, managed by the National Police Chiefs' Council (NPCC), is one of the most mature ANPR networks globally, processing over 40 million number plate reads per day. The system is integrated with the Police National Computer (PNC) and supports automatic flagging of stolen vehicles, uninsured vehicles, and vehicles associated with criminal investigations. Whilst technologically advanced in terms of plate reading throughput and database integration, NAIRI does not incorporate vehicle attribute recognition or Re-ID capabilities. The system also lacks any citizen-facing interface for public engagement with the surveillance infrastructure.

1.5.4 Chinese Skynet and Sharp Eyes Systems

The People's Republic of China has deployed the Skynet and Sharp Eyes surveillance systems, which represent the most technologically comprehensive urban surveillance networks currently in existence. These systems combine ANPR, facial recognition, vehicle attribute recognition, and cross-camera tracking across millions of cameras. Whilst highly effective in terms of surveillance capability, these systems raise significant concerns regarding civil liberties, data privacy, and government overreach that render their direct adoption inappropriate in a democratic context. Furthermore, the prohibitive cost and infrastructure requirements of these systems place them entirely outside the feasibility constraints of Pakistani municipal deployment.

1.5.5 OpenALPR / Rekor Scout

OpenALPR, now commercialized under the Rekor Scout brand, is a widely adopted open-source ANPR solution that provides number plate reading APIs for integration into custom applications. Rekor Scout offers cloud-based plate reading with a REST API, vehicle make and model recognition, and basic alert capabilities. However, the platform does not include Re-ID functionality, citizen engagement features, case management workflow, or offline edge processing. As a commercial SaaS product, it also presents data sovereignty concerns for Pakistani government deployment, as vehicle data would be transmitted to and stored on overseas servers.

1.6 Competitive Analysis

Table 1.1 presents a structured comparative analysis of the existing solutions discussed above against the proposed IVVSP platform across ten key functional and operational dimensions.

Table 1: Competitive Analysis of Existing Solutions vs. IVVSP

Feature / Criterion	PSCA Safe City	PIPS (PITB)	UK NAIRI	China Skynet	Rekor Scout	IVVSP (Proposed)
ANPR / Plate Reading	✓	✗	✓	✓	✓	✓
Vehicle Attribute Recognition	✗	✗	✗	✓	✓	✓
Vehicle Re-Identification	✗	✗	✗	✓	✗	✓
Clone / Duplicate Detection	✗	✗	Partial	✓	✗	✓
Citizen Mobile Application	✗	Partial	✗	✗	✗	✓
Voice-Based Reporting	✗	✗	✗	✗	✗	✓
Case Management Workflow	✗	✓	✗	✗	✗	✓
Watchlist Management	✓	✗	✓	✓	✓	✓
Offline / Edge AI	✗	✗	✗	Partial	✗	✓
Data Sovereignty (Local)	✓	✓	✓	✓	✗	✓

The analysis presented in Table 1.1 clearly demonstrates that no existing solution comprehensively addresses all ten functional dimensions identified as critical to an effective vehicle surveillance and public safety platform for the Pakistani context. The IVVSP platform is uniquely positioned to address this gap by integrating capabilities that are currently distributed across multiple disparate and incompatible systems.

1.7 Proposed Solution

The Intelligent Vehicle Verification, Surveillance & Public Safety Platform using AI (IVVSP) proposes a unified, AI-powered software platform that transcends the limitations of existing

solutions by integrating multi-modal vehicle identification, real-time surveillance, citizen engagement, and intelligent administrative management into a single, cohesive system. The proposed solution is structured around three primary architectural tiers: an AI/ML Processing Layer, a Backend Services Layer, and a Presentation Layer comprising both a web administrative dashboard and a citizen-facing mobile application.

1.7.1 AI and Machine Learning Layer

The AI layer constitutes the technical core of the proposed solution. It comprises three principal subsystems. First, the ANPR Engine, built upon OpenCV and TensorFlow, performs real-time detection and optical character recognition of vehicle number plates from surveillance camera feeds. Second, the Vehicle Attribute Recognition module employs a convolutional neural network trained to classify vehicle color, body type (sedan, SUV, motorcycle, truck, etc.), and, where possible, make and model. Third, the Vehicle Re-Identification Engine applies deep metric learning techniques to match vehicles across multiple camera views using visual appearance features, enabling continuous tracking even when number plates are absent or unreadable. Together, these three subsystems provide a multi-modal identification capability that is substantially more robust than any single-modal approach.

1.7.2 Backend Services Layer

The backend is implemented using Node.js and Express.js, providing a RESTful API architecture that serves both the web dashboard and mobile application. MongoDB is employed as the primary data store, providing the schema flexibility required to accommodate heterogeneous data types including vehicle records, surveillance events, alert logs, citizen reports, case files, and audit trails. Socket.io is integrated for real-time bidirectional communication between the server and client interfaces, enabling live alert delivery and real-time feed updates. Role-Based Access Control (RBAC) is implemented to ensure that system resources are accessible only to appropriately authorized personnel, and all inter-service communications are encrypted using TLS protocols.

1.7.3 Web Administrative Dashboard

The web-based administrative dashboard, developed in React.js, provides law enforcement and administrative personnel with a comprehensive operational interface. Core dashboard modules include: real-time vehicle detection monitoring with live camera feed integration; alert management with configurable notification thresholds; watchlist administration for flagged vehicles and persons of interest; case management with structured workflow stages from

creation through investigation to resolution; analytics and reporting with visualized statistics on vehicle incidents, detection rates, and geographic crime density; system zone and device configuration; and a full audit log viewer for accountability compliance.

1.7.4 Citizen Mobile Application

The citizen-facing mobile application, developed in React Native for cross-platform deployment on both Android and iOS, provides the public with a structured interface for engagement with the surveillance platform. Key citizen features include vehicle status verification by number plate; reporting of suspicious vehicles with photo/video evidence; reporting of lost or stolen number plates; submission of personal vehicle registration for monitoring; submission of traffic complaints; and voice-based report narration for users with limited digital literacy. All citizen-submitted reports are routed through the backend to the administrative dashboard for review and action.

1.7.5 Edge AI and Offline Functionality

Recognizing the reality of intermittent network connectivity in many Pakistani urban and peri-urban environments, the IVVSP platform incorporates edge AI capabilities that enable core vehicle detection and identification functions to operate locally on camera-attached edge computing hardware. Detected events are cached locally and synchronized with the central MongoDB instance upon network restoration, ensuring no surveillance data is lost during connectivity interruptions.

1.8 Gap Analysis

The gap analysis presented below formalizes the deficiencies identified in existing solutions and maps each gap directly to a specific feature of the proposed IVVSP platform. This structured mapping ensures that the design and implementation priorities of the project are directly informed by real-world operational deficiencies.

Table 2: Gap Analysis — Existing Solutions vs. IVVSP Resolutions

Gap ID	Identified Gap	Limitation in Existing Systems	IVVSP Resolution
G-01	Reliance on number plate readability as the sole vehicle identification mechanism	PSCA, UK NAIRI, and Rekor Scout all fail when plates are obscured, damaged, or cloned, with no fallback identification method.	IVVSP combines ANPR with vehicle attribute recognition and Re-ID to maintain identification capability regardless of plate legibility.
G-02	Absence of duplicate and cloned plate detection	No existing deployed Pakistani system identifies vehicles using the same plate at multiple locations simultaneously.	IVVSP performs cross-location temporal analysis to detect plate duplication and automatically generates alerts for administrative review.
G-03	No structured citizen engagement channel integrated with surveillance data	PIPS handles complaints in isolation from surveillance infrastructure. PSCA and NAIRI have no citizen-facing interface.	The IVVSP mobile application connects citizen reports directly to the surveillance backend, enabling cross-referencing of citizen-submitted evidence with AI-detected events.
G-04	Absence of voice-based reporting for citizens with limited digital literacy	All existing systems require text-based input for report submission, creating a barrier for citizens with limited literacy or digital skills.	IVVSP includes a voice narration feature in the mobile application that transcribes and categorizes spoken complaints automatically.
G-05	Lack of structured case lifecycle management	PSCA and NAIRI generate alerts but provide no structured workflow for case investigation and resolution. PIPS manages complaints but is disconnected from surveillance data.	IVVSP implements a full case management workflow from alert generation through assignment, investigation, evidence linking, and formal resolution.
G-06	No offline or edge AI processing capability	All reviewed systems are dependent on continuous network connectivity. Loss of connection	IVVSP supports local edge AI processing on connected devices, with data caught locally and synchronized to the

		results in complete cessation of detection and alerting functions.	central database upon connectivity restoration.
G-07	Data sovereignty and privacy concerns in commercial solutions	Rekor Scout stores vehicle data on overseas commercial servers, creating data sovereignty and privacy compliance risks for government deployment.	IVVSP is designed for on-premises or local-cloud deployment, ensuring all vehicle and citizen data remains within Pakistan's jurisdictional boundaries.
G-08	Absence of analytics and data-driven decision support for authorities	Existing deployed systems in Pakistan produce no actionable analytical outputs. Decisions regarding surveillance of resource deployment are made without data evidence.	IVVSP provides an analytics module within the administrative dashboard that visualizes vehicle incident trends, geographic hotspots, and detection statistics to support evidence-based decision-making.

The eight identified gaps collectively define the design space within which the IVVSP platform has been architected. The gap analysis thereby serves as a traceable bridge between the problem context established in this chapter and the technical solution elaborated in subsequent chapters.

1.9 Chapter Summary

This chapter has established the contextual foundation for the Intelligent Vehicle Verification, Surveillance, & Public Safety Platform using AI. The discussion has demonstrated that Pakistan's rapidly expanding urban vehicular environment presents serious and escalating public safety challenges that existing technological solutions whether local or international are inadequate to address comprehensively. Through a systematic review of five existing systems and a structured gap analysis identifying eight critical deficiencies, the rationale for the IVVSP platform has been firmly established. The proposed solution, comprising an AI/ML processing layer, a Node.js backend, a React.js administrative dashboard, and a React Native mobile application, directly addresses each identified gap through a combination of multi-modal AI-powered vehicle identification, structured citizen engagement, intelligent case management, and offline edge processing capabilities.

CHAPTER # 2

Chapter 2: Software Requirements Specification

2.1 Introduction

This chapter presents the Software Requirements Specification (SRS) for the Intelligent Vehicle Verification, Surveillance & Public Safety Platform using AI (IVVSP). The SRS constitutes a comprehensive, formal record of the functional and non-functional requirements that govern the design, implementation, testing, and acceptance of the system. It serves as the primary contractual reference between the development team and project stakeholders, ensuring that all system behavior is unambiguously defined, traceable, and verifiable.

2.1.1 Purpose

The purpose of this SRS is to provide a precise and complete specification of all requirements for the IVVSP system. This document is intended for use by the development team during design and coding, by the project supervisor during review and evaluation, and as the authoritative reference for system testing and validation. All 19 functional requirements and 25 non-functional requirements defined herein are directly traceable to the project objectives established in Chapter 1 and to the design artefacts presented in Chapters 3 and 4.

2.1.2 Scope

The IVVSP system encompasses three principal software components: (1) an AI/ML processing layer responsible for vehicle detection, plate recognition, attribute classification, and re-identification; (2) a Node.js/Express.js backend API with MongoDB providing data persistence and business logic; and (3) two client-facing interfaces comprising a React.js web administrative dashboard and a React Native citizen mobile application. The system is designed for deployment within the urban surveillance infrastructure of Pakistani municipal authorities.

This SRS covers all functionality within the above scope. Hardware procurement, physical camera installation, third-party telecommunications infrastructure, and NADRA database integration agreements are considered external dependencies and are not specified herein.

2.1.3 Definitions, Acronyms, and Abbreviations

Table 3: Definitions, Acronyms, and Abbreviations

Term	Definition
ANPR	Automatic Number Plate Recognition — automated reading of vehicle plates from images or video using OCR.
API	Application Programming Interface — a set of defined methods for communication between software components.
CNN	Convolutional Neural Network — a class of deep learning models optimized for image classification and recognition tasks.
CNIC	Computerized National Identity Card — the national identity document of Pakistani citizens issued by NADRA.
DFD	Data Flow Diagram — a graphical representation of data movement through a system.
Edge AI	Artificial intelligence executed locally on edge computing hardware, without reliance on central cloud connectivity.
ERD	Entity Relationship Diagram — a model of database entities and their relationships.
FR	Functional Requirement — a specification of a specific behavior or function the system must perform.
IVVSP	Intelligent Vehicle Verification, Surveillance & Public Safety Platform using AI — the system described in this document.
ITS	Intelligent Transportation System — technology-based systems for improving transport infrastructure and management.
JWT	JSON Web Token — a compact, digitally signed token used for secure authentication and authorization.
MongoDB	A NoSQL document-oriented database providing flexible schema storage used as the primary data store in IVVSP.
NADRA	National Database and Registration Authority — Pakistan's government agency responsible for civil registration and identity databases.

NFR	Non-Functional Requirement — a specification of a quality attribute or operational constraint of the system.
NTRC	National Transport Research Centre — Pakistan's government body for transport research and statistics.
OCR	Optical Character Recognition — technology for converting images of text into machine-readable character strings.
OTP	One-Time Password — a temporary, single-use code sent via SMS or email for identity verification.
PII	Personally Identifiable Information — any data that can be used to identify an individual citizen.
PSCA	Punjab Safe Cities Authority — the Pakistani government body responsible for the Lahore Safe City Project.
RBAC	Role-Based Access Control — a security model restricting system access based on a user's assigned role.
Re-ID	Vehicle Re-Identification — the technique of matching the same vehicle across multiple camera views using visual appearance features.
REST	Representational State Transfer — an architectural style for designing networked APIs using standard HTTP methods.
RTSP	Real Time Streaming Protocol — a network protocol used for controlling streaming media servers and IP camera feeds.
SRS	Software Requirements Specification — this document; a formal description of system requirements.
TLS	Transport Layer Security — a cryptographic protocol ensuring encrypted communications over a computer network.
YOLO	You Only Look Once — a real-time object detection architecture used in the IVVSP vehicle detection pipeline.

2.2 Overall Description

2.2.1 Product Perspective

IVVSP is a new, independent software system developed to address identified deficiencies in Pakistan's current vehicle surveillance and public safety infrastructure. The system interfaces with existing physical surveillance hardware through standard RTSP streaming protocols, and optionally with the NADRA vehicle registration database through a government-authorized API. Internally, the system employs microservice-oriented architecture with a centralized MongoDB database, a RESTful API gateway, and a Socket.io real-time event bus connecting the AI processing layer, backend services, and client interfaces.

2.2.2 Product Functions

The principal functions of the IVVSP system are summarized below:

- **AI-Powered ANPR:** Real-time plate detection and OCR from live surveillance camera feeds.
- **Vehicle Attribute Recognition:** Automated classification of vehicle color and body type to supplement plate identification.
- **Vehicle Re-Identification:** Cross-camera vehicle tracking using deep metric learning visual embeddings.
- **Clone and Duplicate Detection:** Cross-location temporal analysis to identify vehicles using the same plate simultaneously at multiple locations.
- **Administrative Dashboard:** Role-based web interface for real-time monitoring, alert management, watchlist control, case management, and analytics.
- **Citizen Mobile Application:** Cross-platform application for vehicle status enquiry, suspicious reporting, complaint submission, and voice-based narration.
- **Edge AI and Offline Operation:** Local processing and event caching during outages with automatic synchronization on reconnection.

2.2.3 User Classes and Characteristics

The IVVSP system serves the following distinct user classes:

- **Super-Administrator:** Highest privilege level; responsible for system configuration, zone and device management, privacy settings, and user provisioning. Technically proficient law enforcement of IT personnel.

- **Zone Administrator:** Responsible for monitoring alerts, managing cases, and administering watchlists within an assigned zone. Operational law enforcement supervisors with moderate technical proficiency.
- **Investigation Officer:** Responsible for case investigation, evidence review, and status updates. Law enforcement officers with basic system proficiency.
- **Citizen:** Members of the public using mobile applications. Variable technical proficiency: voice-based input is provided for accessibility.

2.2.4 Operating Environment

The IVVSP system is designed for the following technical environment:

- **Server:** Ubuntu Server 22.04 LTS; Node.js 20.x; MongoDB 7.0; Python 3.11 (AI layer); Docker 24.x.
- **Web Client:** Google Chrome 120+, Mozilla Firefox 120+, Microsoft Edge 120+ on Windows 10/11 and macOS 13+.
- **Mobile Client:** Android 10 (API Level 29) and above; iOS 14 and above; React Native 0.73+.
- **Edge Devices:** NVIDIA Jetson Nano or equivalent ARM-based SBC with GPU for local AI inference.
- **Network:** Primary operation over TCP/IP with RTSP for camera streams; edge AI fallback on connectivity loss.

2.2.5 Design and Implementation Constraints

The following constraints are imposed upon the design and implementation of IVVSP:

- **Technology Stack:** The system must be implemented using React.js, React Native, Node.js/Express.js, MongoDB, TensorFlow, and OpenCV as specified in the approved project proposal FYP-F25-37.
- **Data Sovereignty:** All citizen and vehicle data must reside on servers physically located within Pakistan. No data shall be transmitted to overseas commercial cloud services.
- **Pakistani Plate Formats:** The ANPR engine must support all current Pakistani provincial plate formats including legacy and NTRC-compliant modern formats.
- **Open-Source Constraint:** The system must be implementable exclusively using open-source AI frameworks and database infrastructure to remain within academic project resource boundaries.

2.2.6 Assumptions and Dependencies

The following assumptions and external dependencies are acknowledged:

- **Camera Availability:** IP cameras with accessible RTSP streams are assumed to be available for integration during system testing.
- **NADRA Integration:** For FYP Phase 1, a simulated vehicle registration dataset is used in lieu of a live NADRA API connection.
- **Network Infrastructure:** Stable broadband connectivity is assumed for normal operation; the edge AI module addresses outage scenarios.
- **Training Data:** A labelled dataset of Pakistani vehicle images (from public datasets supplemented with local samples) is assumed to be obtainable for model training.

2.3 System Requirements

2.3.1 Functional Requirements

The following 19 functional requirements define all mandated system behaviors. Each is presented in a structured six-column table specifying its identifier (ID), name, description, inputs, outputs, and basic workflow. All identifiers from FR-01 through FR-19 are consistent with the use of case diagrams in Chapter 3 and the design diagrams in Chapter 4, ensuring full cross-document traceability.

Table 4: FR-01 — User Registration & Authentication

ID	Name	Description	Input	Output	Basic Workflow
FR-01	**User Registration & Authentication**	The system shall allow administrative personnel and citizens to create accounts and authenticate them securely. Administrators are provisioned by a super-admin. Citizens self-register via mobile application. Role-Based Access Control (RBAC) governs permitted operations after logging in.	Full name, email, password Role (Admin/ Citizen) CNIC number (citizens) Phone number for OTP verification	JWT access to the token on success Role-specific dashboard loaded. Error message on invalid credentials OTP SMS dispatched for 2FA	**1.** User opens the web dashboard or mobile application and selects Register. **2.** User completes the registration form with all mandatory fields. **3.** System validates inputs; an OTP is dispatched to the provided phone number. **4.** User enters the OTP; system verifies identity. **5.** Account is created, and the appropriate RBAC role is assigned. **6.** User logs in with email and password; a signed JWT is returned. **7.** User is redirected to the role-specific home screen.

Table 5: FR-02 — Monitor Vehicle Detections in Real Time

ID	Name	Description	Input	Output	Basic Workflow
FR-02	**Monitor Vehicle Detections in Real Time**	The system shall provide authorized administrators with a live surveillance dashboard displaying real-time vehicle detection events from all connected camera feeds via WebSocket. Events include detected number plates, vehicle attributes, timestamps, and camera zone identifiers.	Live video frames from IP cameras Camera zone ID and timestamp Admin authentication token	Real-time detection event cards on dashboard Detected plate string with confidence score. Vehicle color and type labels Zone ID and timestamp per detection Alert badge count updated automatically	**1.** Admin logs in and navigates to the 'Live Monitor' module. **2.** System establishes a WebSocket connection to the backend event stream. **3.** The AI processing layer continuously runs ANPR and attribute recognition on incoming frames. **4.** Each detection event is published via Socket.io to all connected admin clients. **5.** Dashboard renders each event card with plate, attributes, zone, and timestamp. **6.** Watchlist-matched plates trigger red highlighting and automatic alert. **7.** Admin may click any event card to expand full detection details.

Table 6: FR-03 — Configure System Zones & Connected Devices

ID	Name	Description	Input	Output	Basic Workflow
FR-03	**Configure System Zones & Connected Devices**	The system shall allow super-administrators to define geographic surveillance zones, register camera devices within those zones, set alert thresholds per zone, and monitor device health status in real time.	Zone name and geographic boundary coordinates Camera device name, IP address, stream URL Alert sensitivity thresholds Device assigned zone ID	Zone record created in database with unique Zone ID Camera device registered and assigned to zone. Device health status: Online / Offline/ Degraded Confirmation message on successful save	**1.** Super-admin navigates to 'Zone & Device Configuration'. **2.** Admin defines a new zone by name and geographic boundary on an interactive map. **3.** System saves the zone and generates a unique Zone ID. **4.** Admin registers a camera by entering IP address, streaming URL, and assigning a zone. **5.** System verifies stream connectivity and records of device health. **6.** Admin sets alert sensitivity thresholds for the zone. **7.** New zone and device appear immediately on the live monitor.

Table 7: FR-04 — Manage Alerts Generated by the System

ID	Name	Description	Input	Output	Basic Workflow
FR-04	**Manage Alerts Generated by the System**	The system shall automatically generate, display, and track alerts for suspicious vehicular events including watchlist matches, duplicate plate sightings, and speed anomalies. Admins may acknowledge, escalate, dismiss, or link alerts to active investigation cases.	Triggered detection event with plate, zone, timestamp Alert type: Watchlist Hit / Clone / Speed / Anomaly Admin action: Acknowledge / Escalate / Dismiss / Link to Case	Alert record with unique Alert ID Push notification to responsible admin(s) Alert status: New / Acknowledged / Escalated / Closed Audit log entry recording the action and actor	**1.** The AI layer or rule engine detects a triggering condition and raises an alert. **2.** System creates an Alert record with ID, classification, severity, and timestamp. **3.** Real-time push notification dispatched to all online admins. **4.** Alert appears in a notification panel with color-coded severity (Red / Amber / Blue). **5.** Admin reviews the alert snapshot, plate, and zone. **6.** Admin selects an action: Acknowledge, Escalate, Dismiss, or Link to Case. **7.** Alert status and audit log are updated with the action and actor identity.

Table 8: FR-05 — Handle Case Workflows (Creation to Resolution)

ID	Name	Description	Input	Output	Basic Workflow
FR-05	**Handle Case Workflows (Creation to Resolution)**	The system shall provide a structured case lifecycle enabling administrators to create investigation cases, assign them to officers, progress them through defined stages, and formally resolve them. A complete, immutable audit trail is maintained throughout.	Case title, type, description, priority Linked alert or report IDs Assigned officer (admin user ID) Status transitions: Open / In Progress / Under Review / Closed Resolution notes and evidence attachments	Case record with unique Case ID Officer notification on assignment Stage transition logged with timestamp and actor. Closed-case summary report generated. Audit trail entry for each action	**1.** Admin identifies an incident requiring investigation from an alert or citizen report. **2.** Admin creates a case entering title, type, description, priority, and linked source. **3.** System assigns a unique Case ID and sets status to 'Open'. **4.** Admin assigns the case to an officer; officer receives an in-app notification. **5.** Officer sets status to 'In Progress' and adds investigation notes and evidence. **6.** Officer sets status to 'Under Review' for senior approval. **7.** Senior admin reviews evidence adds resolution notes and formally closes the case. **8.** System generates a case summary; audit log records all stage transitions.

Table 9: FR-06 — Link Reports & Evidence with System Data

ID	Name	Description	Input	Output	Basic Workflow
FR-06	**Link Reports & Evidence with System Data**	The system shall allow administrators to associate citizen-submitted reports and upload multimedia evidence (images, video, audio) with AI-generated detection events and active investigation cases, creating a unified evidential record.	Citizen report ID or detection event ID Target case ID Uploaded files: JPEG, PNG, MP4, MP3, PDF Admin annotation text	Evidence records with unique Evidence ID Report linked to case and detection events. Files stored in secure document repository Evidence timeline updated on case detail page	**1.** Admin opens an active case and navigates to the 'Evidence' tab. **2.** Admin searches for the citizen report or detection event to link. **3.** System displays the record with metadata and thumbnail. **4.** Admin confirms the link; system associates the record with the case. **5.** Admin optionally uploads additional evidence files. **6.** System stores files securely and assigns Evidence IDs. **7.** Evidence timeline updates chronologically on the case page.

Table 10: FR-07 — Define and Manage Privacy Settings

ID	Name	Description	Input	Output	Basic Workflow
FR-07	**Define and Manage Privacy Settings**	The system shall allow super-administrators to configure data retention policies, access control rules, field-level anonymization, and citizen consent records to ensure compliance with applicable data protection regulations.	Retention period (days) per data category Role-permission matrix updates Anonymization toggle per data field Citizen consent record	Updated privacy configuration saved. Automated purging schedule activated. Access restriction applied immediately Consent log entry recorded per citizen	**1.** Super-admin navigates to 'Privacy & Compliance' settings. **2.** Admin sets data retention periods per category (detections, reports, audit logs). **3.** System activates automated purging jobs for records exceeding retention limits. **4.** Admin updates the role-permission matrix; changes take effect on next user login. **5.** Admin enables field anonymization (e.g., CNIC masked in exports). **6.** All configuration changes are logged in the audit trail with actor and timestamp.

Table 11: FR-08 — Maintain Activity Logs for Accountability

ID	Name	Description	Input	Output	Basic Workflow
FR-08	**Maintain Activity Logs for Accountability**	The system shall automatically generate and preserve immutable audit logs for all significant user actions, data access operations, and configuration changes. Logs shall be searchable, filterable, and exportable for compliance and legal review.	System-generated event (action, API call, config change) User ID, role, IP address, timestamp Filter parameters: date range, user, action type	Immutable log entry per significant event Filtered log list for authorized admin Exportable report in CSV or PDF format	**1.** System auto-captures a log entry on every significant event. **2.** Each entry records: actor ID, role, action type, resource, timestamp, IP address. **3.** Entries are written to an append-only MongoDB collection preventing modification. **4.** Admin navigates to 'Audit Logs' and applies filters. **5.** System returns matching entries in a paginated, sortable table. **6.** Admin exports selected entries as CSV or PDF for compliance submission.

Table 12: FR-09 — Manage Watchlists for Flagged Vehicles

ID	Name	Description	Input	Output	Basic Workflow
FR-09	**Manage Watchlists for Flagged Vehicles**	The system shall allow authorized administrators to create and maintain watchlists of number plates of investigative interest. Immediate high-severity alerts are generated when a watch listed plate is detected by the surveillance network.	Number plate string. Reason: Stolen / Wanted / Suspect Expiry date (optional) Responsible officer assignment	Watchlist entry with unique Watchlist ID Confirmation of successful addition High-severity alert on detection of watch listed plate. Entry status: Active / Expired / Removed	**1.** Admin navigates to 'Watchlist Management. **2.** Admin enters the plate, selects a reason category, and optionally sets an expiry date. **3.** System checks for duplicate entries to prevent conflicts. **4.** Entry is saved and flagged for real-time matching by the ANPR engine. **5.** On detection of the watch listed plate, a 'High' severity alert is immediately raised. **6.** Alert notification is pushed to the assigned officer and zone administrator. **7.** Admin may deactivate or remove entries at any time; all changes are audited.

Table 13: FR-10 — Check Vehicle Status Using Number Plate

ID	Name	Description	Input	Output	Basic Workflow
FR-10	**Check Vehicle Status Using Number Plate**	The system shall allow authenticated citizens to query a vehicle's number plate and receive its publicly permissible registration status. Sensitive law enforcement data is withheld; only safe public-facing fields are returned.	Number plate string (typed or camera-scanned) Citizen authentication token	Status: Registered / Unregistered / Reported Stolen Public flag: Suspicious / Clear Date of last verification Error message for invalid plate format	**1.** Citizen navigates to 'Vehicle Status Check' in the mobile application. **2.** Citizen types of the plate number or scans using the device camera. **3.** System validates the plate format against Pakistani registration rules. **4.** Backend queries the vehicle database and watchlist. **5.** System composes a public-safe response with only permitted status fields. **6.** Application displays the result with a color-coded indicator (Green / Amber / Red). **7.** Search action is logged with citizen ID and timestamp for audit purposes.

Table 14: FR-11 — View Limited Safety-Related Information

ID	Name	Description	Input	Output	Basic Workflow
FR-11	**View Limited Safety-Related Information**	The system shall provide authenticated citizens with access to publicly permissible safety information including traffic advisories, anonymized zone-level incident summaries, and the status of their own submitted reports. No operational or sensitive data is exposed.	Citizen authentication token Zone or city filter (optional) Own report ID (for status check)	List of active public traffic advisories Anonymized zone-level incident count summary Status of citizen's own reports No cross-citizen or sensitive data returned	**1.** Citizen logs in and navigate to the 'Safety Information' section. **2.** System retrieves active public advisories from the admin notice board. **3.** Citizen optionally filters by zone or city for localized information. **4.** Citizen selects My Reports to check the status of their own submissions. **5.** System returns only the authenticated citizen's own records. **6.** Results are displayed in a readable list with status badges and timestamps.

Table 15: FR-12 — Report Suspicious Vehicles with Evidence

ID	Name	Description	Input	Output	Basic Workflow
FR-12	**Report Suspicious Vehicles with Evidence**	The system shall allow authenticated citizens to submit reports of suspicious vehicle activity with photographic or video evidence. Reports are routed to the zone administrator and auto-cross-referenced with AI surveillance data.	Number plate (optional) Description of suspicious activity GPS location or address Attached photos or video: JPEG, PNG, MP4 Citizen authentication token	Report with unique Report ID Citizen confirmation notification Report in admin review queue Auto-cross-reference result with watchlist and detection history	**1.** Citizen navigates to 'Report Suspicious Vehicle'. **2.** Citizen fills in vehicle description and suspected activity. **3.** Citizen attaches photographic or video evidence. **4.** Citizen confirms GPS location. **5.** System validates mandatory fields and uploads evidence securely. **6.** Report is created and routed to the administrator's queue. **7.** System cross-references the plate (if provided) with watchlist and detection history. **8.** Citizen receives confirmation notification with their Report ID.

Table 16: FR-13 — Report Lost or Stolen Number Plates

ID	Name	Description	Input	Output	Basic Workflow
FR-13	**Report Lost or Stolen Number Plates**	The system shall allow authenticated citizens to formally report loss or theft of their vehicle number plate. The plate is automatically flagged in the surveillance watchlist to prevent misuse and alert authorities upon subsequent detection.	Plate number string. Report type: Lost / Stolen Date and location of incident Vehicle registration document (image upload) Citizen authentication token	Report with unique Report ID Plate flagged in surveillance watchlist Admin notification for ownership verification Citizen confirmation with Report ID	**1.** Citizen navigates to 'Report Lost/Stolen Plate'. **2.** Citizen selects report type and enters the plate number. **3.** Citizen provides the incident date, location, and uploads the registration document. **4.** System validates the plate format and checks existing records. **5.** Report is submitted; plate is automatically added to the watchlist. **6.** Zone administrator receives a notification to verify ownership and confirm the flag. **7.** Citizen receives confirmation with Report ID and advisory on the next steps.

Table 17: FR-14 — Register Personal Vehicle for Monitoring

ID	Name	Description	Input	Output	Basic Workflow
FR-14	**Register Personal Vehicle for Monitoring**	The system shall allow authenticated citizens to voluntarily register their personal vehicle(s) for active surveillance monitoring. The citizen receives automated notifications when their plate is detected at any surveillance camera.	Plate number string. Vehicle make, model, color, year. Registration document (image upload) Notification preference: Push / SMS / Both Citizen authentication token	Vehicle registered to citizen profile Vehicle added to personalized monitoring list Notification dispatched on each detection Confirmation with Vehicle Registration ID	**1.** Citizen navigates to 'My Vehicles' and selects 'Add Vehicle'. **2.** Citizen enters plate, make, model, color, and year. **3.** Citizen uploads the vehicle registration document as proof of ownership. **4.** System validates the document and checks for duplicate registrations. **5.** Vehicle is added to the citizen's monitoring profile. **6.** ANPR engine updates to include the plate in the personalized notification list. **7.** Each detection triggers a push or SMS notification to the registered citizen. **8.** Citizen views detection history in the 'My Vehicles' section.

Table 18: FR-15 — Submit Public Complaints (Traffic Issues)

ID	Name	Description	Input	Output	Basic Workflow
FR-15	**Submit Public Complaints (Traffic Issues)**	The system shall provide citizens with a structured channel for submitting traffic-related complaints such as reckless driving, illegal parking, signal violations, and road hazards. Voice-based narration is supported for accessibility. Complaints are categorized and routed to the appropriate authority.	Category: Reckless Driving / Illegal Parking / Signal Violation / Road Hazard / Other Vehicle plate (if applicable) Description text or voice recording GPS location or address Supporting photo/video (optional)	Complaint with unique Complaint ID Complaint routed to appropriate authority queue Citizen confirmation notification with ID Status updates as complaint progresses	**1.** Citizen navigates to 'Submit Complaint'. **2.** Citizen selects the complaint category. **3.** Citizen enters a text description or taps the microphone to record a voice narration. **4.** If voice input is selected, the system transcribes the narration and populates the description field. **5.** Citizen optionally enters the offending plate number and attaches evidence of media. **6.** Citizen confirms GPS location. **7.** System submits the complaint, creates a Complaint ID, and routes it to the authority. **8.** Citizen receives confirmation; status updates are delivered as the complaint is processed.

Table 19: FR-16 — Detect and Read Number Plates (ANPR Engine)

ID	Name	Description	Input	Output	Basic Workflow
FR-16	**Detect and Read Number Plates (ANPR Engine)**	The system shall automatically detect and read vehicle number plates from live surveillance camera feeds using an AI-powered ANPR engine built on OpenCV and TensorFlow. The engine shall function under adverse conditions including low light, motion blur, and partial occlusion, and shall support all current Pakistani plate formats.	Live video frame from surveillance camera Camera zone ID and timestamp Confidence threshold parameter	Detected plate string (OCR result) Bounding box coordinates in frame Confidence score (0.00–1.00) Detection event record saved to database. Event published to real-time dashboard	**1.** Camera streams video frames to the AI processing layer at the configured rate. **2.** The YOLO-based vehicle detection model identifies vehicle regions in each frame. **3.** Plate localization model extracts the number plate region from each vehicle. **4.** The OCR model reads plate characters and produces a text string with confidence score. **5.** Detections meeting the confidence threshold are accepted; others are discarded. **6.** Accepted detection events are saved to the MongoDB detections collection. **7.** Events are published via Socket.io to the dashboard and cross-referenced with the watchlist.

Table 20: FR-17 — Identify Vehicle Attributes (Color & Type)

ID	Name	Description	Input	Output	Basic Workflow
FR-17	**Identify Vehicle Attributes (Color & Type)**	The system shall automatically classify vehicle color and body type (sedan, SUV, motorcycle, truck, van, bus) from surveillance camera frames using CNN. Attribute data supplements to ANPR results to maintain identification capability when plates are unreadable.	Vehicle region-of-interest image cropped from detection frame. Camera zone ID and timestamp Attribute classification model configuration	Vehicle color label (e.g., White, Black, Silver, Red) Vehicle body type label (e.g., Sedan, SUV, Motorcycle) Confidence score per attribute Attribute record linked to parent detection event	**1.** Upon vehicle detection, the bounding box region is cropped from the frame. **2.** Cropped image is passed to the attribute classification of CNN. **3.** Color classification head predicts color label and confidence score. **4.** Body type classification head predicts type label and confidence score. **5.** Both predictions are appended to the parent detection event record. **6.** When ANPR fails, attribute data enables vehicle search by color and type. **7.** Attribute data is displayed on the admin dashboard alongside plate and timestamp.

Table 21: FR-18 — Match Vehicles via Re-Identification (Re-ID Engine)

ID	Name	Description	Input	Output	Basic Workflow
FR-18	**Match Vehicles via Re-Identification (Re-ID Engine)**	The system shall employ a Vehicle Re-ID engine using deep metric learning to match the same physical vehicle across multiple camera views when its number plate is absent, obscured, or has changed. This enables continuous cross-camera vehicle tracking and clone/duplicate plate detection.	Vehicle appearance embedded from current detection. Candidate embeddings from detection history (configurable time window) Cosine similarity threshold Zone transition rules	Re-ID result: Matched / No Match Matched detection record ID Cosine similarity score Updated cross-camera travel path Clone/Duplicate alert when same plate found in multiple zones simultaneously	**1.** A new vehicle detection event is generated by the ANPR engine. **2.** The Re-ID feature extractor generates visual embedding from the vehicle image. **3.** System queries detection history for candidate embeddings within the time window. **4.** Cosine similarity is computed between the new embedding and each candidate. **5.** A match is declared if the highest similarity score exceeds the threshold. **6.** The matched vehicle's cross-camera travel path is updated with the new sighting. **7.** If the same plate is detected in two geographically distant zones within an implausible time limit, a Clone/Duplicate alert is raised.

Table 22: FR-19 — Offline Data Storage & Synchronization

ID	Name	Description	Input	Output	Basic Workflow
FR-19	**Offline Data Storage & Synchronization**	The system shall support edge AI operation wherein ANPR and attribute recognition run locally on camera-attached hardware without continuous internet connectivity. Detection events are cached locally during offline periods and automatically synchronized with the central MongoDB database upon network restoration.	Detection events generated during offline period Network connectivity status signal. Edge device ID and sync credentials Local SQLite cache file	Events processed and caught locally during outage. Sync batch uploaded automatically on reconnection Conflict resolution log for overlapping records. Sync status indicator on admin dashboard	**1.** Edge device detects loss of network and switches to offline processing mode. **2.** ANPR and attribute models continue to run locally; events are written to the local SQLite cache. **3.** Admin dashboard shows the device as 'Offline – Caching'. **4.** On network restoration, the edge device starts a synchronization handshake with the backend. **5.** Cached events are transmitted to the backend via the synchronization of API. **6.** Backend confirms and deduplicates records before committing to MongoDB. **7.** Local cache is cleared; device status reverts to 'Online'. **8.** A sync summary (events uploaded, duplicates skipped, errors) is recorded in the audit log.

2.3.2 Non-Functional Requirements

The following 25 non-functional requirements specify the quality attributes, operational constraints, and compliance standards that the IVVSP system must satisfy. They are organized into eight categories: Performance (NFR-01 to NFR-04), Security (NFR-05 to NFR-09), Reliability (NFR-10 to NFR-12), Usability (NFR-13 to NFR-15), Scalability (NFR-16 to NFR-18), Maintainability (NFR-19 to NFR-20), Availability (NFR-21), Portability (NFR-22 to NFR-23), and Privacy & Compliance (NFR-24 to NFR-25). Each requirement includes a measurable metric or target to support objective verification.

Table 23: Non-Functional Requirements — IVVSP

ID	Category	Requirement	Metric / Target	Priority
NFR-01	**Performance**	The ANPR engine shall process each video frame and return a detection result within 200 milliseconds under normal operating conditions at 1080p resolution.	Plate detection latency ≤ 200 ms per frame @ 1080p	**High**
NFR-02	**Performance**	The web administrative dashboard shall load the first view and establish the WebSocket connection within 3 seconds on a 50 Mbps broadband connection.	Dashboard load time ≤ 3 seconds (50 Mbps)	**High**
NFR-03	**Performance**	The backend API shall support a minimum of 500 concurrent users (admin and citizen) without API response time exceeding 500 milliseconds.	API response time ≤ 500 ms at 500 concurrent users	**High**
NFR-04	**Performance**	The mobile application shall return vehicle status query results within 5 seconds under 4G LTE cellular data conditions.	Vehicle status API response ≤ 5 seconds on 4G LTE	Medium
NFR-05	**Security**	All client-to-backend communications shall be encrypted using TLS 1.2 or higher. Unencrypted HTTP	100% of API endpoints enforce HTTPS / TLS 1.2+	**High**

		connections shall be rejected with a 301 redirect.		
NFR-06	**Security**	User passwords shall be hashed using bcrypt with a minimum cost factor of 12 before storage. Plaintext passwords shall never be stored or logged at any time.	bcrypt cost factor ≥ 12; zero plaintext password storage	**High**
NFR-07	**Security**	The RBAC implementation shall ensure that citizens cannot access any administrative module, and administrators cannot access data outside their assigned zone without explicit grant.	Zero unauthorized cross-role data access in penetration tests	**High**
NFR-08	**Security**	JWT access tokens shall expire after 60 minutes. Refresh tokens shall expire after 7 days. Token revocation on logout shall be enforced via a server-side deny-list.	Access token TTL = 60 min; refresh token TTL = 7 days	**High**
NFR-09	**Security**	All public-facing API endpoints shall enforce rate limit of 100 requests per minute per source IP address. Requests exceeding the limit shall receive an HTTP 429 response.	Rate limit: 100 req/min per IP; HTTP 429 on breach	Medium
NFR-10	**Reliability**	The central backend system shall maintain a minimum uptime of 99.5% over any rolling 30-day period, excluding scheduled maintenance windows of no more than 2 hours per calendar month.	Uptime ≥ 99.5% per 30-day window; scheduled downtime ≤ 2 hrs/month	**High**
NFR-11	**Reliability**	The edge AI subsystem shall sustain local vehicle detection and cache all events without data loss for a minimum of 72 continuous hours during network outages.	Offline operation ≥ 72 hrs; zero cached event data loss	**High**

NFR-12	**Reliability**	The system shall perform automated database backups every 24 hours. Recovery from a complete system failure shall be achievable within a Recovery Time Objective (RTO) of 4 hours.	Backup interval = 24 hrs; RTO ≤ 4 hours	**High**
NFR-13	**Usability**	A first-time citizen user shall be able to register an account, complete identity verification, and submit their first report within 5 minutes without requiring external assistance.	Task completion ≤ 5 minutes for new citizen in usability testing	Medium
NFR-14	**Usability**	The voice complaint submission feature shall achieve a speech-to-text transcription accuracy of at least 85% for standard Pakistani English and Urdu narrations on the validation dataset.	Speech-to-text accuracy ≥ 85% on validation set	Medium
NFR-15	**Usability**	The administrative dashboard shall provide contextual help tooltips for all major interactive elements, and a searchable in-application user guide shall always be accessible.	100% of primary UI elements have tooltips; in-app guide present	Low
NFR-16	**Scalability**	The backend API layer shall support zero-downtime horizontal scaling such that additional server instances can be added behind a load balancer without architectural changes.	Zero-downtime horizontal scale-out via load balancer	**High**
NFR-17	**Scalability**	The MongoDB schema and indexing strategy shall sustain query response times below 500 milliseconds at a dataset size of 10 million detection events, 1 million citizen reports, and 500,000 case records.	Query response ≤ 500 ms at 10M detection records with indexes	**High**

NFR-18	**Scalability**	The system shall support the provisioning of new surveillance camera zones by a super-administrator entirely through the administrative user interface within 15 minutes, without code changes.	New zone provisionable via admin UI in ≤ 15 minutes	Medium
NFR-19	**Maintainability**	The backend codebase shall adhere to the MVC architectural pattern. A minimum of 70% test coverage shall be achieved on all service and controller modules as measured by the Jest testing framework.	Test coverage ≥ 70% on backend service and controller layers	Medium
NFR-20	**Maintainability**	All backend API endpoints shall be documented using the Open API 3.0 specification. Documentation shall be kept synchronized with the deployed API throughout the development of lifecycle.	100% of API endpoints documented in Open API 3.0 spec	Medium
NFR-21	**Availability**	The system shall provide a real-time infrastructure health dashboard visible to super-administrators, displaying CPU usage, memory consumption, API response times, and camera device connectivity status, updated every 30 seconds.	Health dashboard refresh interval = 30 seconds; all metrics visible	Medium
NFR-22	**Portability**	The React Native mobile application shall function correctly on Android 10 (API Level 29) and above, and iOS 14 and above, covering at least 95% of active smartphones in the Pakistani market.	Functional on Android 10+ and iOS 14+; tested on 5 device form factors	**High**
NFR-23	**Portability**	All backend services shall be containerized using Docker. The complete system shall be deployable in	Full deployment via Docker Compose in ≤ 10 minutes	Medium

		a clean environment using Docker Compose within 10 minutes.		
NFR-24	**Privacy & Compliance**	All personally identifiable information (PII) including CNIC numbers, phone numbers, and vehicle registration details shall be encrypted at rest using AES-256 encryption within the MongoDB database.	AES-256 encryption applied to all PII fields at rest	**High**
NFR-25	**Privacy & Compliance**	Unlinked surveillance detection of event records shall be automatically purged after 90 days. Records linked to active cases shall be retained for the case duration plus an additional 180 days, after which they shall be archived.	Automated purge at 90 days for unlinked events; case-linked events retained per policy	**High**

2.4 Chapter Summary

This chapter has presented a comprehensive Software Requirements Specification for the IVVSP platform. The 19 functional requirements defined in Section 2.3.1 collectively specify all mandated system behaviors across three principal user classes — administrative personnel, citizens, and automated AI subsystems — covering the complete functional scope of the approved project proposal FYP-F25-37. The 25 non-functional requirements in Section 2.3.2 establish measurable quality targets for performance, security, reliability, usability, scalability, maintainability, availability, portability, and privacy compliance.

The requirement identifiers FR-01 through FR-19 and NFR-01 through NFR-25 defined in this chapter serve as the authoritative reference for all subsequent design and implementation artefacts. Chapter 3 proceeds to develop the use case analysis, providing individual use case diagrams and structured description tables for each of the 19 functional requirements identified herein.

CHAPTER # 3

Chapter 3: Use Case Analysis

3.1 Introduction

This chapter presents the use of case analysis for the Intelligent Vehicle Verification, Surveillance, and Public Safety Platform using AI (IVVSP). Use case analysis constitutes a structured method for specifying the interactions between actors and the system, ensuring that all functional requirements identified in Chapter 2 are traceable to observable system behaviors. Each use case corresponds directly to one of the 19 functional requirements (FR-01 through FR-19) defined in the Software Requirements Specification.

This chapter is organized into three parts. Section 3.2 presents the aggregated system-wide use case diagram, providing a high-level view of all actors and use cases within the IVVSP boundary. Section 3.3 presents individual use case diagrams and structured use case description tables for each of the 19 functional requirements. All use case identifiers (UC-01 through UC-19) correspond directly to the requirement identifiers FR-01 through FR-19 and are consistent with the design diagrams presented in Chapter 4.

3.2 Actors

The following actors interact with the IVVSP system:

- **Super-Administrator:** Highest-privilege internal user; manages system configuration, privacy, zones, and user provisioning.
- **Zone Administrator:** Manages alerts, cases, and watchlists within an assigned zone; primary operational user of the web dashboard.
- **Investigation Officer:** Handles case investigation, evidence review, and audit log access.
- **Citizen:** External user of the mobile application; checks vehicle status, submits reports, and views of safety information.
- **AI / System:** Internal automated actors including the ANPR engine, Re-ID engine, rule engine, attribute CNN, and edge device; these generate detection events and alerts without human intervention.

3.3 Aggregated System Use Case Diagram

Figure 3.1 presents the aggregated system use case diagram for the IVVSP platform. It illustrates the complete set of use cases grouped into three functional domains Administrative Functions, Citizen Functions, and AI & System Functions and maps each use case to its

interacting actor(s). Dependency relationships between use cases are shown with dashed arrows.

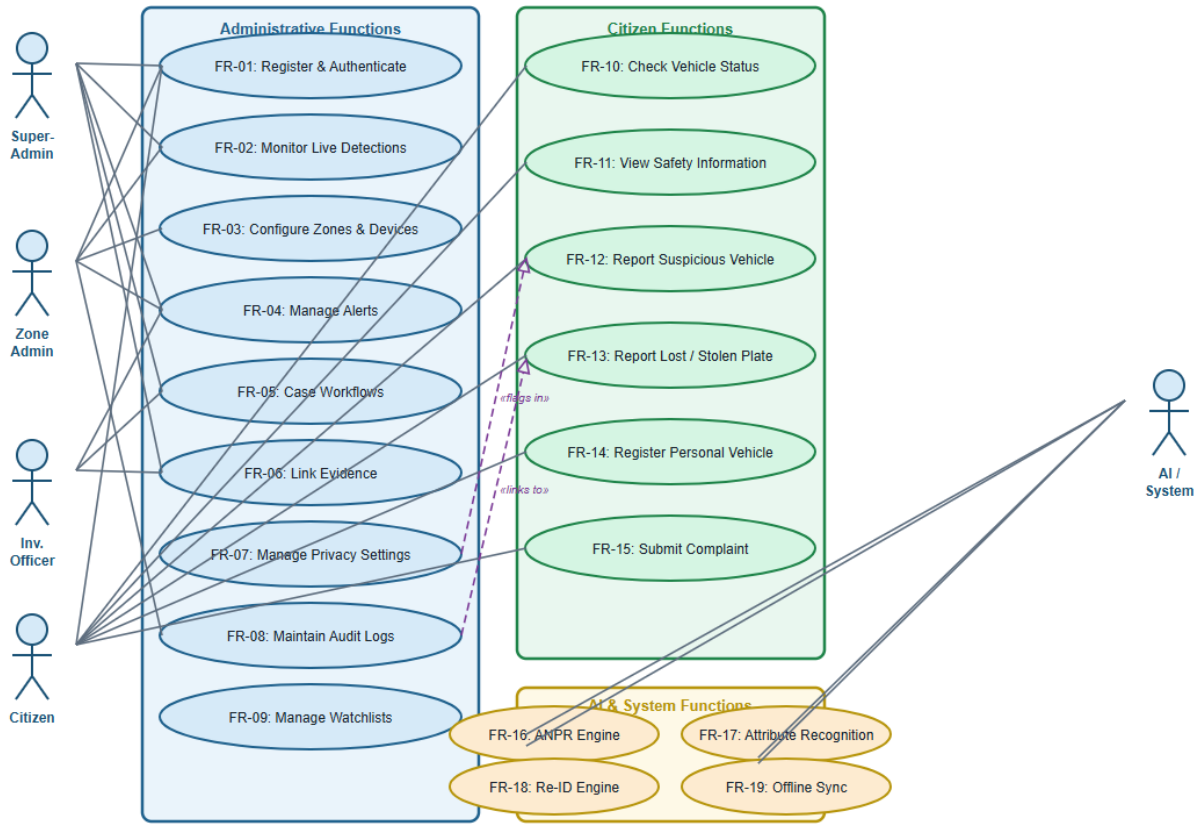


Figure 1: Aggregated System Use Case Diagram — IVVSP

3.4 Individual Use Case Diagrams and Description Tables

The following sections present individual use case diagrams and structured description tables for each of the 19 functional requirements. Each subsection contains: (a) the use case diagram illustrating actors and their relationships to the use case, and (b) the use case description table specifying actors, pre-conditions, post-conditions, main flow, alternative flow, and exception flow.

3.4.1 UC-01: User Registration & Authentication

Figure 3.2 illustrates the use case diagram for UC-01 (User Registration & Authentication), corresponding to functional requirement FR-01. The diagram shows all participating actors and their relationships to the use case, including included and extended sub-cases.

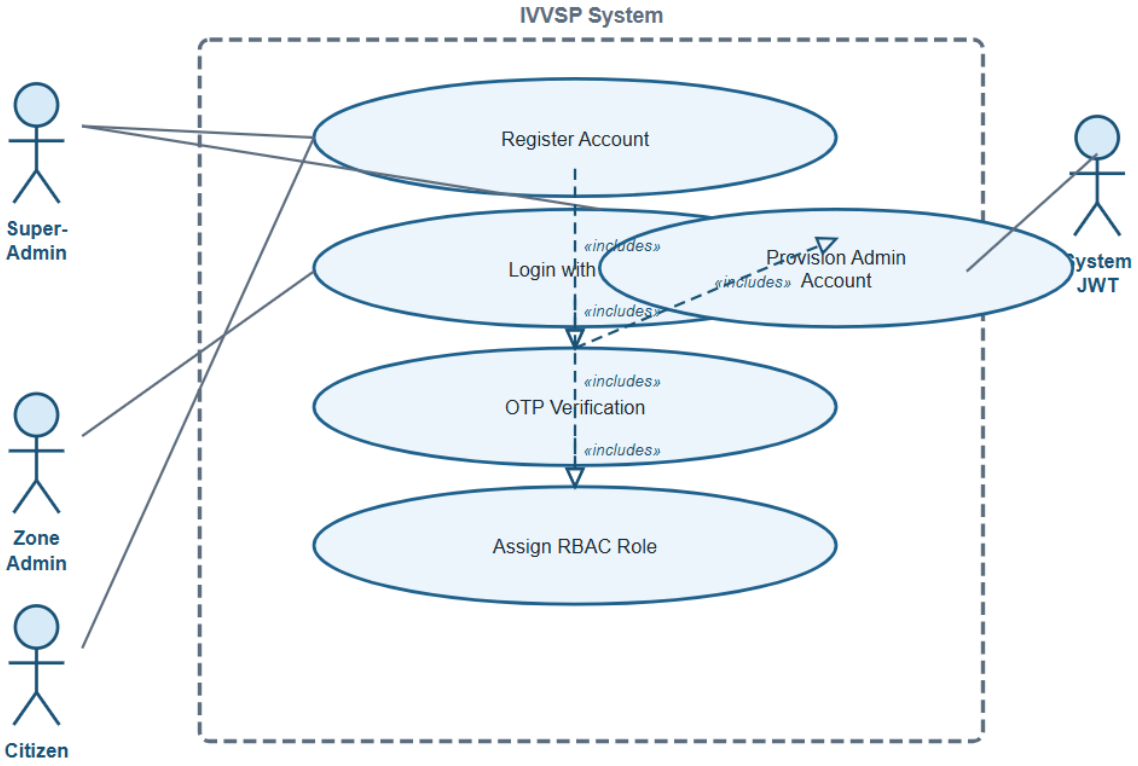


Figure 2: Use Case Diagram — User Registration & Authentication

Table 3.1 presents the structured use case description for UC-01, specifying the main flow, alternative flows, and exception handling for this interaction.

Table 24: Use Case Description — User Registration & Authentication

Field	Description
Use Case ID	UC-01
Use Case Name	User Registration & Authentication
Actor(s)	Administrator (Super-Admin, Zone Admin), Citizen
Description	Allows administrative personnel and citizens to create accounts and log in securely. RBAC assigns appropriate permissions upon account creation. Two-factor authentication via OTP is enforced for all users.
Pre-condition	The IVVSP application is accessible. The user has a valid email address and phone number. For admin accounts, a super-admin must exist.
Post-condition	A verified user account is created with the correct RBAC role. A signed JWT access token is issued. The user is directed to their role-specific dashboard.
Main Flow	1. User navigates to the Registration page. 2. User fills in full name, email, password, CNIC (citizen) or role (admin). 3. The system sends an OTP to the registered phone number. 4. User enters the OTP for verification. 5. System creates the account, assigns the RBAC role, and redirects it to the home screen. 6. On subsequent visits, the user logs in with email and password to receive a JWT token.
Alternative Flow	Admin accounts are created by a super-administrator through the admin provisioning panel rather than the self-registration flow.
Exception Flow	If OTP expires before entry, the system prompts the user to request a new OTP. If credentials are invalid, an error message is displayed and login is denied.
Related FR	FR-01
Priority	High

3.4.2 UC-02: Monitor Vehicle Detections in Real Time

Figure 3.3 illustrates the use case diagram for UC-02 (Monitor Vehicle Detections in Real Time), corresponding to functional requirement FR-02. The diagram shows all participating actors and their relationships to the use case, including included and extended sub-cases.

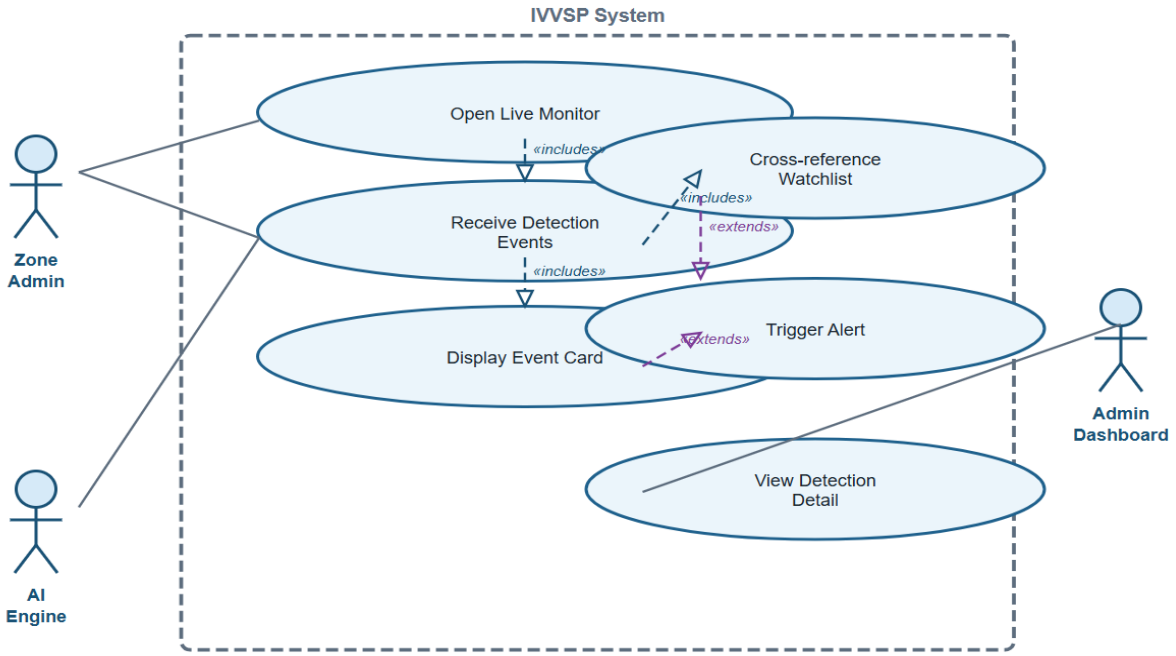


Figure 3: Use Case Diagram — Monitor Vehicle Detections in Real Time

Table 3.2 presents the structured use case description for UC-02, specifying the main flow, alternative flows, and exception handling for this interaction.

Table 25: Use Case Description — Monitor Vehicle Detections in Real Time

Field	Description
Use Case ID	UC-02
Use Case Name	Monitor Vehicle Detections in Real Time
Actor(s)	Zone Administrator, Super-Administrator
Description	Provides authorized administrators with a live surveillance dashboard that streams vehicle detection events from all connected cameras via WebSocket. Each event shows the detected plate, vehicle attributes, timestamp, and zone.
Pre-condition	Admin is authenticated. At least one camera device is registered, online, and streaming. The AI processing layer is active.
Post-condition	Detection events are displayed on the live monitor in real time. Watchlist-matched detections trigger alerts automatically.
Main Flow	1. Admin logs in and opens the 'Live Monitor' module. 2. System opens a WebSocket connection to the backend event stream. 3. AI layer processes incoming frames and publishes detection events via Socket.io. 4. Dashboard renders each event as a card with plate, attributes, zone, and timestamp. 5. Watchlist-matched plates are highlighted in red, and an alert is raised. 6. Admin clicks a card to expand the full detection detail view.
Alternative Flow	If no cameras are online, the dashboard displays a 'No Active Feeds' notice but remains open.
Exception Flow	If the WebSocket connection drops, the UI shows a 'Connection Lost – Reconnecting' banner and attempts to automatic reconnection every 5 seconds.
Related FR	FR-02
Priority	High

3.4.3 UC-03: Configure System Zones & Connected Devices

Figure 3.4 illustrates the use case diagram for UC-03 (Configure System Zones & Connected Devices), corresponding to functional requirement FR-03. The diagram shows all participating actors and their relationships to the use case, including included and extended sub-cases.

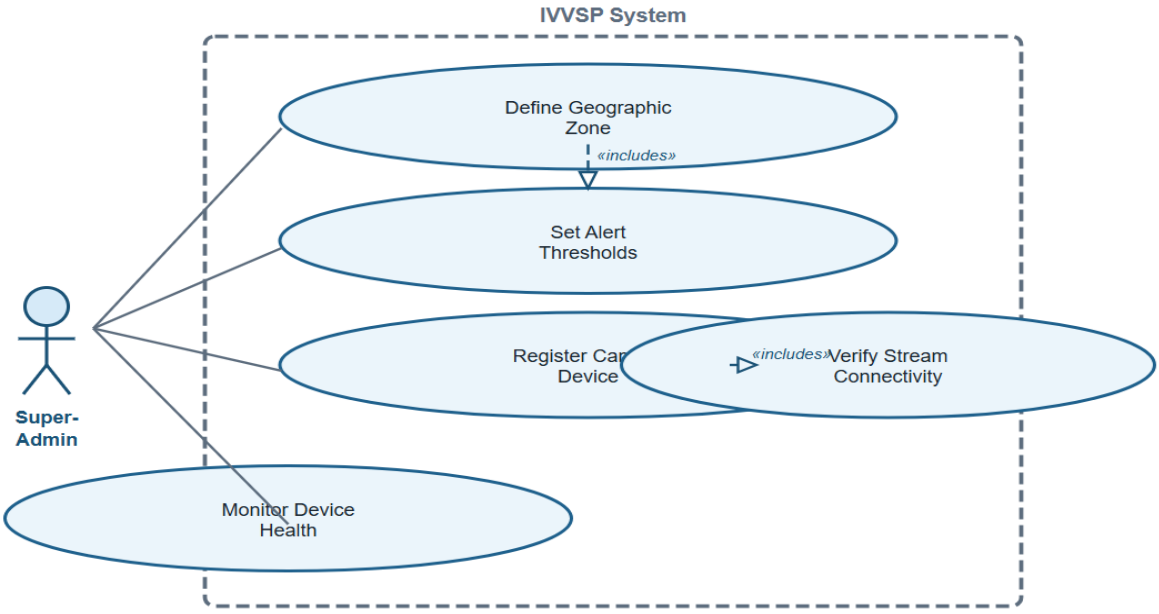


Figure 4: Use Case Diagram — Configure System Zones & Connected Devices

Table 3.3 presents the structured use case description for UC-03, specifying the main flow, alternative flows, and exception handling for this interaction.

Table 26: Use Case Description — Configure System Zones & Connected Devices

Field	Description
Use Case ID	UC-03
Use Case Name	Configure System Zones & Connected Devices
Actor(s)	Super-Administrator
Description	Allows the super-administrator to define geographic surveillance zones, register IP cameras within those zones, set alert sensitivity thresholds, and monitor device health status.
Pre-condition	Super-admin is authenticated. IP cameras are physically installed and accessible on the network.
Post-condition	Zones and devices are saved to the database. Cameras begin streaming and appear in the live monitor. Device health status is displayed.
Main Flow	1. Super-admin navigates to 'Zone & Device Configuration'. 2. Admin draws zone boundaries on the interactive map and names for the zone. 3. System generates a unique Zone ID and saves the record. 4. Admin adds a camera by entering its IP address, streaming URL, and assigning it to a zone. 5. System verifies connectivity to the stream and sets device health status. 6. Admin sets alert thresholds for the zone. 7. Zones and cameras appear on the live monitor.
Alternative Flow	If stream connectivity fails, the device status is set to 'Offline' and the admin is notified.
Exception Flow	If the stream URL format is invalid, the system rejects entry and displays a validation error.
Related FR	FR-03
Priority	High

3.4.4 UC-04: Manage Alerts Generated by the System

Figure 3.5 illustrates the use case diagram for UC-04 (Manage Alerts Generated by the System), corresponding to functional requirement FR-04. The diagram shows all participating actors and their relationships to the use case, including included and extended sub-cases.

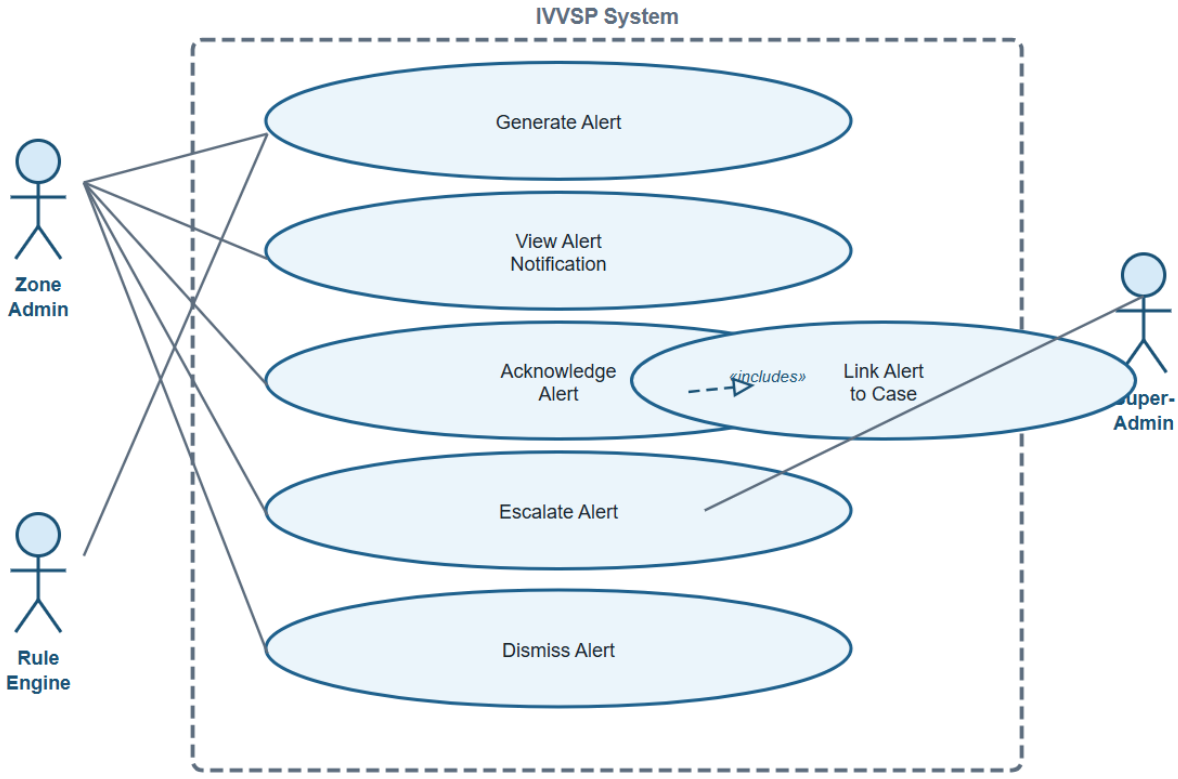


Figure 5: Use Case Diagram — Manage Alerts Generated by the System

Table 3.4 presents the structured use case description for UC-04, specifying the main flow, alternative flows, and exception handling for this interaction.

Table 27: Use Case Description — Manage Alerts Generated by the System

Field	Description
Use Case ID	UC-04
Use Case Name	Manage Alerts Generated by the System
Actor(s)	Zone Administrator, Super-Administrator, Rule Engine (System)
Description	The system automatically generates alerts for watchlist matches, duplicate plate detections, and anomalies. Administrators may acknowledge, escalate, dismiss, or link alerts to investigation cases.
Pre-condition	The ANPR engine is running. Watchlist entries are configured. Admin is authenticated and online.
Post-condition	Alert status is updated and recorded in the audit log. Escalated alerts notify senior administrators.
Main Flow	1. Rule engine or AI detects a triggering event and creates an alert. 2. System records the alert with ID, type, severity, and timestamp. 3. Push notification is sent to all online admins. 4. Admin views the alert on the notification panel. 5. Admin selects an action: Acknowledge, Escalate, Dismiss, or Link to Case. 6. Status and audit logs are updated with the action and actor.
Alternative Flow	Dismissed alerts are archived with a false-positive label and do not trigger further notifications.
Exception Flow	If the admin is offline, the alert remains in 'New' status until reviewed. Unacknowledged alerts older than 30 minutes escalate automatically.
Related FR	FR-04
Priority	High

3.4.5 UC-05: Handle Case Workflows (Creation to Resolution)

Figure 3.6 illustrates the use case diagram for UC-05 (Handle Case Workflows (Creation to Resolution)), corresponding to functional requirement FR-05. The diagram shows all participating actors and their relationships to the use case, including included and extended sub-cases.

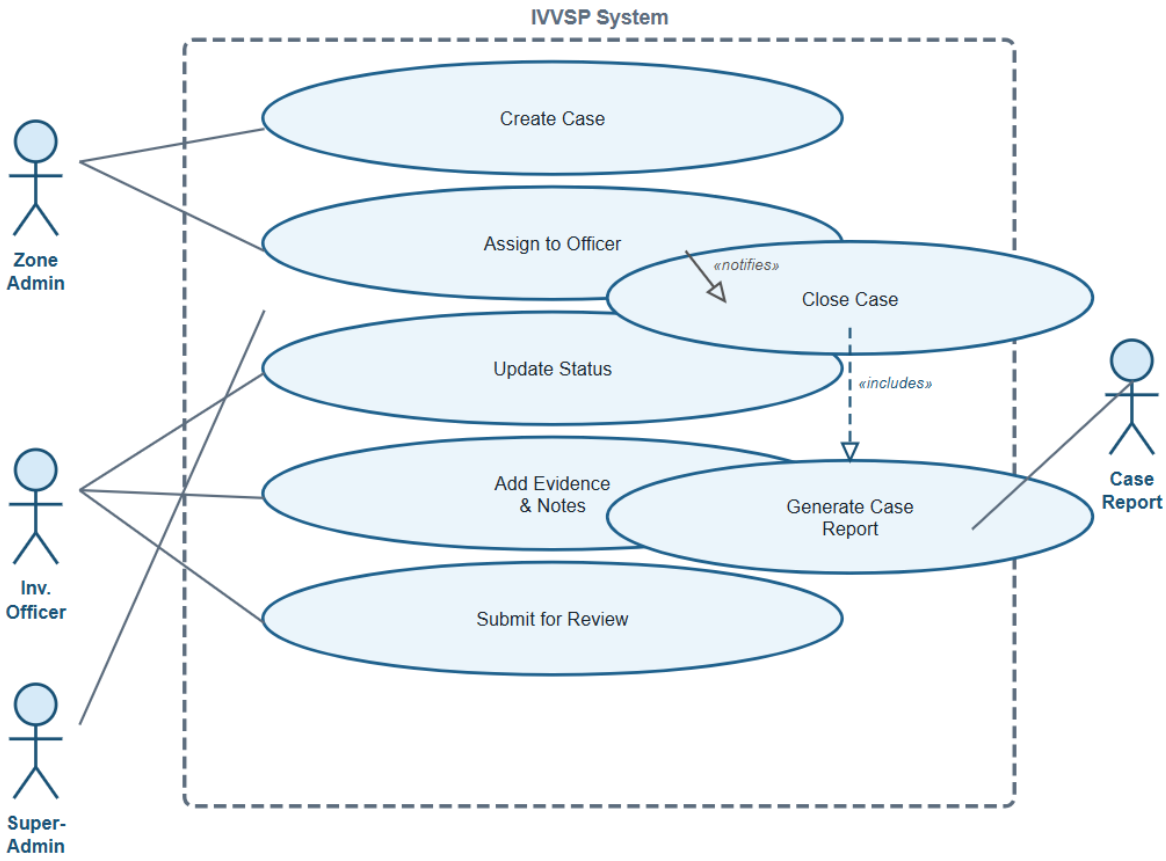


Figure 6: Use Case Diagram — Handle Case Workflows (Creation to Resolution)

Table 3.5 presents the structured use case description for UC-05, specifying the main flow, alternative flows, and exception handling for this interaction.

Table 28: Use Case Description — Handle Case Workflows (Creation to Resolution)

Field	Description
Use Case ID	UC-05
Use Case Name	Handle Case Workflows (Creation to Resolution)
Actor(s)	Zone Administrator, Investigation Officer, Super-Administrator
Description	Provides a structured case lifecycle from creation through investigation to formal resolution, with full audit logging at each stage.
Pre-condition	An alert or citizen report exists as the basis for the case. Users have appropriate RBAC roles assigned.
Post-condition	Case is closed with a documented outcome. Audit trail captures all stage transitions. A case summary report is generated.
Main Flow	1. Admin creates a case linked to an alert or report, entering title, type, description, and priority. 2. System assigns a unique Case ID and sets status to 'Open'. 3. Admin assigns the case to an officer; the officer is notified. 4. Officer updates status to 'In Progress', adds notes and evidence. 5. Officer sets status to 'Under Review'. 6. Senior admin reviews, adds resolution notes, and closes the case. 7. System generates a case summary report.
Alternative Flow	A case may be re-opened if new evidence emerges after initial closure, with super-admin authorization.
Exception Flow	If an assigned officer is unavailable, the case is returned to 'Open', and a new assignment is made.
Related FR	FR-05
Priority	High

3.4.6 UC-06: Link Reports & Evidence with System Data

Figure 3.7 illustrates the use case diagram for UC-06 (Link Reports & Evidence with System Data), corresponding to functional requirement FR-06. The diagram shows all participating actors and their relationships to the use case, including included and extended sub-cases.

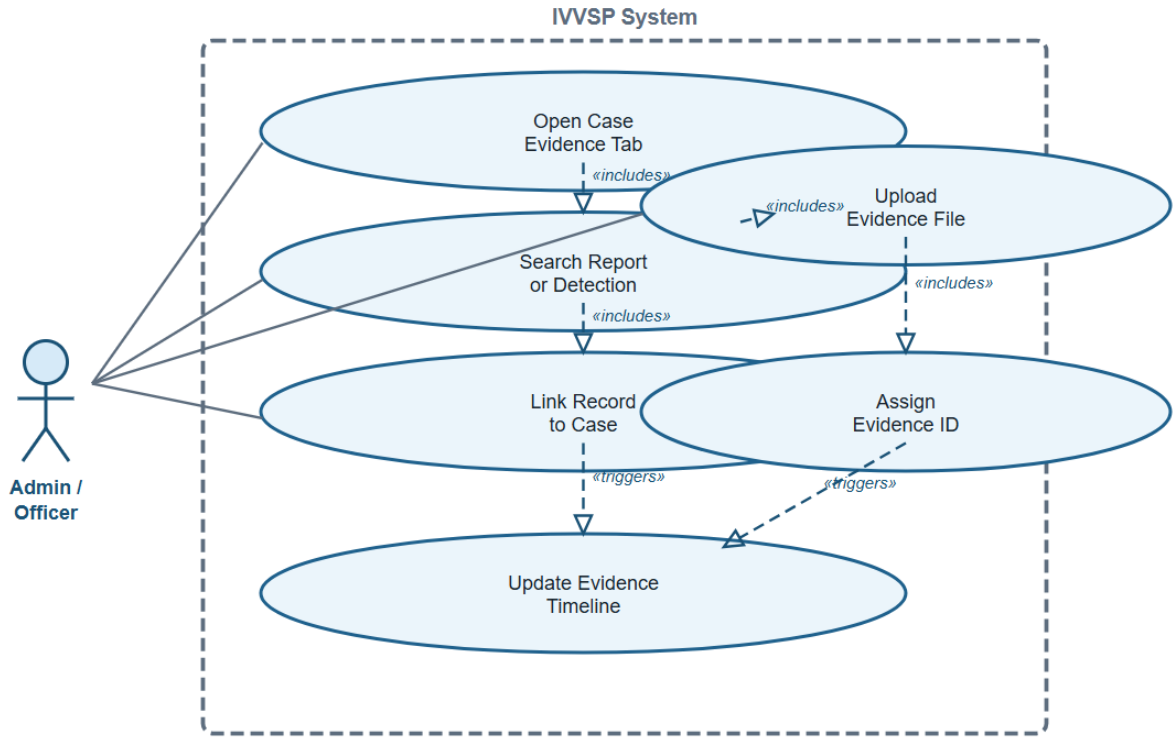


Figure 7: Use Case Diagram — Link Reports & Evidence with System Data

Table 3.6 presents the structured use case description for UC-06, specifying the main flow, alternative flows, and exception handling for this interaction.

Table 29: Use Case Description — Link Reports & Evidence with System Data

Field	Description
Use Case ID	UC-06
Use Case Name	Link Reports & Evidence with System Data
Actor(s)	Zone Administrator, Investigation Officer
Description	Allow administrators to link citizen reports and multimedia evidence to AI for detection of events and active investigation cases, forming a unified evidential record.
Pre-condition	An active case exists. Citizen reports or detection events exist in the database. Users have minimum level access.
Post-condition	Evidence is linked to the case and stored securely. The case evidence timeline is updated.
Main Flow	1. Admin opens an active case and navigates to the 'Evidence' tab. 2. Admin searches for a citizen report or detection event to link. 3. System displays the record with metadata. 4. Admin confirms the link; system associates the record with the case. 5. Admin optionally uploads additional evidence files. 6. System stores files, assigns Evidence IDs, and updates the timeline.
Alternative Flow	Multiple reports or detection events may be linked to the same case in a single session.
Exception Flow	Uploading unsupported file formats (not JPEG, PNG, MP4, MP3, PDF) triggers a validation error.
Related FR	FR-06
Priority	Medium

3.4.7 UC-07: Define and Manage Privacy Settings

Figure 3.8 illustrates the use case diagram for UC-07 (Define and Manage Privacy Settings), corresponding to functional requirement FR-07. The diagram shows all participating actors and their relationships to the use case, including included and extended sub-cases.

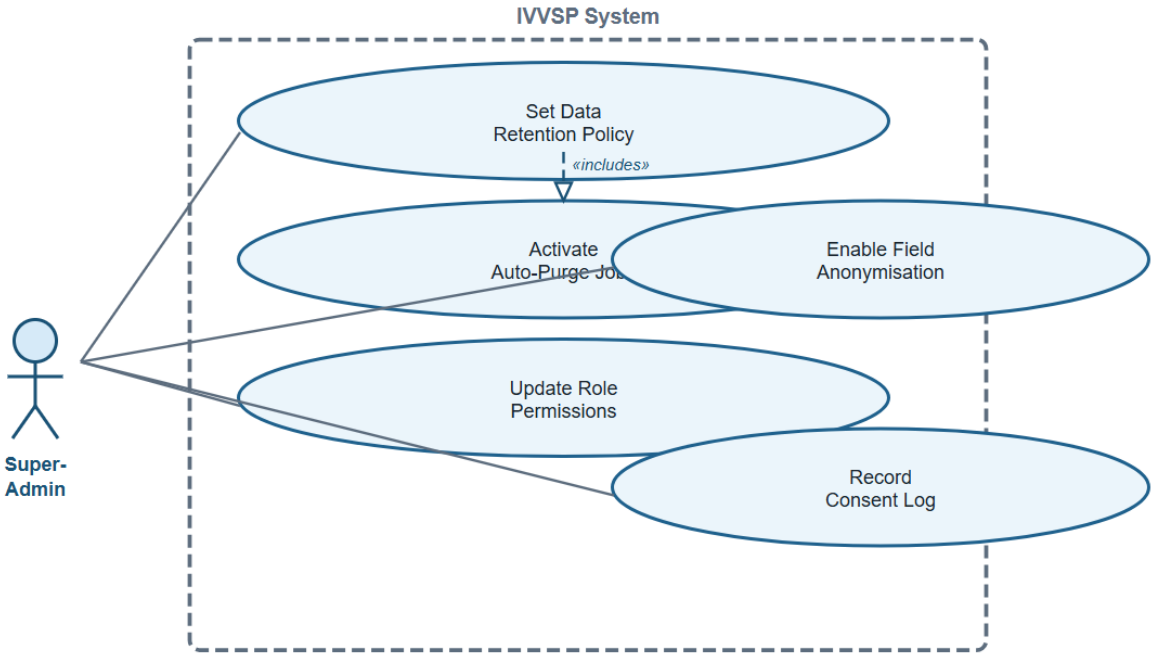


Figure 8: Use Case Diagram — Define and Manage Privacy Settings

Table 3.7 presents the structured use case description for UC-07, specifying the main flow, alternative flows, and exception handling for this interaction.

Table 30: Use Case Description — Define and Manage Privacy Settings

Field	Description
Use Case ID	UC-07
Use Case Name	Define and Manage Privacy Settings
Actor(s)	Super-Administrator
Description	Enables the super-administrator to configure data retention policies, role-based permission matrices, field anonymization, and citizen consent records to ensure regulatory compliance.
Pre-condition	Super-admin is authenticated. The system has active data in the database.
Post-condition	Privacy configuration is saved. Automated purging jobs are scheduled. Role permissions take effect on the next user's login.
Main Flow	1. Super-admin navigates 'Privacy & Compliance'. 2. Admin sets retention periods per data category. 3. System schedules automated purge jobs. 4. Admin updates the role-permission matrix. 5. Admin enables anonymization for selected data fields. 6. All changes are recorded in the audit trail.
Alternative Flow	Field anonymization may be applied selectively per export operation rather than system wide.
Exception Flow	If a retention period conflicts with an active legal case to hold, the system prompts the admin and prevents purging of affected records.
Related FR	FR-07
Priority	High

3.4.8 UC-08: Maintain Activity Logs for Accountability

Figure 3.9 illustrates the use case diagram for UC-08 (Maintain Activity Logs for Accountability), corresponding to functional requirement FR-08. The diagram shows all participating actors and their relationships to the use case, including included and extended sub-cases.

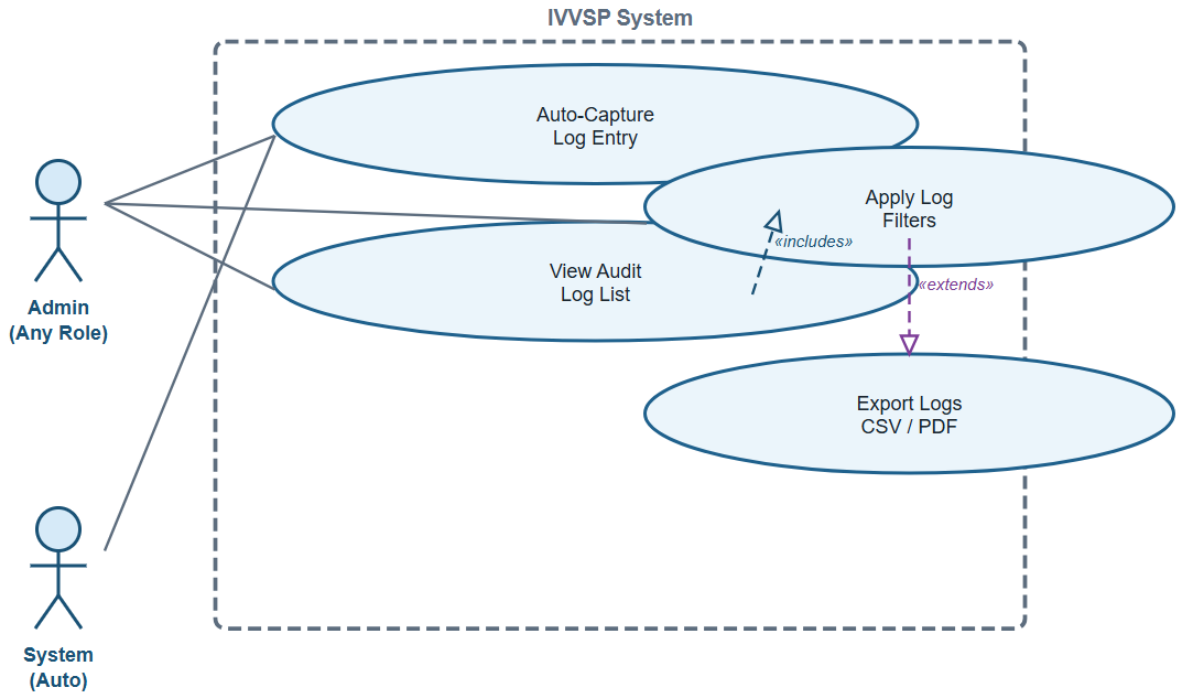


Figure 9: Use Case Diagram — Maintain Activity Logs for Accountability

Table 3.8 presents the structured use case description for UC-08, specifying the main flow, alternative flows, and exception handling for this interaction.

Table 31: Use Case Description — Maintain Activity Logs for Accountability

Field	Description
Use Case ID	UC-08
Use Case Name	Maintain Activity Logs for Accountability
Actor(s)	System (Auto), Zone Administrator, Super-Administrator
Description	The system automatically generates immutable audit log entries for all significant actions. Admins may search, filter, and export logs for compliance and legal review.
Pre-condition	The system is operational. An authorized admin is logged in to access logs.
Post-condition	Log entries are preserved and immutable. Filtered or exported logs are delivered to the admin.
Main Flow	1. System auto-captures a log entry for each significant event. 2. Each entry records actor, role, action, resource, timestamp, and IP. 3. Logs are stored in an append-only MongoDB collection. 4. Admin navigates to 'Audit Logs' and applies filters. 5. System returns paginated, sortable results. 6. Admin exports selected entries as CSV or PDF.
Alternative Flow	Super-admins may view logs for all users; zone admins may only view logs within their zone.
Exception Flow	Export failures due to large result sets trigger a background job that delivers the export via email notification.
Related FR	FR-08
Priority	High

3.4.9 UC-09: Manage Watchlists for Flagged Vehicles

Figure 3.10 illustrates the use case diagram for UC-09 (Manage Watchlists for Flagged Vehicles), corresponding to functional requirement FR-09. The diagram shows all participating actors and their relationships to the use case, including included and extended sub-cases.

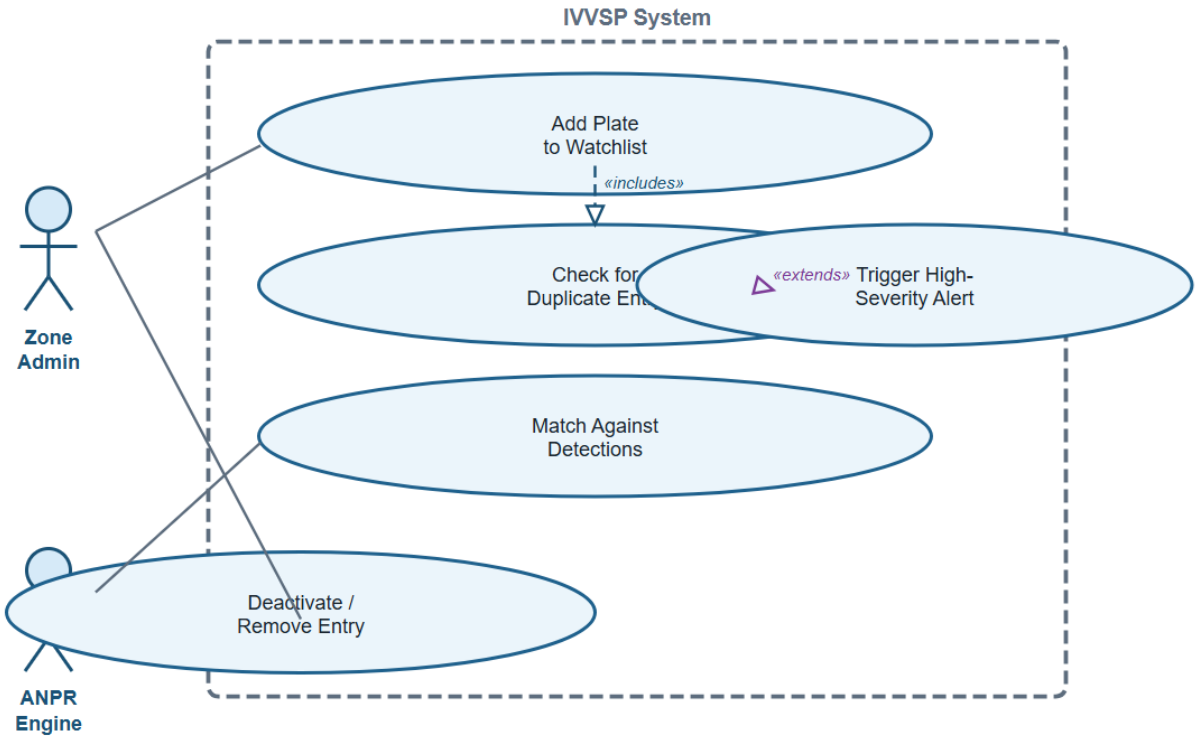


Figure 10: Use Case Diagram — Manage Watchlists for Flagged Vehicles

Table 3.9 presents the structured use case description for UC-09, specifying the main flow, alternative flows, and exception handling for this interaction.

Table 32: Use Case Description — Manage Watchlists for Flagged Vehicles

Field	Description
Use Case ID	UC-09
Use Case Name	Manage Watchlists for Flagged Vehicles
Actor(s)	Zone Administrator, Super-Administrator, ANPR Engine (System)
Description	Allows administrators to add, monitor, and manage watchlist entries of number plates. Immediate high-severity alerts are raised when a watch listed plate is detected.
Pre-condition	Admin is authenticated. The ANPR engine is active and monitors camera feeds.
Post-condition	Watchlist entry is saved and active. ANPR engine includes the plate in real-time matching. Alerts are generated for detection.
Main Flow	1. Admin navigates to 'Watchlist Management. 2. Admin enters plate, selects reason, and optionally sets expiry. 3. System checks for duplicates. 4. Entry saved and flagged in the ANPR engine. 5. On detection, a 'High' severity alert is raised. 6. Alert notification is pushed to the assigned officer and zone admin.
Alternative Flow	Watchlist entries may be bulk imported via a CSV file to upload.
Exception Flow	If a plate is already on the watchlist, the system prevents duplicate entry and notifies the admin.
Related FR	FR-09
Priority	High

3.4.10 UC-10: Check Vehicle Status Using Number Plate

Figure 3.11 illustrates the use case diagram for UC-10 (Check Vehicle Status Using Number Plate), corresponding to functional requirement FR-10. The diagram shows all participating actors and their relationships to the use case, including included and extended sub-cases.

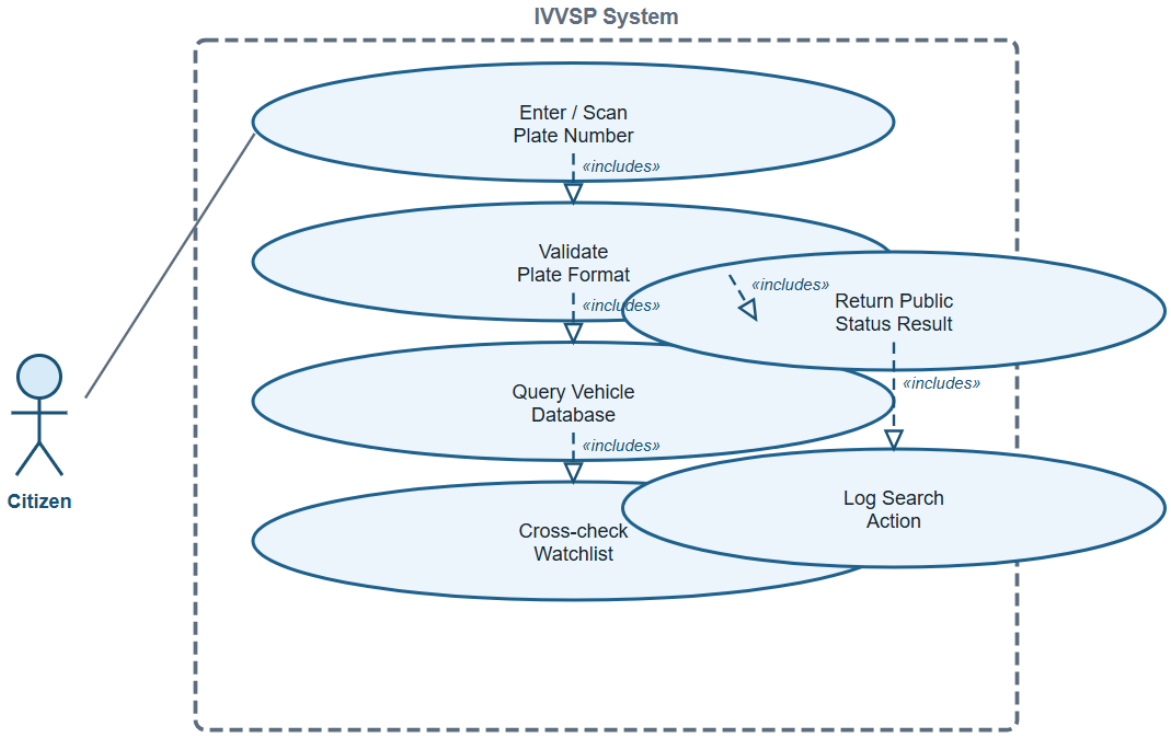


Figure 11: Use Case Diagram — Check Vehicle Status Using Number Plate

Table 3.10 presents the structured use case description for UC-10, specifying the main flow, alternative flows, and exception handling for this interaction.

Table 33: Use Case Description — Check Vehicle Status Using Number Plate

Field	Description
Use Case ID	UC-10
Use Case Name	Check Vehicle Status Using Number Plate
Actor(s)	Citizen
Description	Allows authenticated citizens to look up a vehicle's public registration status by entering or scanning its number plate. Only publicly permissible information is returned.
Pre-condition	Citizens are authenticated. The vehicle database is populated.
Post-condition	A color-coded status result is displayed. The search is logged for audit.
Main Flow	1. Citizen navigates to 'Vehicle Status Check'. 2. Citizen types or scans the plate number. 3. System validates the plate format. 4. Backend queries the vehicle database and watchlist. 5. System returns a public-safe response. 6. App displays the result with a color-coded indicator. 7. Search is logged.
Alternative Flow	Citizens may use the device camera to scan a physical plate, with the OCR result auto-populated in the input field.
Exception Flow	If the plate format is invalid, an error is displayed and no query is made. If the vehicle is not found, a 'Not Registered' status is shown.
Related FR	FR-10
Priority	High

3.4.11 UC-11: View Limited Safety-Related Information

Figure 3.12 illustrates the use case diagram for UC-11 (View Limited Safety-Related Information), corresponding to functional requirement FR-11. The diagram shows all participating actors and their relationships to the use case, including included and extended sub-cases.

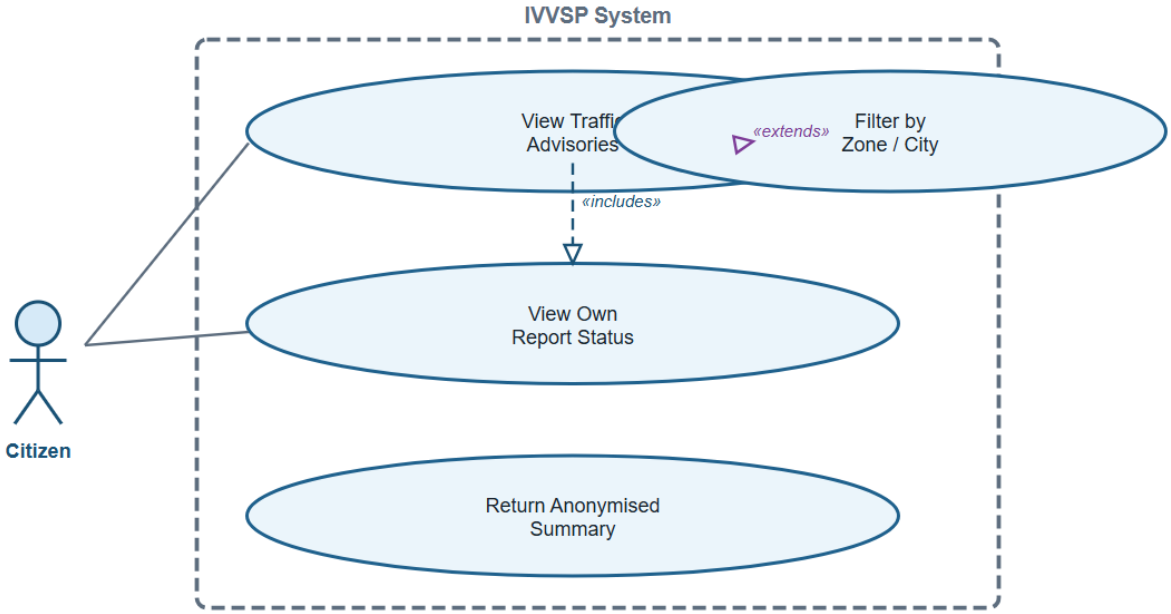


Figure 12: Use Case Diagram — View Limited Safety-Related Information

Table 3.11 presents the structured use case description for UC-11, specifying the main flow, alternative flows, and exception handling for this interaction.

Table 34: Use Case Description — View Limited Safety-Related Information

Field	Description
Use Case ID	UC-11
Use Case Name	View Limited Safety-Related Information
Actor(s)	Citizen
Description	Provides authenticated citizens with access to publicly permissible safety content including traffic advisories, anonymized incident summaries, and their own report statuses.
Pre-condition	Citizens are authenticated. Admin has published at least one advisory.
Post-condition	Relevant safety information is displayed. Citizens' own report statuses are accurately reflected.
Main Flow	1. Citizen navigates to 'Safety Information. 2. System retrieves and displays active public advisories. 3. Citizen optionally filters by zone or city. 4. Citizen selects 'My Reports' to view their submission history. 5. System returns only the citizen's own records with status badges.
Alternative Flow	If no advisories are published, the system shows a 'No Active Advisories' message.
Exception Flow	Cross-citizen data access is blocked at the API level by token validation.
Related FR	FR-11
Priority	Medium

3.4.12 UC-12: Report Suspicious Vehicles with Evidence

Figure 3.13 illustrates the use case diagram for UC-12 (Report Suspicious Vehicles with Evidence), corresponding to functional requirement FR-12. The diagram shows all participating actors and their relationships to the use case, including included and extended sub-cases.

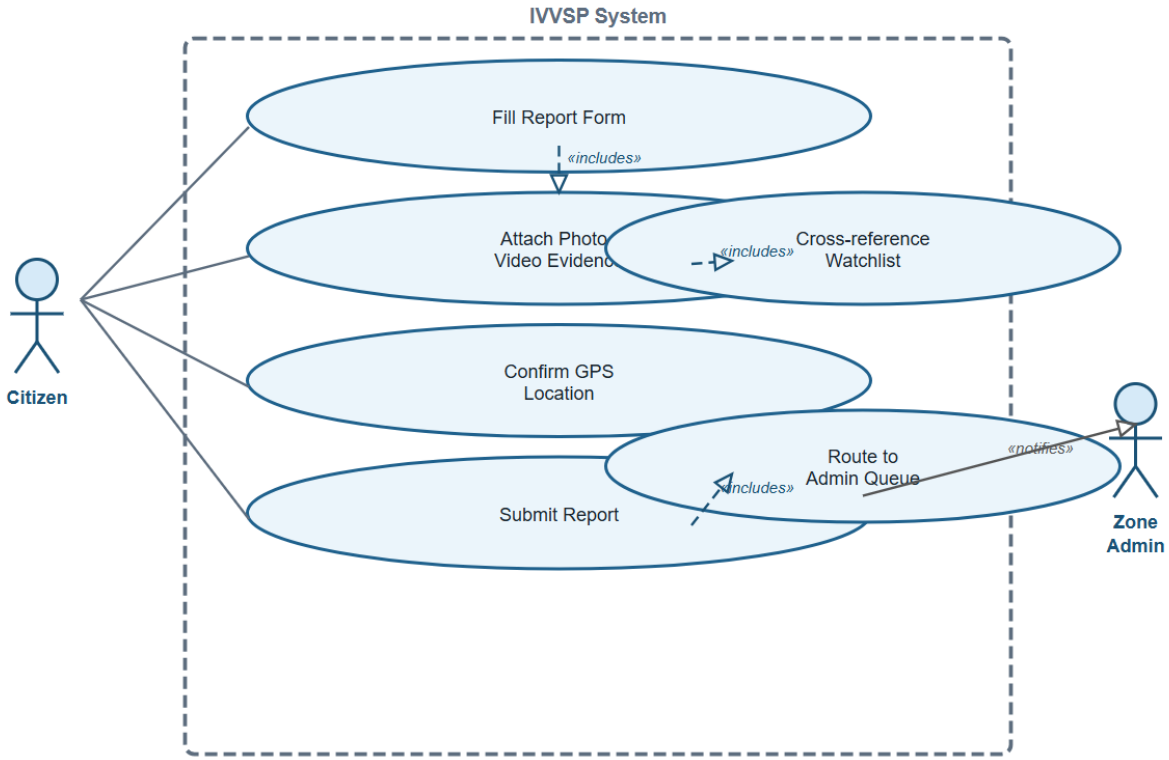


Figure 13: Use Case Diagram — Report Suspicious Vehicles with Evidence

Table 3.12 presents the structured use case description for UC-12, specifying the main flow, alternative flows, and exception handling for this interaction.

Table 35: Use Case Description — Report Suspicious Vehicles with Evidence

Field	Description
Use Case ID	UC-12
Use Case Name	Report Suspicious Vehicles with Evidence
Actor(s)	Citizen, Zone Administrator
Description	Enables citizens to submit detailed reports of suspicious vehicular activity with multimedia evidence. Reports are automatically cross-referenced with surveillance data and routed to the zone administrator.
Pre-condition	Citizens are authenticated. GPS is available on the device.
Post-condition	Report is submitted with a unique ID. Admin is notified. The report is cross-referenced with a watchlist and detection history.
Main Flow	1. Citizen opens a 'Report Suspicious Vehicle'. 2. Citizens fill in vehicle descriptions and suspected activity. 3. Citizen attaches photo or video evidence. 4. Citizen confirms the GPS location. 5. System validates fields and uploads evidence. 6. Report is created and routed to the zone admin queue. 7. System cross-references the plate if provided. 8. Citizen receives confirmation with Report ID.
Alternative Flow	If GPS is unavailable, citizens may manually enter an address.
Exception Flow	If mandatory fields are missing, the system displays inline validation errors and prevents submission.
Related FR	FR-12
Priority	High

3.4.13 UC-13: Report Lost or Stolen Number Plates

Figure 3.14 illustrates the use of case diagrams for UC-13 (Report Lost or Stolen Number Plates), corresponding to functional requirement FR-13. The diagram shows all participating actors and their relationships to the use case, including included and extended sub-cases.

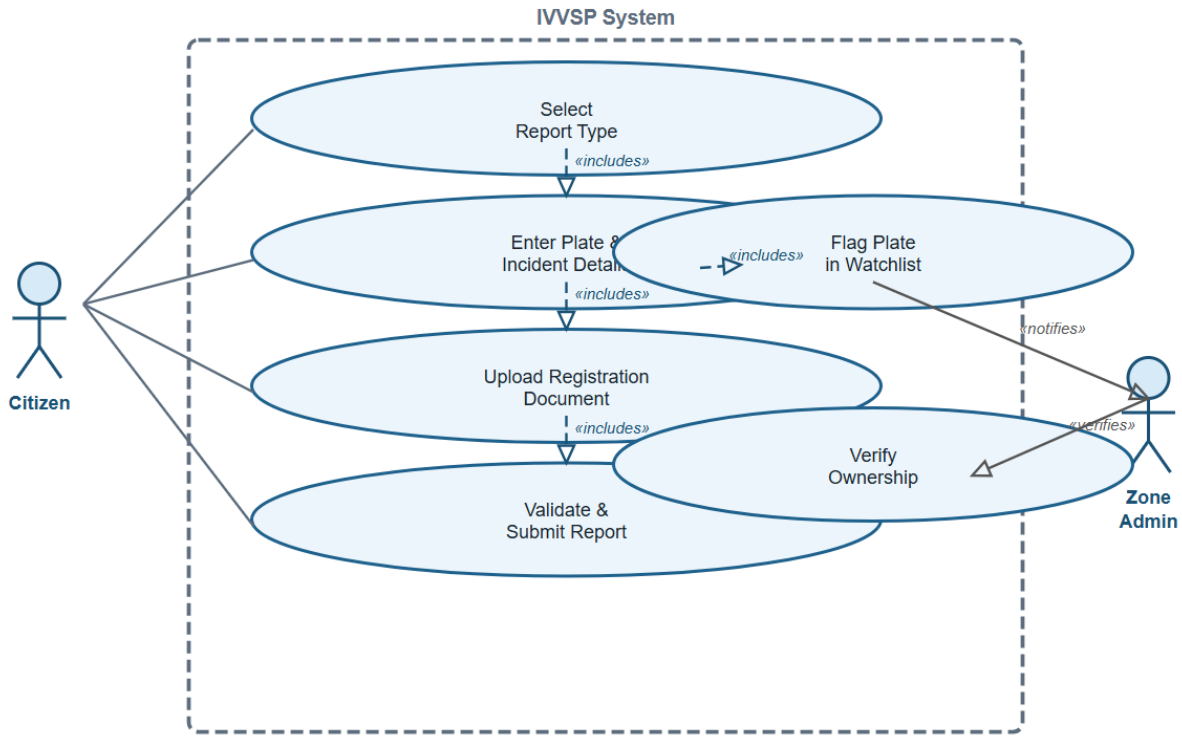


Figure 14: Use Case Diagram — Report Lost or Stolen Number Plates

Table 3.13 presents the structured use case description for UC-13, specifying the main flow, alternative flows, and exception handling for this interaction.

Table 36: Use Case Description — Report Lost or Stolen Number Plates

Field	Description
Use Case ID	UC-13
Use Case Name	Report Lost or Stolen Number Plates
Actor(s)	Citizen, Zone Administrator
Description	Allow citizens to formally report a lost or stolen number plate. The plate is auto flagged in the watchlist to prevent fraudulent use and alert authorities if detected.
Pre-condition	Citizens are authenticated. Citizens possess a vehicle registration document.
Post-condition	Report is submitted with a unique ID. Plate is flagged in the watchlist pending admin verification.
Main Flow	1. Citizen navigates to 'Report Lost/Stolen Plate'. 2. Citizen selects report type and enters the plate. 3. Citizen provides date, location, and uploads the registration document. 4. System validates format and checks existing records. 5. Report submitted; plate added to watchlist. 6. Zone admin notified to verify ownership. 7. Citizens receive confirmation.
Alternative Flow	If the plate has already been reported, the system notifies the citizen and does not create a duplicate entry.
Exception Flow	If the uploaded document is rejected as invalid format, the system prompts the citizen to re-upload.
Related FR	FR-13
Priority	High

3.4.14 UC-14: Register Personal Vehicle for Monitoring

Figure 3.15 illustrates the use case diagram for UC-14 (Register Personal Vehicle for Monitoring), corresponding to functional requirement FR-14. The diagram shows all participating actors and their relationships to the use case, including included and extended sub-cases.

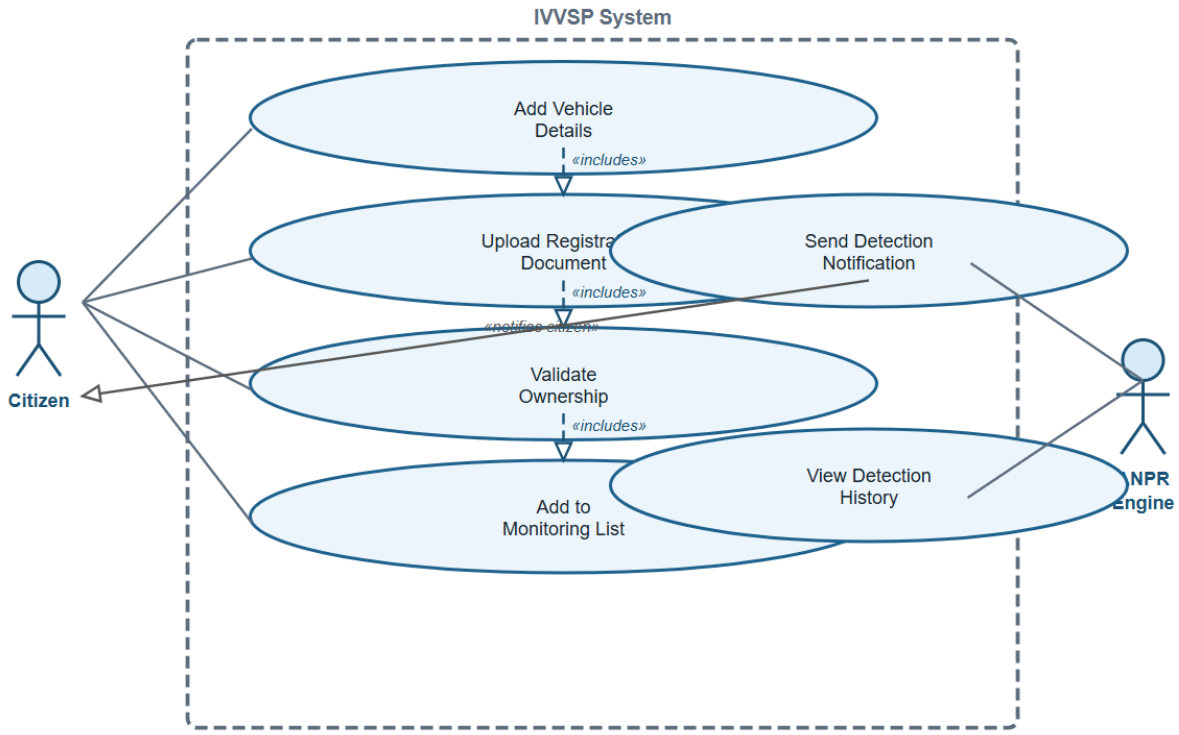


Figure 15: Use Case Diagram — Register Personal Vehicle for Monitoring

Table 3.14 presents the structured use case description for UC-14, specifying the main flow, alternative flows, and exception handling for this interaction.

Table 37: Use Case Description — Register Personal Vehicle for Monitoring

Field	Description
Use Case ID	UC-14
Use Case Name	Register Personal Vehicle for Monitoring
Actor(s)	Citizen, ANPR Engine (System)
Description	Allows citizens to voluntarily register their personal vehicles for active monitoring. Notifications are sent whenever the vehicle's plate is detected by any surveillance camera.
Pre-condition	Citizens are authenticated. Citizens hold a valid vehicle registration document.
Post-condition	Vehicles are registered and added to the personalized monitoring list. The ANPR engine is updated to trigger notifications on detection.
Main Flow	1. Citizen navigates to 'My Vehicles' and selects 'Add Vehicle'. 2. Citizen enters plate, make, model, color, and year. 3. Citizen uploads the registration document. 4. System validates and checks for duplicates. 5. Vehicles are added to the monitoring profile. 6. The ANPR engine is updated. 7. Notifications are sent on each detection. 8. Citizens may view detection history.
Alternative Flow	Citizens may update notification preferences (Push, SMS, or Both) at any time from the 'My Vehicles' section.
Exception Flow	If document validation fails, the vehicle is not registered, and the citizen is prompted to resubmit.
Related FR	FR-14
Priority	Medium

3.4.15 UC-15: Submit Public Complaints (Traffic Issues)

Figure 3.16 illustrates the use case diagram for UC-15 (Submit Public Complaints (Traffic Issues)), corresponding to functional requirement FR-15. The diagram shows all participating actors and their relationships to the use case, including included and extended sub-cases.

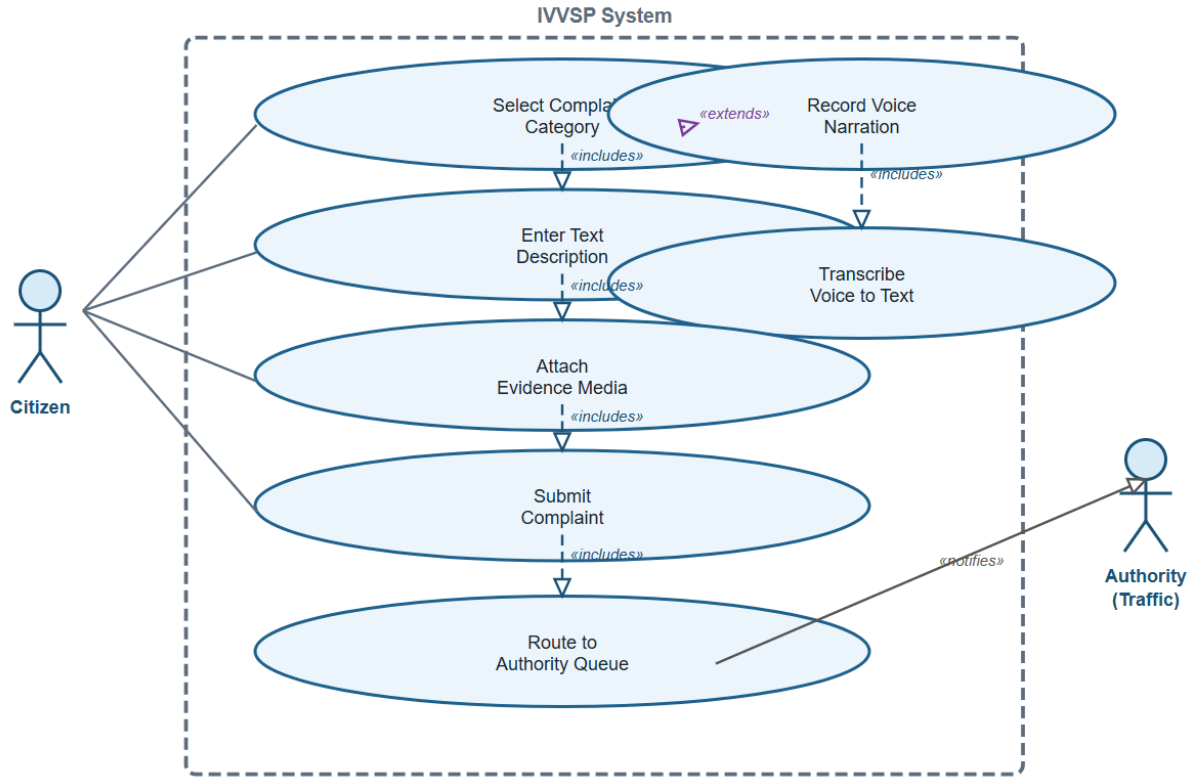


Figure 16: Use Case Diagram — Submit Public Complaints (Traffic Issues)

Table 3.15 presents the structured use case description for UC-15, specifying the main flow, alternative flows, and exception handling for this interaction.

Table 38: Use Case Description — Submit Public Complaints (Traffic Issues)

Field	Description
Use Case ID	UC-15
Use Case Name	Submit Public Complaints (Traffic Issues)
Actor(s)	Citizen, Authority (Traffic Police)
Description	Provides citizens with a structured, voice-enabled channel to submit traffic complaints. Complaints are categorized and automatically routed to the responsible authority.
Pre-condition	Citizens are authenticated. Device has microphone access (for voice input). GPS is available.
Post-condition	Complaint is submitted with a unique ID. Authority is notified. Citizens receive status updates.
Main Flow	1. Citizen navigates to 'Submit Complaint'. 2. Citizens select the complaint category. 3. Citizen types of description or records of voice narration. 4. If voice: system transcribes and populates description field. 5. Citizen optionally attaches evidence and enters the offending plate. 6. Citizen confirms the GPS location. 7. System submits, creates Complaint ID, and routes to authority. 8. Citizens receive confirmation.
Alternative Flow	Voice narration may be recorded in either Urdu or Pakistani English.
Exception Flow	If voice transcription accuracy is below the accepted threshold, the citizen is prompted to review and correct the transcribed text before submission.
Related FR	FR-15
Priority	Medium

3.4.16 UC-16: Detect and Read Number Plates (ANPR Engine)

Figure 3.17 illustrates the use case diagram for UC-16 (Detect and Read Number Plates (ANPR Engine)), corresponding to functional requirement FR-16. The diagram shows all participating actors and their relationships to the use case, including included and extended sub-cases.

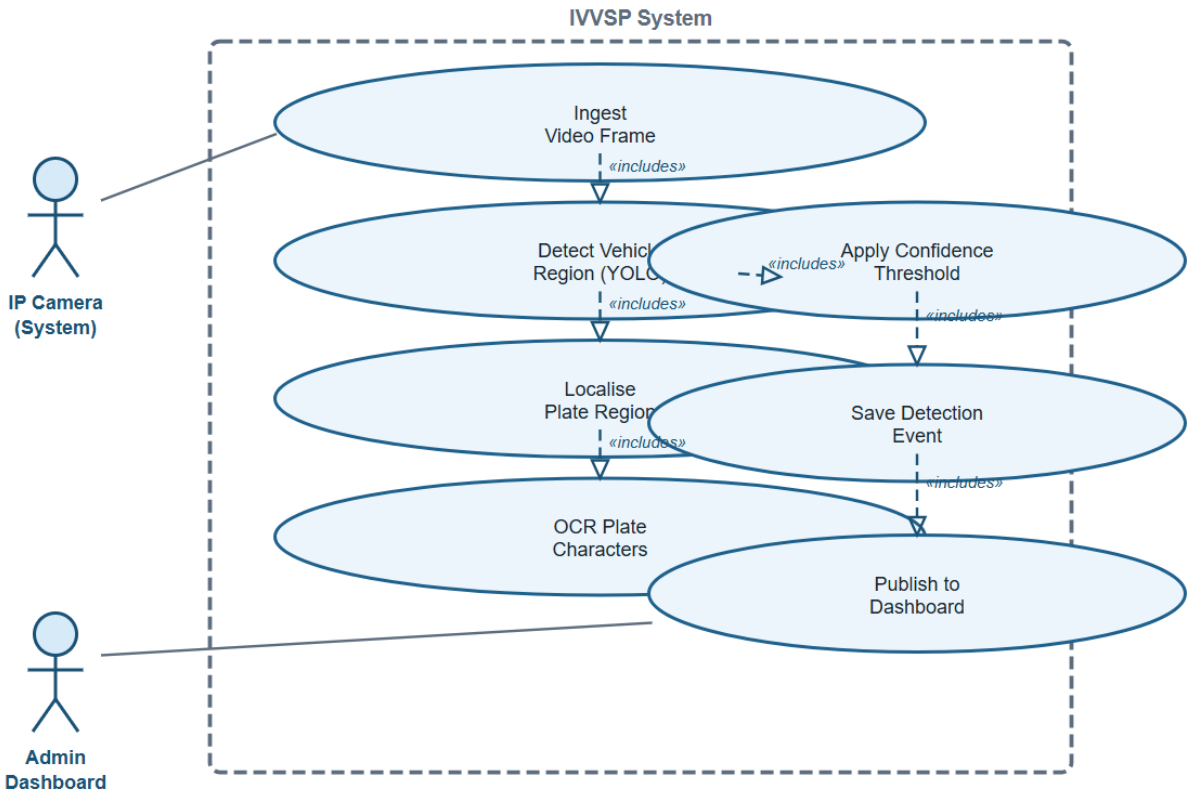


Figure 17: Use Case Diagram — Detect and Read Number Plates (ANPR Engine)

Table 3.16 presents the structured use case description for UC-16, specifying the main flow, alternative flows, and exception handling for this interaction.

Table 39: Use Case Description — Detect and Read Number Plates (ANPR Engine)

Field	Description
Use Case ID	UC-16
Use Case Name	Detect and Read Number Plates (ANPR Engine)
Actor(s)	IP Camera (System), AI Processing Engine (System), Admin Dashboard
Description	The AI-powered ANPR engine continuously processes video frames to detect OCR vehicle number plates. It supports adverse conditions and all Pakistani plate formats.
Pre-condition	Cameras are streaming. The YOLO detection and OCR models are loaded. Confidence threshold is configured.
Post-condition	Detected plate strings with confidence scores are saved to the database and pushed to the dashboard. Watchlist matches trigger alerts.
Main Flow	1. Camera streams frames to the AI layer. 2. The YOLO model identifies vehicle regions. 3. Plate localization model extracts the plate region. 4. The OCR model reads the characters and produces a confidence score. 5. Detections above the threshold are accepted. 6. Detection events are saved to MongoDB. 7. Events are published on the dashboard via Socket.io and cross-referenced with the watchlist.
Alternative Flow	Frames with no vehicles are discarded without generating detection records.
Exception Flow	If OCR confidence falls below threshold, the detection is discarded and logged as a low-confidence event for review.
Related FR	FR-16
Priority	High

3.4.17 UC-17: Identify Vehicle Attributes (Color & Type)

Figure 3.18 illustrates the use case diagram for UC-17 (Identify Vehicle Attributes (Color & Type)), corresponding to functional requirement FR-17. The diagram shows all participating actors and their relationships to the use case, including included and extended sub-cases.

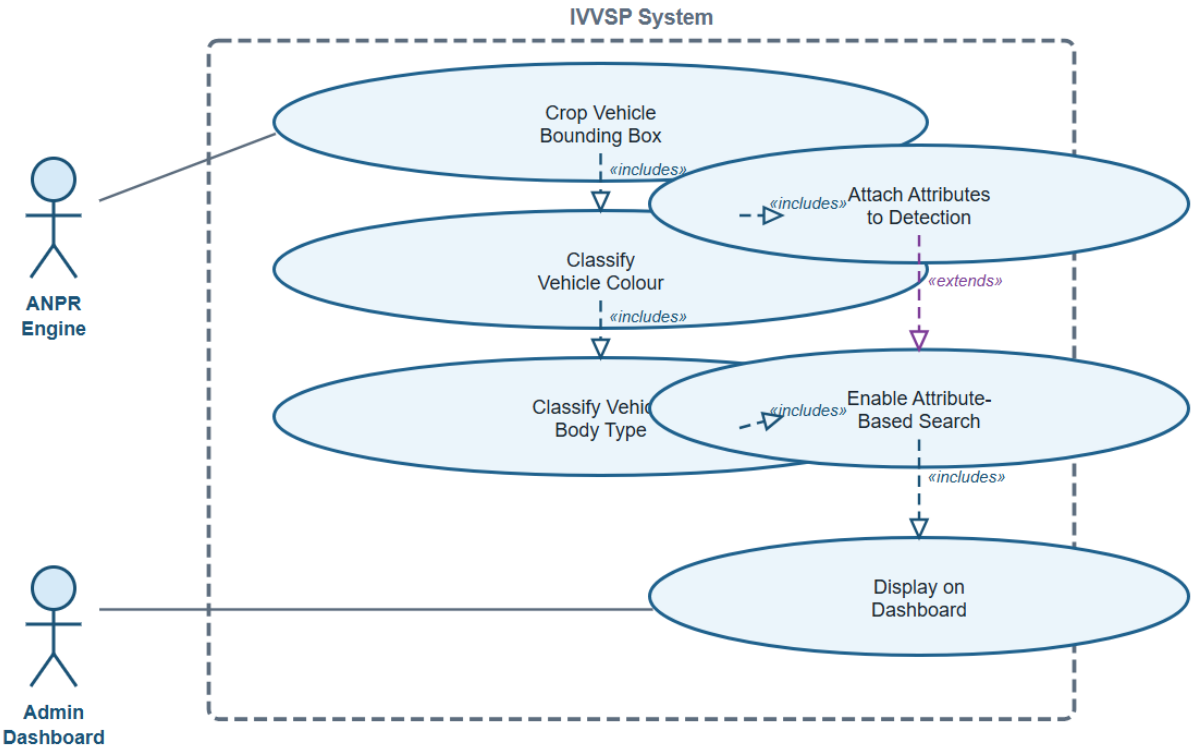


Figure 18: Use Case Diagram — Identify Vehicle Attributes (Color & Type)

Table 3.17 presents the structured use case description for UC-17, specifying the main flow, alternative flows, and exception handling for this interaction.

Table 40: Use Case Description — Identify Vehicle Attributes (Color & Type)

Field	Description
Use Case ID	UC-17
Use Case Name	Identify Vehicle Attributes (Color & Type)
Actor(s)	ANPR Engine (System), AI Attribute CNN (System), Admin Dashboard
Description	A CNN classifies vehicle color and body type from each detected vehicle image, supplementing ANPR results to maintain identification when plates are unreadable.
Pre-condition	Vehicle bounding box is available from ANPR detection. The attribute classification model is loaded.
Post-condition	Color and type of attributes are stored in the detection record and displayed on the dashboard.
Main Flow	1. ANPR engine provides vehicle bound box crops. 2. Crop is passed to the attribute of CNN. 3. Color classification head predicts the color label and confidence. 4. Body type head predicts the type of label and confidence. 5. Both predictions are appended to the detection record. 6. When ANPR fails, attributing data supports manual or automated vehicle search. 7. Attributes are displayed on the admin dashboard.
Alternative Flow	Where both ANPR and attribute recognition succeed, all fields are stored together in a single detection document.
Exception Flow	If the vehicle's crop region is too small or blurred, the attribute CNN returns a low-confidence result, and the attribute fields are flagged as uncertain.
Related FR	FR-17
Priority	High

3.4.18 UC-18: Match Vehicles via Re-Identification (Re-ID Engine)

Figure 3.19 illustrates the use case diagram for UC-18 (Match Vehicles via Re-Identification (Re-ID Engine)), corresponding to functional requirement FR-18. The diagram shows all participating actors and their relationships to the use case, including included and extended sub-cases.

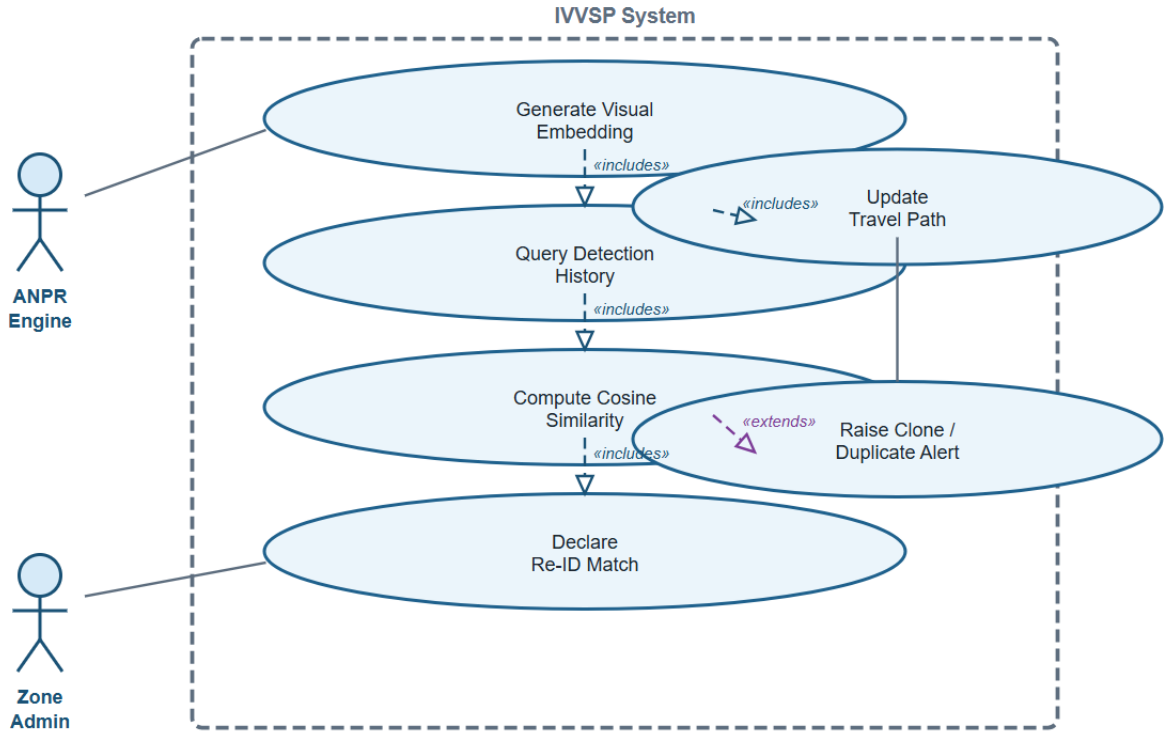


Figure 19: Use Case Diagram — Match Vehicles via Re-Identification (Re-ID Engine)

Table 3.18 presents the structured use case description for UC-18, specifying the main flow, alternative flows, and exception handling for this interaction.

Table 41: Use Case Description — Match Vehicles via Re-Identification (Re-ID Engine)

Field	Description
Use Case ID	UC-18
Use Case Name	Match Vehicles via Re-Identification (Re-ID Engine)
Actor(s)	ANPR Engine (System), Re-ID Engine (System), Zone Administrator
Description	The Re-ID engine matches vehicles across multiple camera views using deep metric learning visual embeddings, enabling cross-camera tracking and clone plate detection.
Pre-condition	At least two detection events exist in the system within the configured time window. The Re-ID model is loaded.
Post-condition	Vehicle travel path is updated. Clone/Duplicate alerts are raised for impossible simultaneous detections.
Main Flow	1. The ANPR engine triggers a new detection event. 2. Re-ID extractor generates visual embedding. 3. System queries detection history for candidate embeddings. 4. Cosine similarity is computed against each candidate. 5. A match is declared if similarity exceeds the threshold. 6. Travel path is updated with the new sighting. 7. If the same plate appears in two distant zones simultaneously, a Clone/Duplicate alert is raised.
Alternative Flow	The configurable time window allows administrators to adjust the matching window from 5 minutes to 24 hours depending on the zone coverage area.
Exception Flow	If no candidate embedding exists within the time window, the detection is recorded as a new, unmatched vehicle entry.
Related FR	FR-18

3.4.19 UC-19: Offline Data Storage & Synchronization

Figure 3.20 illustrates the use case diagram for UC-19 (Offline Data Storage & Synchronization), corresponding to functional requirement FR-19. The diagram shows all participating actors and their relationships to the use case, including included and extended sub-cases.

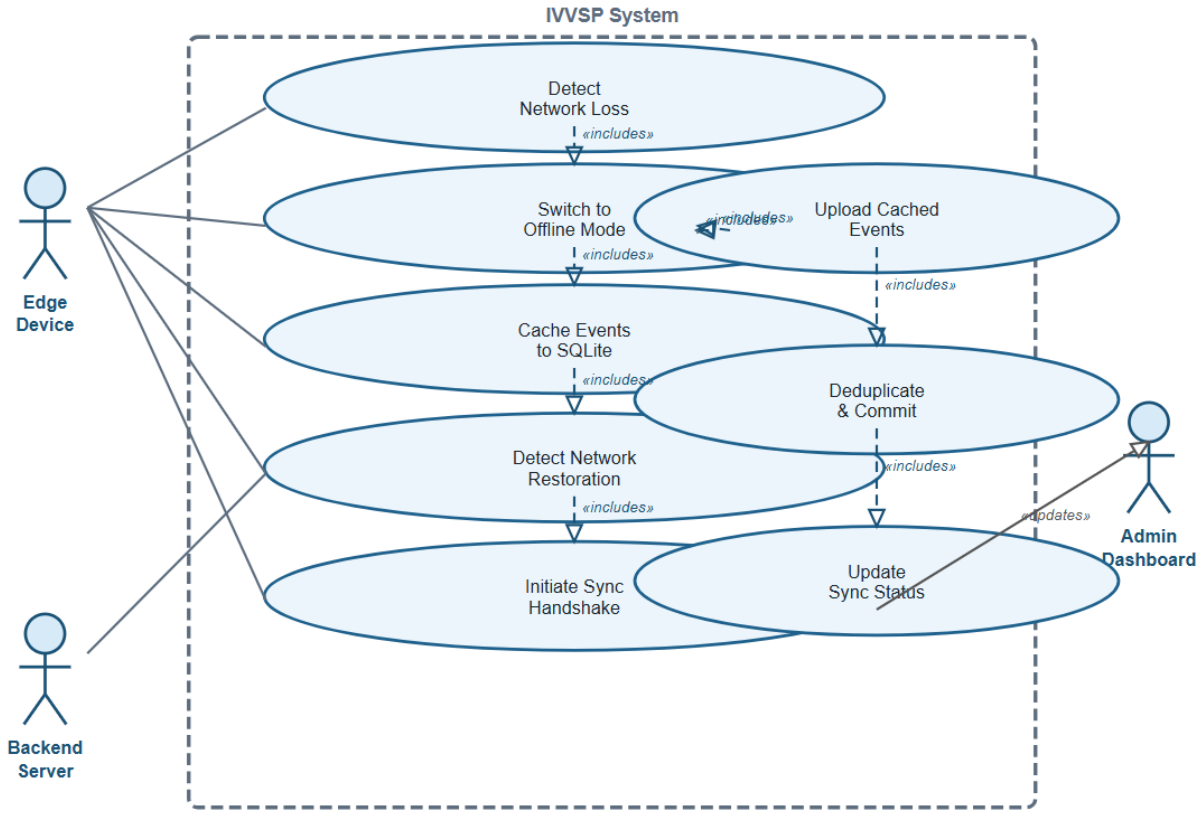


Figure 20: Use Case Diagram — Offline Data Storage & Synchronization

Table 3.19 presents the structured use case description for UC-19, specifying the main flow, alternative flows, and exception handling for this interaction.

Table 42: Use Case Description — Offline Data Storage & Synchronization

Field	Description
Use Case ID	UC-19
Use Case Name	Offline Data Storage & Synchronization
Actor(s)	Edge Device (System), Backend Server (System), Admin Dashboard
Description	Edge AI devices continue to detect and cache events locally during network outages, then synchronize automatically with the central MongoDB database upon reconnection.
Pre-condition	Edge devices have active local models loaded. Network connectivity has been lost.
Post-condition	All cached events are synchronized to MongoDB without data loss. The sync summary is logged. Device status returns to 'Online'.
Main Flow	1. Edge device detects network loss and switches to offline mode. 2. ANPR and attribute models continue running events written to SQLite cache. 3. The dashboard shows the device as 'Offline – Caching'. 4. On network restoration, edge device initiates sync handshake. 5. Cached events are transmitted to the backend. 6. Backend deduplicates and commits to MongoDB. 7. Local cache is cleared; device status reverts to 'Online'. 8. A sync summary is recorded in the audit log.
Alternative Flow	If partial sync fails in mid-transmission, the system retries the last successful record on the next connection.
Exception Flow	If the local cache reaches storage capacity during an extended outage, the oldest events are archived to secondary storage, and the admin is alerted.
Related FR	FR-19
Priority	High

3.5 Chapter Summary

This chapter has presented the complete use case analysis for the IVVSP platform. The aggregated system uses case diagrams in Section 3.3 established in the overall interaction landscape, identifying five actor classes and 19 use cases spanning administrative, citizen, and AI-driven functional domains. Section 3.4 presented individual use case diagrams and structured description tables for each of the 19 use cases, covering actors, pre-conditions, post-conditions, main flows, alternative flows, and exception handling.

Chapter 4 proceeds with the system design, translating the use case requirements into architectural, structural, and behavioral design artefacts including the System Design and Architecture Diagrams.

CHAPTER # 4

Chapter 4: System Design

4.1 Introduction

This chapter presents the complete system design for the Intelligent Vehicle Verification, Surveillance, and Public Safety Platform using AI (IVVSP). The design artefacts translate the functional and non-functional requirements specified in Chapter 2 and the use of case analysis of Chapter 3 into concrete architectural, structural, and behavioral specifications that govern implementation. The chapter is organized into eleven sections, each presenting a distinct design view of the system alongside a formal written description of the artefact and its key components.

4.2 System Architecture Diagram

The system architecture diagram defines the high-level structural decomposition of the IVVSP platform into its constituent tiers, components, and communication pathways. The architecture follows a five-tier layered pattern ensuring separation of concerns, scalability, and maintainability across the platform.

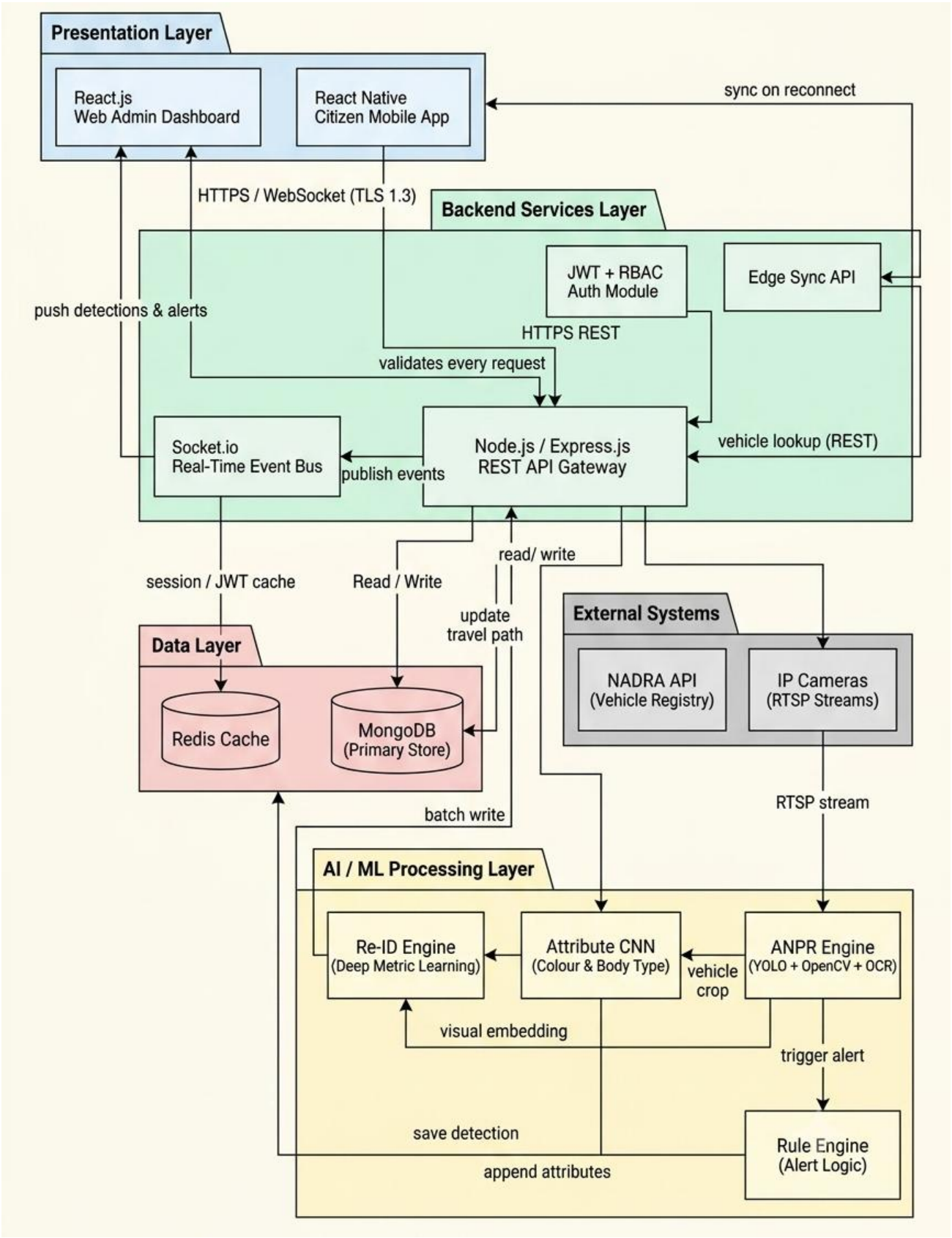


Figure 21: IVVSP System Architecture Diagram

The Presentation Layer consists of two client-facing interfaces: the React.js web administrative dashboard, which is the primary interface for law enforcement personnel delivering real-time detection feeds via WebSocket, and the React Native citizen mobile application, which provides vehicle status checks, report submission, and complaint filing across both Android and iOS platforms. All client-server communication is secured using HTTPS with TLS 1.2 or higher.

The Backend Services Layer is implemented using Node.js and Express.js, providing a RESTful API gateway serving both client interfaces. A JWT and RBAC Authentication Module enforces role-based access control on all endpoints. Socket.io drives the real-time bidirectional event bus that streams detection events and alerts to connected admin clients, while a dedicated Edge Sync API handles batch synchronization from offline edge devices.

The AI/ML Processing Layer ingests live RTSP streams from connected IP cameras and executes a sequential pipeline: the ANPR Engine (built on YOLO + OpenCV + TensorFlow OCR) reads number plates; the Attribute CNN classifies vehicle color and body type; and the Re-ID Engine computes deep metric learning embeddings for cross-camera vehicle matching. The Rule Engine applies configurable business logic including watchlist cross-referencing and clone plate temporal analysis and publishes alerts to the backend.

The Data Layer uses MongoDB as the single primary data store for all application data, deployed within Pakistan's jurisdictional boundaries to satisfy the data sovereignty constraint. All PII fields are encrypted at rest using AES-256. The Edge Layer enables continuous AI operation during network outages: camera-attached Jetson Nano devices run local ANPR and Attribute models, writing to a local SQLite cache until connectivity is restored and synchronization can occur.

4.3 Entity Relationship Diagram (ERD)

The Entity Relationship Diagram presents the complete data model for the IVVSP system, identifying 12 principal MongoDB collections and mapping their relationships, cardinalities, and application-level foreign key references. MongoDB's document model is used; foreign keys represent application-level references rather than enforced database constraints, consistent with NoSQL best practices.

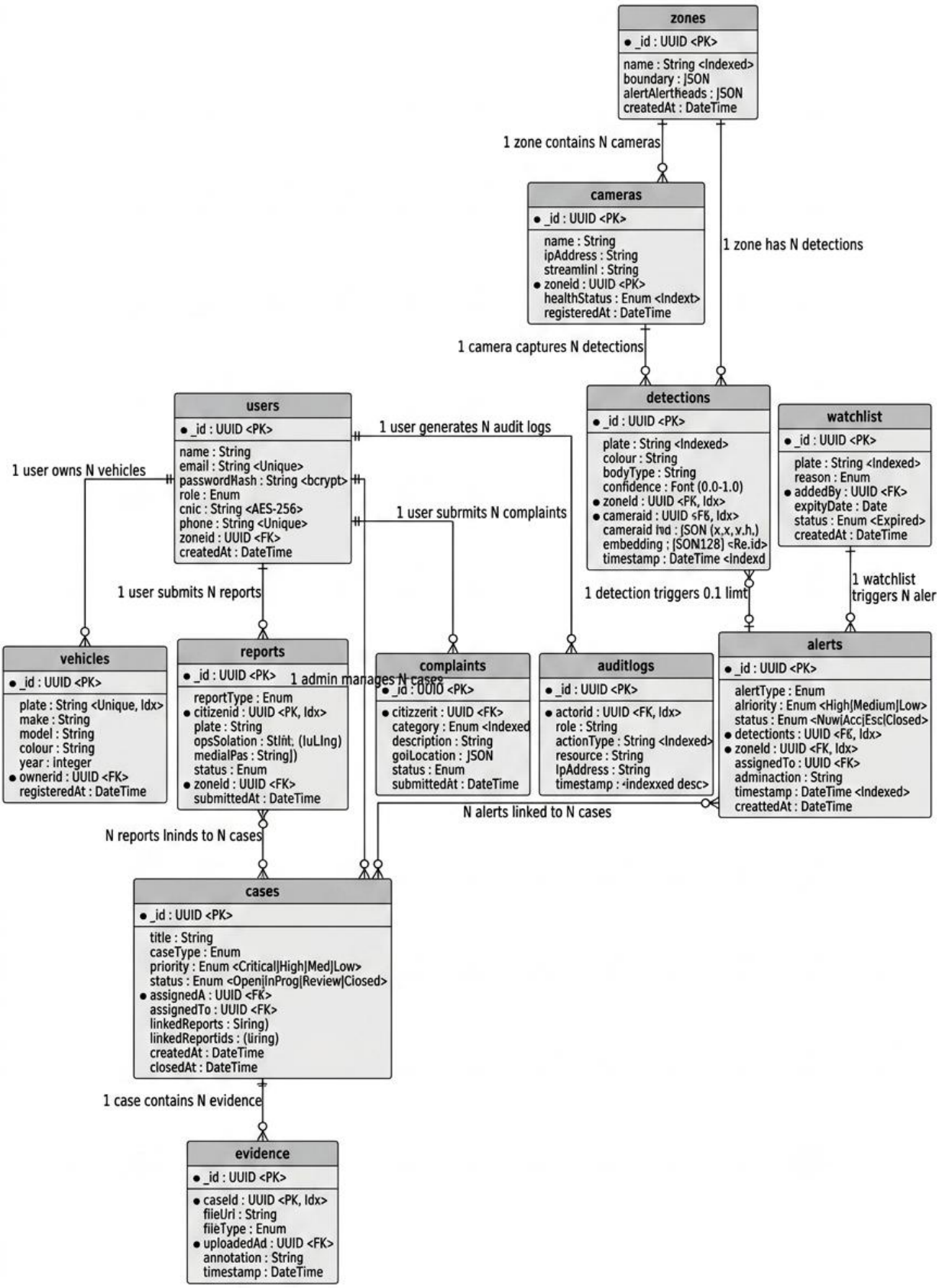


Figure 22: Entity Relationship Diagram — IVVSP MongoDB Data Model

The User collection is the central identity entity and carries an RBAC role discriminator (super_admin, zone_admin, officer, citizen). Citizens own Vehicle records and submit Report and Complaint instances. Each Detection record is produced by a specific Camera within a Zone, carries a 128-dimensional Re-ID embedding for cross-camera matching, and is the trigger source for Alert instances. Watchlist entries independently trigger high-severity Alerts when matched. Alerts and Reports are linked to Cases by administrative personnel; Cases aggregate Evidence records over their lifecycle. The Audit Log collection is append-only, capturing every significant system event with actor identity, action type, and timestamp.

The six primary collections defined in the Data Dictionary are presented in Tables 4.1 through 4.6. Table 4.7 presents the complete Logical Schema across all 12 collections.

Table 43: Data Dictionary — Users Collection

Field Name	Data Type	Constraint	Description
_id	String (UUID)	PK, Required	Auto-generated unique identifier for the user document.
name	String	Required, Max:100	Full name of the registered user (admin or citizen).
email	String	Required, Unique	Email address used for authentication and notifications.
passwordHash	String	Required	bcrypt-hashed password; cost factor ≥ 12. Plain text is never stored.
role	Enum	Required	User role: super_admin | zone_admin | officer | citizen.
cnic	String	Required (citizens)	13-digit CNIC number (AES-256 encrypted).
phone	String	Required, Unique	Mobile phone number used for OTP two-factor authentication.
zoneId	String (FK)	Nullable	References the Zone collection; null for citizens and super-admins.
createdAt	DateTime	Required, Auto	ISO 8601 timestamp of account creation.

Table 44: Data Dictionary — Detections Collection

Field Name	Data Type	Constraint	Description
_id	String (UUID)	PK, Required	Auto-generated unique identifier for the detection event.
plate	String	Indexed	Detected number plate string (OCR output); null if unreadable.
color	String	Required	Vehicle color classification label (e.g., White, Black, Silver).
bodyType	String	Required	Vehicle body type label (e.g., Sedan, SUV, Motorcycle, Truck).
confidence	Float	Required, 0.0–1.0	OCR confidence score for plate reading.
zoneId	String (FK)	Required, Indexed	References the Zone in which the detection occurred.
cameraId	String (FK)	Required, Indexed	References for the Camera device that captured the frame.
boundingBox	JSON Object	Required	Pixel coordinates of the detected plate bounding box {x,y,w,h}.
embedding	Float Array	Required	128-dimensional Re-ID feature embedding vector for cross-camera matching.
timestamp	DateTime	Required, Indexed	ISO 8601 timestamp of the detection event.

Table 45: Data Dictionary — Alerts Collection

Field Name	Data Type	Constraint	Description
_id	String (UUID)	PK, Required	Auto-generated unique identifier for the alert.
alertType	Enum	Required	Classification: Watchlist Hit | Clone | Speed Anomaly | Suspicious Activity.
severity	Enum	Required	Alert severity level: High | Medium | Low.
status	Enum	Required	Lifecycle status: New | Acknowledged | Escalated | Closed.
detectionId	String (FK)	Required, Indexed	References the Detection event that triggered the alert.
zoneId	String (FK)	Required, Indexed	Zone in which triggering detection occurred.
assignedTo	String (FK)	Nullable	References the admin User assigned to handle this alert.
adminAction	String	Nullable	Description of the admin action taken.
timestamp	DateTime	Required, Indexed	ISO 8601 timestamp of alert generation.
resolvedAt	DateTime	Nullable	ISO 8601 timestamp when the alert status changed to Closed.

Table 46: Data Dictionary — Cases Collection

Field Name	Data Type	Constraint	Description
_id	String (UUID)	PK, Required	Auto-generated unique identifier for the investigation case.
title	String	Required, Max:200	Short descriptive title of the investigation case.
caseType	Enum	Required	Classification: Vehicle Theft | Plate Clone | Traffic Offence | Other.
description	String	Required	Full narrative description of the incident under investigation.
priority	Enum	Required	Investigation priority level: Critical | High | Medium | Low.
status	Enum	Required	Case status: Open | InProgress | Under Review | Closed.
created By	String (FK)	Required	References to the admin User who created the case.
assignedTo	String (FK)	Nullable	References to the officer User to whom the case is assigned.
linkedAlertIds	String Array	Nullable	Array of Alert IDs linked as the source of this case.
linkedReportIds	String Array	Nullable	Array of citizen Report IDs linked as evidence sources.
createdAt	DateTime	Required, Auto	ISO 8601 timestamp of case creation.
closedAt	DateTime	Nullable	ISO 8601 timestamp of case closure.
resolutionNotes	String	Nullable	Free-text resolution summary entered by the senior admin on closure.

Table 47: Data Dictionary — Watchlist Collection

Field Name	Data Type	Constraint	Description
_id	String (UUID)	PK, Required	Auto-generated unique identifier for the watchlist entry.
plate	String	Required, Indexed	Number plate string flagged for real-time ANPR surveillance matching.
reason	Enum	Required	Reason for flagging: Stolen | Wanted | Suspect | LostPlate.
addedBy	String (FK)	Required	References for the admin User who added this entry.
expiryDate	Date	Nullable	Optional expiry date after which the entry status changes to Expired.
status	Enum	Required	Entry status: Active | Expired | Removed.
createdAt	DateTime	Required, Auto	ISO 8601 timestamp of watchlist entry creation.

Table 48: Data Dictionary — Reports Collection

Field Name	Data Type	Constraint	Description
_id	String (UUID)	PK, Required	Auto-generated unique identifier for the citizen report.
reportType	Enum	Required	Report classification: SuspiciousVehicle | Lost Plate | Stolen Plate.
citizenId	String (FK)	Required, Indexed	References to the User who submitted the report.
plate	String	Nullable, Indexed	Number plate string mentioned in the report (if provided by citizen).
description	String	Required	Narrative description of the incident as provided by the citizen.
gpsLocation	JSON Object	Required	Geolocation at time of submission: {lat, lng, accuracy}.
mediaUrls	String Array	Nullable	Array of secure storage URLs for uploaded photos and video files.
status	Enum	Required	Processing status: Pending | Under Review | Resolved | Dismissed.
zoneId	String (FK)	Required	Zone routed based on GPS coordinates of the report.
submittedAt	DateTime	Required, Auto	ISO 8601 timestamp of report submission.

Table 49: Logical Schema — All IVVSP Collections

Collection	Primary Key	Foreign Keys	Indexes	Relationships
users	_id (UUID)	zoneId → zones._id	_id, email (unique), phone (unique), role, zoneId	One user belongs to one zone (admins); one zone has many admins
zones	_id (UUID)	—	_id, name	One zone contains many cameras and many detections
cameras	_id (UUID)	zoneId → zones._id	_id, zoneId, healthStatus	One camera belongs to one zone; one camera produces many detections
vehicles	_id (UUID)	ownerId → users._id	_id, plate (unique), ownerId	One citizen owns many vehicles; one vehicle generates many detections
detections	_id (UUID)	zoneId → zones._id, cameraId → cameras._id	_id, plate, zoneId, cameraId, timestamp (descending)	Many detections per camera and zone; one detection triggers zero or one alert
watchlist	_id (UUID)	addedBy → users._id	_id, plate (unique), status, expiryDate	One watchlist entry triggers many alerts on detection
alerts	_id (UUID)	detectionId → detections._id, zoneId → zones._id, assignedTo → users._id	_id, status, severity, zoneId, timestamp (descending)	Many alerts link to many cases; one alert trigger one case at most initially
cases	_id (UUID)	createdBy → users._id, assignedTo → users._id	_id, status, priority, assignedTo, createdAt	One case links to many alerts and many reports; one case contains many evidence records
evidence	_id (UUID)	caseId → cases._id, uploadedBy → users._id	_id, caseId	Many evidence records belong to one case
reports	_id (UUID)	citizenId → users._id, zoneId → zones._id	_id, citizenId, plate, status, zoneId	Many reports link to one case; one citizen submits many reports
complaints	_id (UUID)	citizenId → users._id	_id, citizenId, status, category	One citizen submits many complaints; routed to authority queue

auditlogs	_id (UUID)	actorId → users._id	_id, actorId, timestamp (descending), action	Append-only; one log entry per significant system event

4.4 Data Flow Diagrams

Data Flow Diagrams (DFDs) provide a graphical representation of the flow of data through the IVVSP system. External entities are depicted as labelled rectangles; processes as rounded rectangles; and data stores as open-ended rectangles. Two levels of DFD are presented: Level 0 (Context Diagram) and Level 1 (Process Decomposition).

4.4.1 DFD Level 0 — Context Diagram

The Level 0 context diagram treats the entire IVVSP platform as a single process and identifies all external entities that exchange data with it: Citizens, Law Enforcement Administrators, the IP Camera Network, the NADRA Vehicle Registry, and the Traffic Authority. Citizens submit registrations, vehicle queries, reports, and complaints, and receive status results, confirmations, and safety advisories. Administrators submit login credentials, alert actions, case management operations, and configurations, and receive live detection feeds, alert notifications, case reports, and audit logs. The IP Camera Network provides continuous RTSP video streams. The NADRA API enables bidirectional vehicle registration lookups. The Traffic Authority receives routed complaints and watchlist notifications.

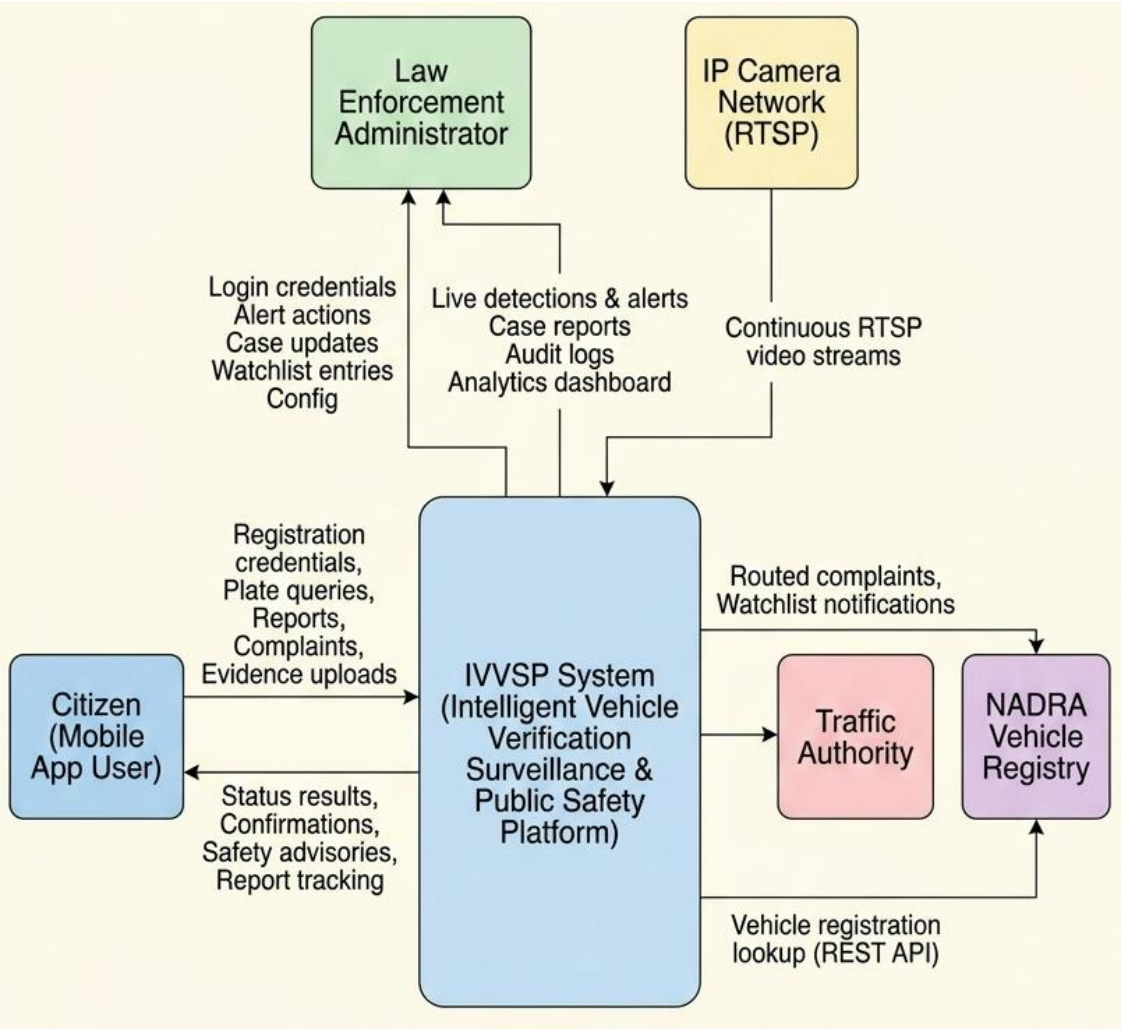


Figure 23: DFD Level 0 — IVVSP Context Diagram

4.4.2 DFD Level 1 — Process Decomposition

The Level 1 diagram decomposes the IVVSP system into six principal processes across seven data stores. P1 (User Authentication & Authorization) validates credentials against D1 (Users), issues JWT tokens, and writes authentication events to D7 (Audit Logs). P2 (AI Vehicle Detection Pipeline) ingests RTSP frames, executes the ANPR–Attribute–ReID pipeline, saves events to D2 (Detections), cross-references D6 (Watchlist), and publishes triggering events to P3. P3 (Alert Generation & Management) creates alert records in D3 (Alerts) and notifies administrators. P4 (Case & Evidence Management) manages the full case lifecycle across D4 (Cases & Evidence), reading D3 and D5, and logging all actions to D7. P5 (Citizen Reporting & Complaints) stores reports in D5 (Reports & Complaints), cross-references D6, notifies admins, and routes complaints to the Traffic Authority. P6 (Edge AI & Synchronization) receives synchronization batches from offline Edge Devices, writes buffered events to D2, and updates D7 with sync summaries.

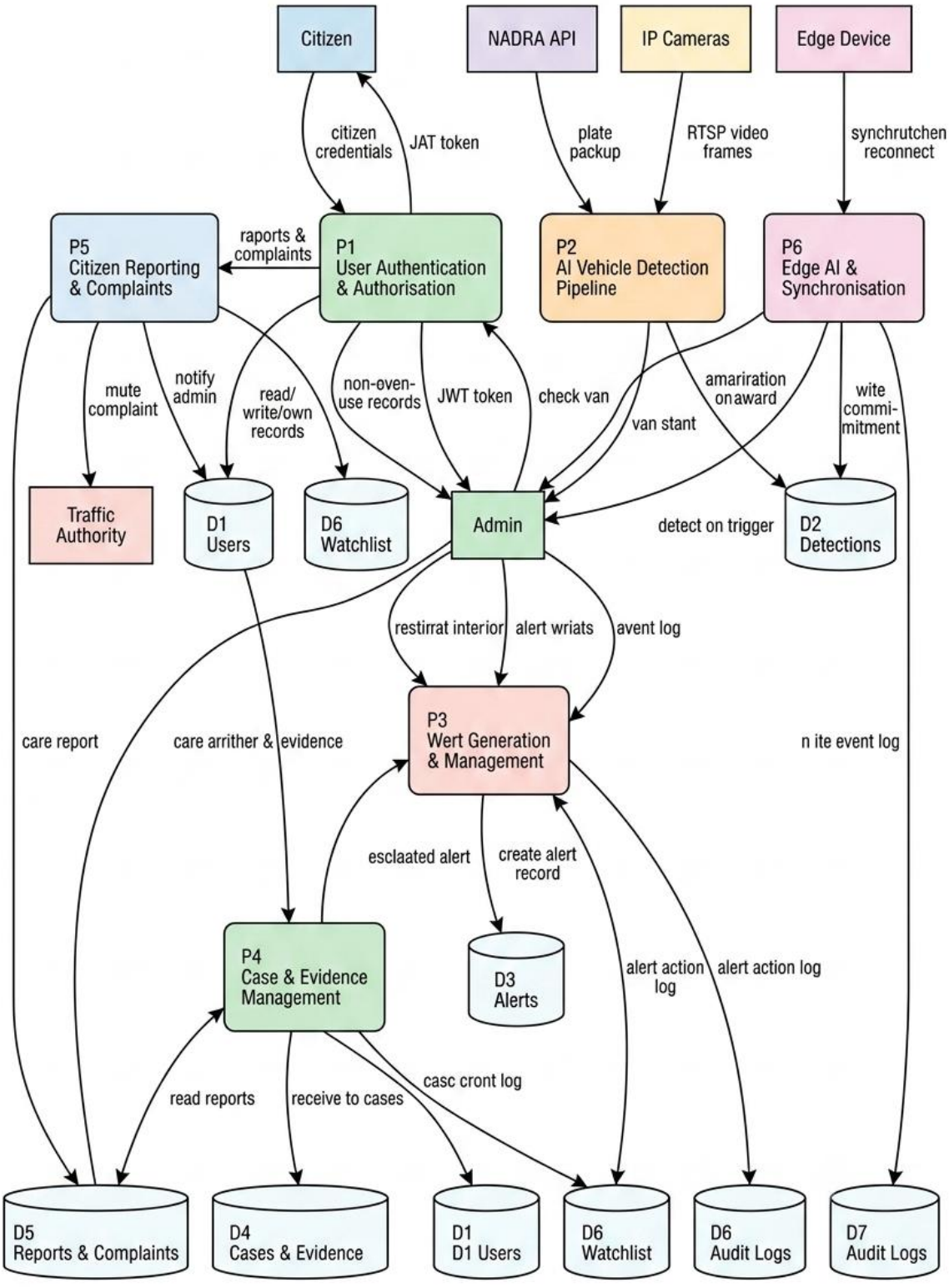


Figure 24: DFD Level 1 — IVVSP Process Decomposition

4.5 Class Diagram

The Class Diagram illustrates the 12 principal domain classes of the IVVSP platform, their attributes, operations, and inter-class relationships spanning the Backend Services Layer and

AI/ML Processing Layer. Associations, dependencies, and composition relationships are shown using standard UML notation with multiplicity labels.

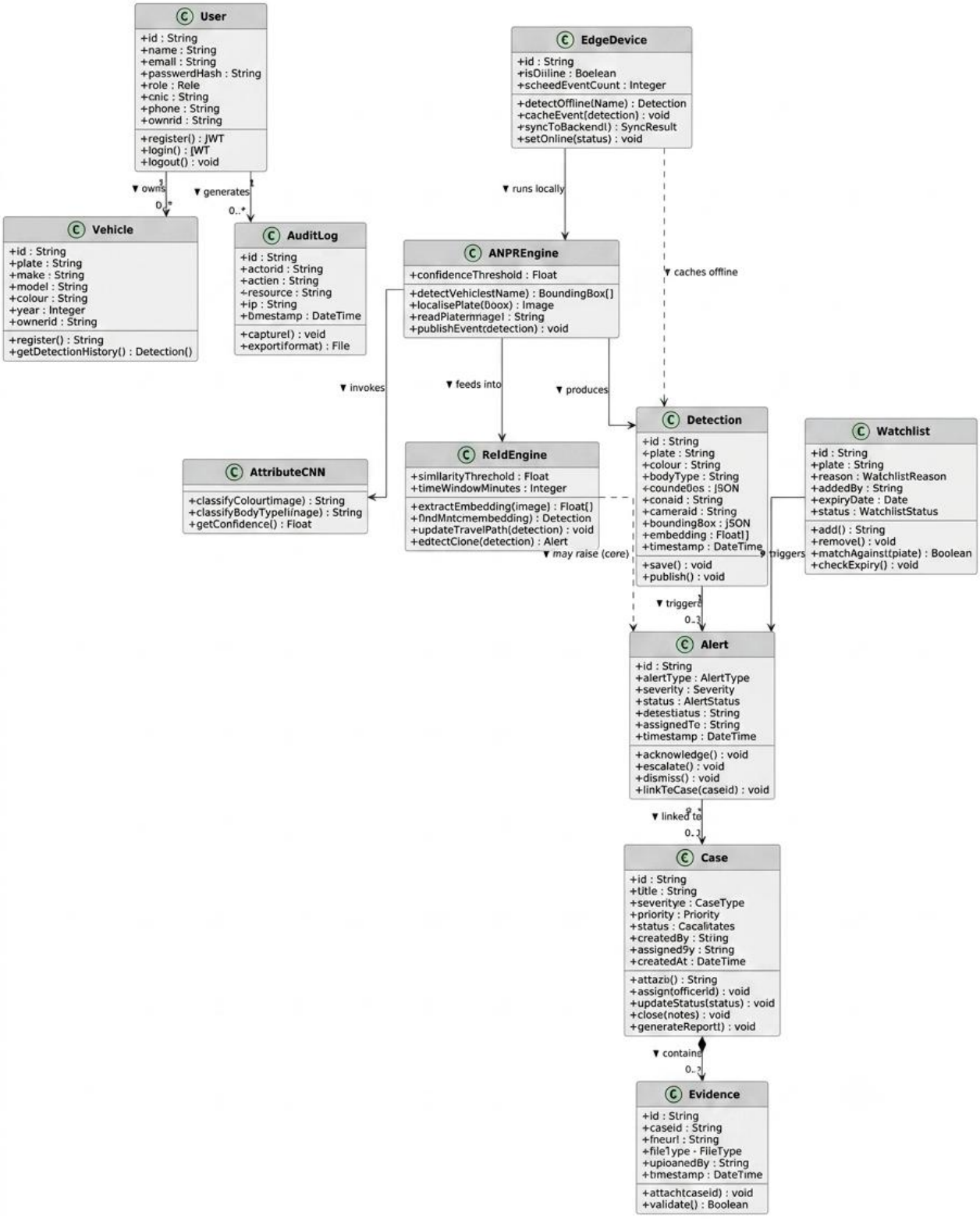


Figure 25: Class Diagram — IVVSP Platform

The User class is the central identity entity; citizen instances own Vehicle records and submit Report and Complaint instances. The ANPR Engine orchestrates the detection pipeline, invoking the AttributeCNN for color and body type classification and feeding the ReIDEngine with visual embeddings for cross-camera matching. The Detection class is the primary AI pipeline output and acts as the trigger source for Alert instances. The Watchlist class independently triggers Alerts when a stored plate matches a live Detection. Alert instances are linked to Case instances by administrative personnel, Cases aggregate Evidence records. The Edge Device class encapsulates offline capability by running a local ANPR Engine instance and managing its own SQLite cache before synchronizing via the Backend Sync API. The Audit Log class is write-only, capturing immutable records of every significant system event.

4.6 Activity Diagram

The Activity Diagram presents the main end-to-end operational workflow of the IVVSP system using UML swim lane notation. The four swim lanes correspond to the IP Camera / Edge Layer, the AI Processing Layer, the Backend / Rule Engine, and the Admin Dashboard. The diagram traces the complete lifecycle of a vehicle detection event — from initial frame capture through AI analysis, watchlist cross-referencing, alert generation, and ultimately to case creation and formal closure.

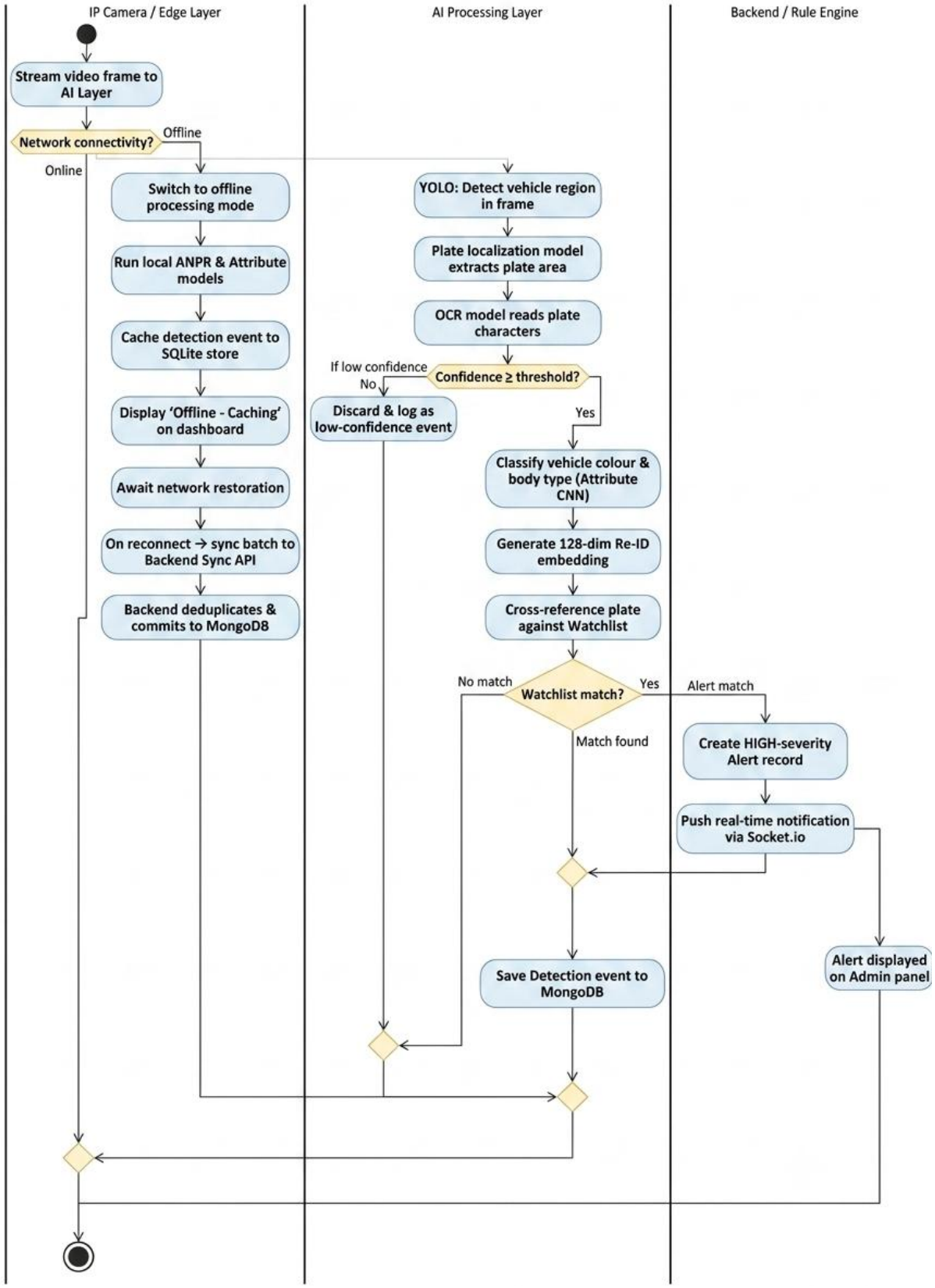


Figure 26: Activity Diagram — IVVSP Main System Operational Flow

The flow begins when the IP Camera streams video frames to the AI Processing Layer. A decision node evaluates network connectivity: if the edge device is offline, events are cached to the local SQLite store and synchronized upon reconnection; if online, frames are passed directly to the YOLO-based vehicle detection model. The ANPR engine localizes and OCR-reads the number plate and applies to the configured confidence threshold detections below the threshold are discarded and logged as low-confidence events. Accepted detections trigger parallel execution: vehicle color and body type are classified via the Attribute CNN, and a 128-dimensional Re-ID embedding is generated for cross-camera tracking. The plate is then cross-referenced against the Watchlist. A watchlist match causes the Rule Engine to generate a high-severity alert, which is immediately pushed via Socket.io to the Admin Dashboard. The zone administrator reviews the alert and may acknowledge it, dismiss it as a false positive, or creates a structured investigation case. The case passes through the OPEN → IN PROGRESS → UNDER REVIEW → CLOSED workflow, with an immutable audit log entry recorded at each stage of transition.

4.7 Sequence Diagram

The Sequence Diagram illustrates the chronological message of exchange between system objects for the primary scenario of a citizen reporting a suspicious vehicle, which subsequently triggers a watchlist alert and leads to investigation case creation. Six lifelines are defined: the Citizen Mobile App, the Mobile Backend API, the AI/ANPR Engine, the MongoDB Database, the Zone Admin Dashboard, and the Case Management Module.

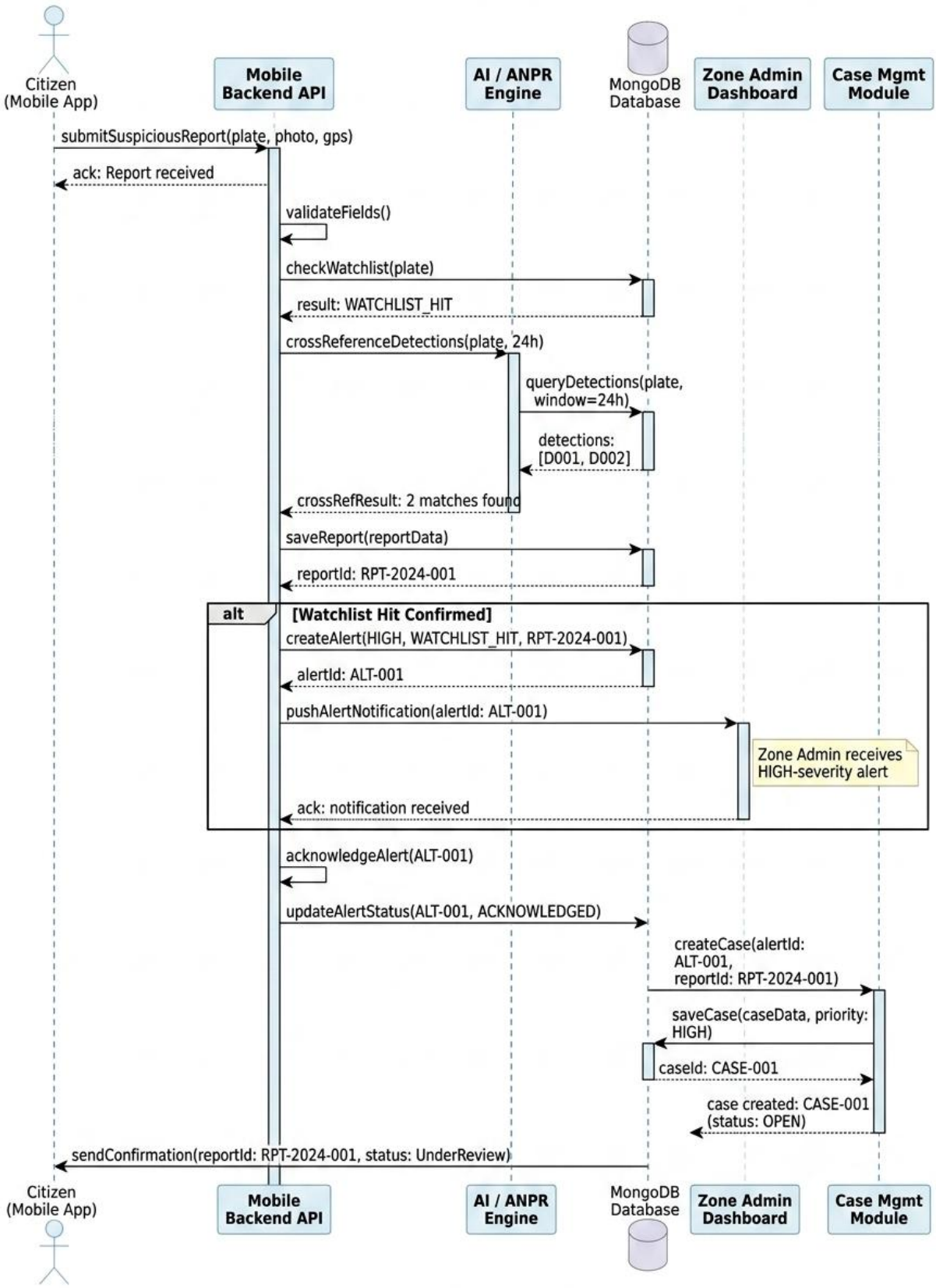


Figure 27: Sequence Diagram — Suspicious Vehicle Report, Alert & Case Creation Flow

The sequence is initiated when the citizen invokes submitSuspiciousReport() supplying the vehicle plate, photo evidence, and GPS coordinates. The backend API acknowledges receipt synchronously, then validates the submitted fields and calls checkWatchlist(plate) against MongoDB, which returns a WATCHLIST_HIT result. The backend next invokes crossReferenceDetections() on the ANPR Engine, which queries 24-hour detection history from MongoDB and returns matching detection records D001 and D002. The combined report data persisted to MongoDB, generating a unique Report ID (RPT-2024-001).

Because a watchlist hit was confirmed, the interaction enters the Watchlist Hit alternative fragment: the backend calls createAlert(HIGH, WATCHLIST_HIT), persists the alert to MongoDB (generating ALT-001), and pushes a real-time notification to the Zone Admin Dashboard via Socket.io. The zone administrator acknowledges the alert and invokes createCase() on the Case Management Module, supplying the alert and report IDs. The Case Management Module persists the case to MongoDB (generating CASE-001) and returns the case ID to the admin. Finally, the backend dispatches a confirmation notification containing the Report ID back to the citizen mobile application, closing the primary interaction.

4.8 Collaboration Diagram

The Collaboration Diagram (referred to as a Communication Diagram in UML 2.x) presents the vehicle detection and alert workflow with emphasis on the structural relationships and message links between participating objects rather than their temporal ordering. Numbered message labels on the association links indicate execution sequence. Nine objects participate in this collaboration: IPCamera, ANPR Engine, AttributeCNN, Detection, Watchlist, ReIDEngine, Alert, MongoDB, and ZoneAdmin.

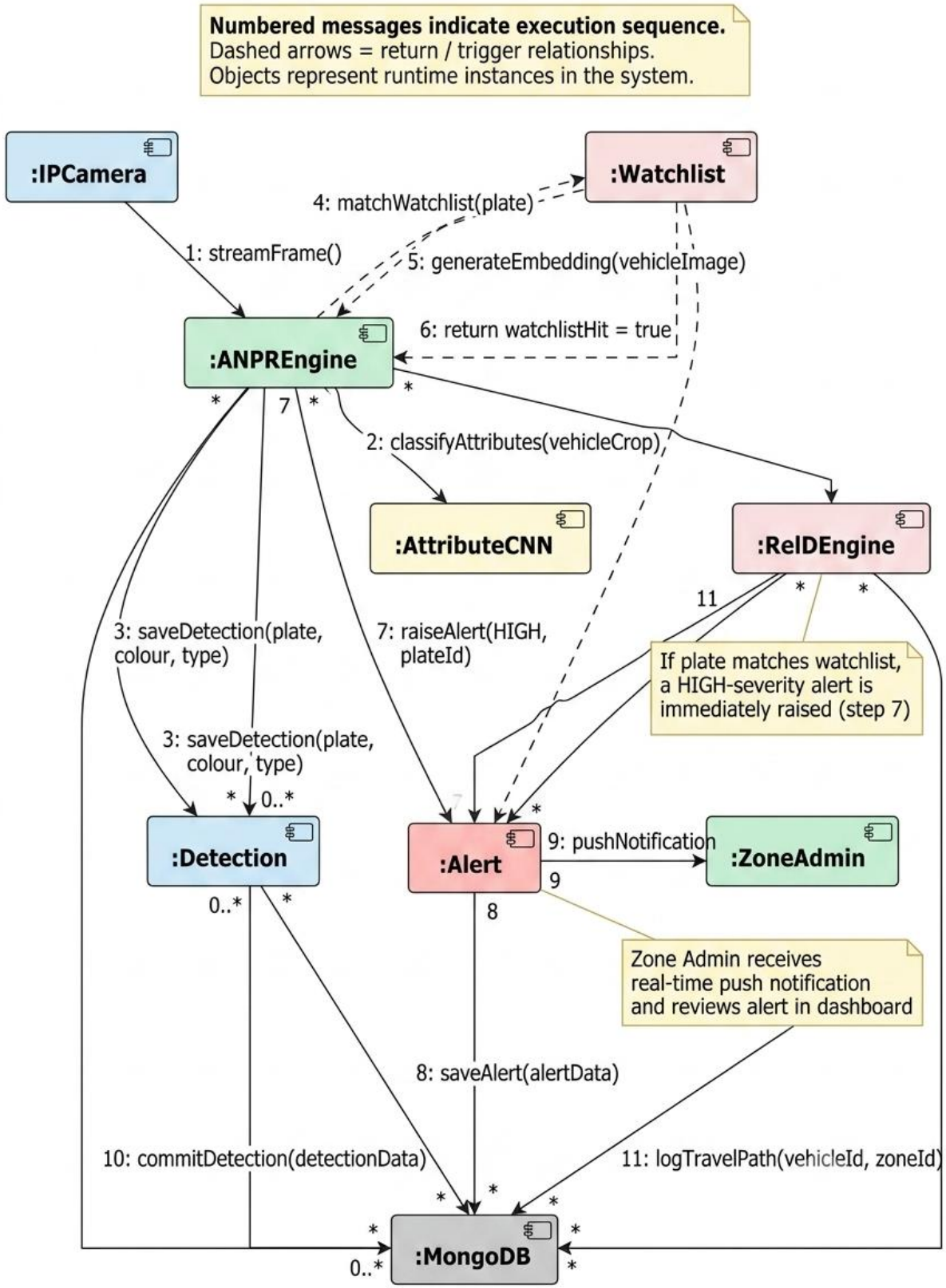


Figure 28: Collaboration Diagram — Vehicle Detection & Alert Workflow

The collaboration begins with the IPCamera sending streamFrame() [1] to the ANPR Engine. The ANPREngine concurrently invokes classifyAttributes() [2] on the AttributeCNN, saveDetection() [3] to create a Detection record, matchWatchlist(plate) [4] on the Watchlist object, and generateEmbedding() [5] on the ReIDEngine. The Watchlist returns watchlistHit: true [6], causing the Watchlist to raise raiseAlert(HIGH) [7] on the Alert object. The Alert object persists via saveAlert() [8] to MongoDB and pushes pushNotification() [9] to the ZoneAdmin. Simultaneously, the Detection record is committed to MongoDB [10] and the ReIDEngine logs to the updated cross-camera travel path [11].

This diagram confirms that the IVVSP object model is correctly decoupled: the Detection, Watchlist, Alert, and ReIDEngine objects each communicate through well-defined message interfaces without any direct coupling to the presentation layer components. The numbered message sequence provides an auditable trace of the object interaction that can be directly mapped back to the Sequence Diagram in Section 4.7 for cross-verification.

4.9 State Transition Diagram

The State Transition Diagram presents two concurrent finite state machines governing the lifecycle of the two most critical domain entities in the IVVSP system: the Investigation Case and the Alert. These two state machines together define the complete incident management pipeline from initial automated detection through human investigation to formal resolution.

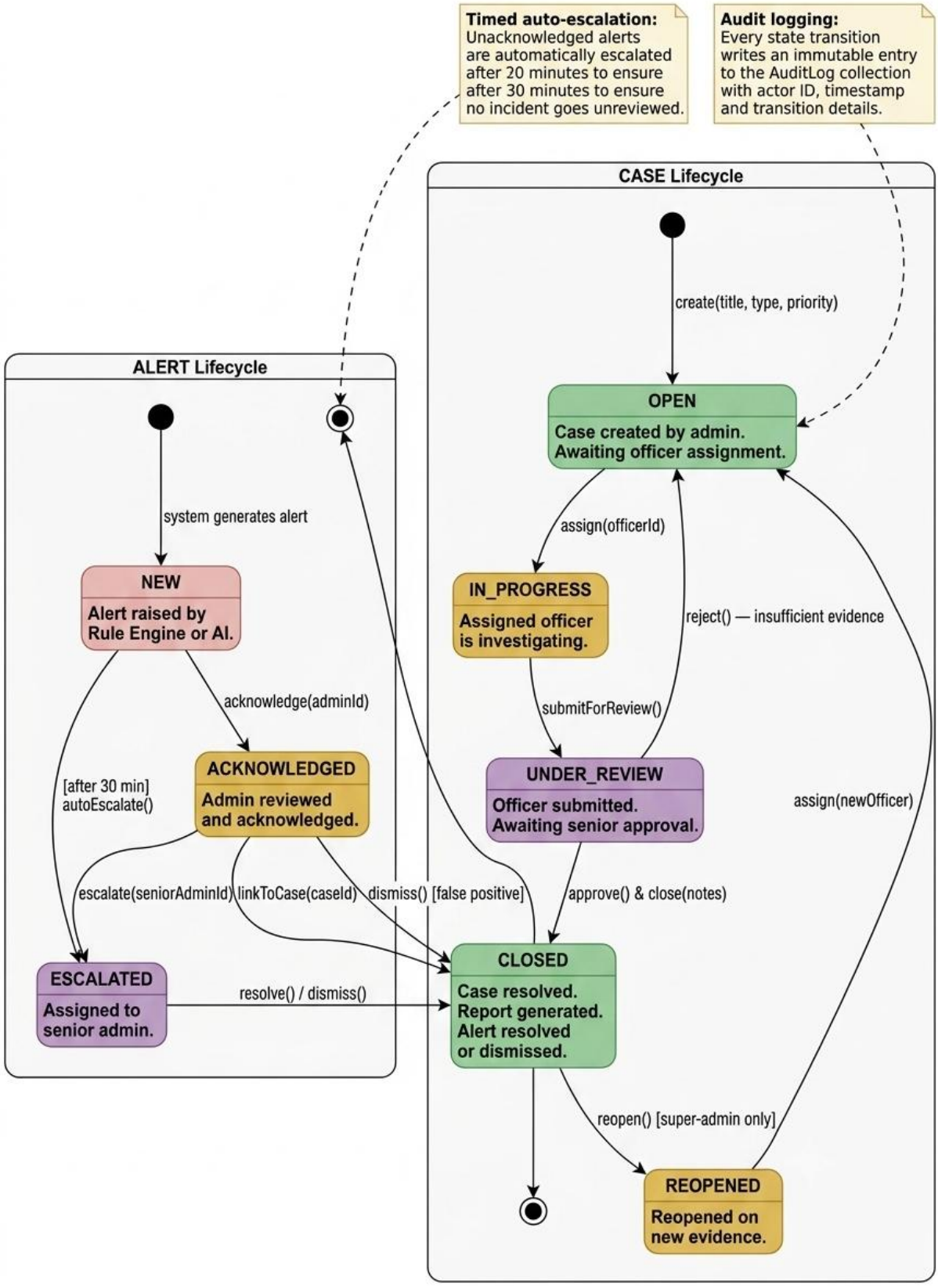


Figure 29: State Transition Diagram — Case & Alert Lifecycles

The Case Lifecycle state machine (upper section) begins at the initial pseudo-state, immediately transitioning to the OPEN state upon case creation. The assign(officer) guard event triggers the transition to IN PROGRESS, notifying the assigned investigation officer. Upon completing their investigation, the officer invokes submitForReview (), transitioning the case to UNDER REVIEW for senior administrator approval. The senior administrator may either approve and invoke close (), transitioning the case to CLOSED with a generated case summary report, or reject () the submission, returning the case to OPEN for reassignment. A case in CLOSED state may be RE-OPENED with super-admin authorization if new evidence emerges it transitions to RE-OPENED and subsequently returns to OPEN upon new officer assignment. The final state is reached from CLOSED, depicted as a filled circle within a ring.

The Alert Lifecycle state machine (lower section) begins immediately upon system generation of an alert event, placing the alert in the NEW state. If a zone administrator acknowledges the alert within the configured timeout window (30 minutes), it transitions to ACKNOWLEDGED. Unacknowledged alerts are automatically escalated to the ESCALATED state by a timed system trigger, depicted as a dashed arc with a guard annotation. From both ACKNOWLEDGED and ESCALATED states, the alert may be resolved either via dismiss (marking it as a false positive and archiving it) or via linkToCase() transitioning it to the CLOSED state. The final pseudo-state is reached by CLOSED.

4.10 Component Diagram

The Component Diagram presents the software architecture of the IVVSP platform at the component level, identifying the principal deployable software units within each architectural tier and the interfaces through which they are connected. Standard UML component notation is used: each component is represented as a rectangular box with the UML component icon (a rectangle with two smaller rectangles on its left edge). Provided interfaces are shown as lollipop symbols; required interfaces as socket symbols. Five primary subsystems and one external systems group are identified.

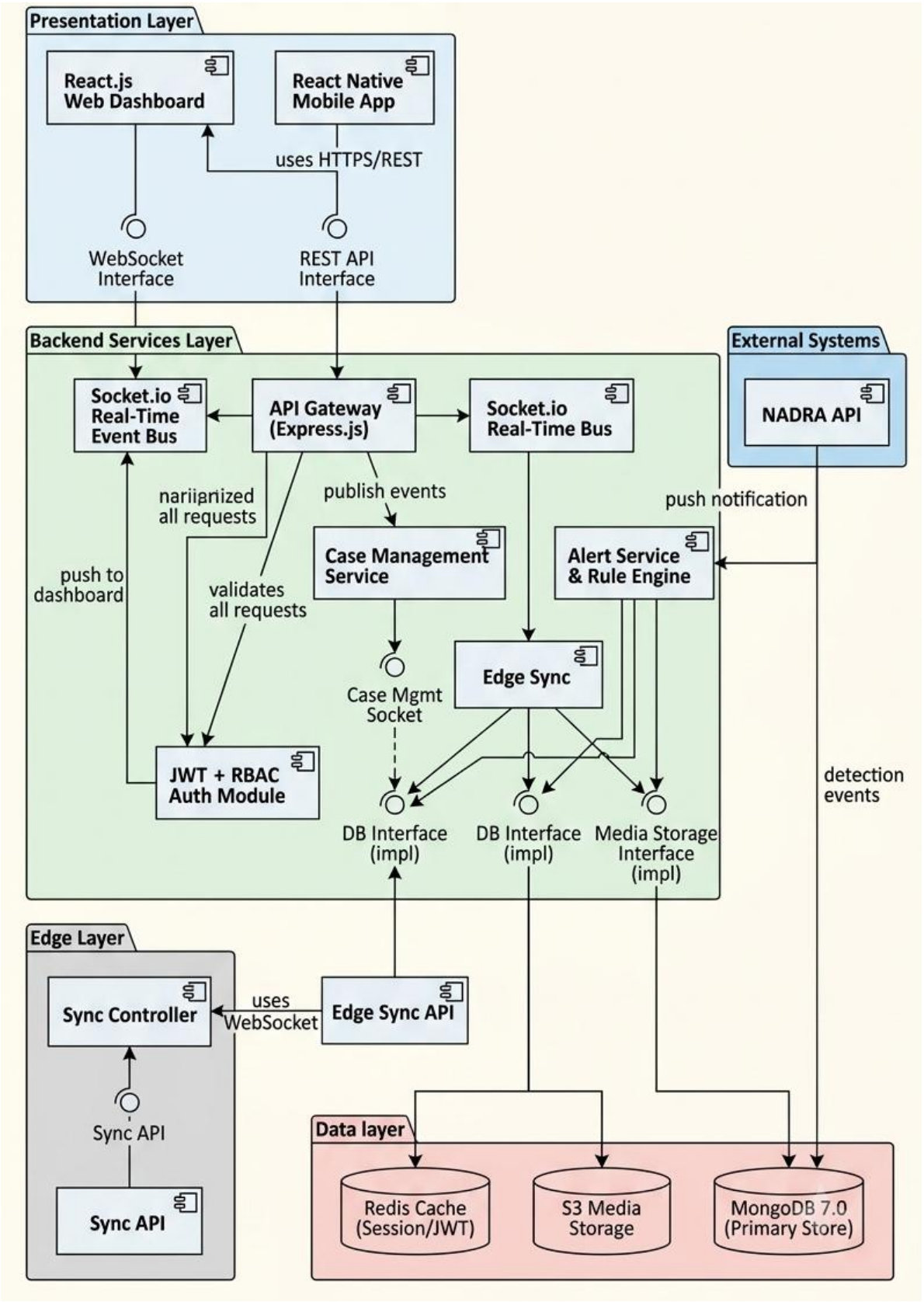


Figure 30: Component Diagram — IVVSP Platform

The Presentation Layer subsystem contains two components: the React.js Web Dashboard (which renders the administrative interface and consumes real-time WebSocket events) and the React Native Mobile App (which provides the citizen-facing interface). Both components expose a REST API required interface consumed by the API Gateway in the Backend Services Layer.

The Backend Services Layer subsystem contains six components: the Express.js API Gateway (primary request router), the JWT + RBAC Auth Module (authenticates and authorizes every inbound request), the Socket.io Event Bus (streams real-time detection events and alerts to connected admin clients), the Edge Sync API (accepts batch synchronization payloads from offline edge devices), the Case Management Service (implements the case lifecycle business logic), and the Alert Service (implements the alert generation and notification rules engine). The API Gateway and Socket.io Event Bus expose a Database Interface required interface consumed by the MongoDB component in the Data Layer.

The Data Layer subsystem comprises three components: MongoDB (the primary document data store for all application collections), Redis Cache (used for session management and JWT deny-listing to enforce logout revocation), and S3-compatible Media Storage (for secure evidence file storage). The AI/ML Processing Layer subsystem contains the ANPR Engine, Attribute CNN, Re-ID Engine, and Rule Engine all of which expose Detection Events and Alert Trigger provided interfaces consumed by the Backend Services Layer. The Edge Layer subsystem contains the Local ANPR & Attribute Models, the SQLite Local Cache, and the Sync Controller, the last of which exposes a Sync API provided interface to the Backend Sync API upon connectivity restoration.

4.11 Deployment Diagram

The Deployment Diagram presents the physical infrastructure topology of the IVVSP platform, mapping all software artefacts to the hardware nodes on which they execute. UML 3D node notation is used throughout: each node is depicted as a three-dimensional box representing a physical device or server. Communication links between nodes are annotated with the network protocol employed. Eight node types are identified across four network segments.

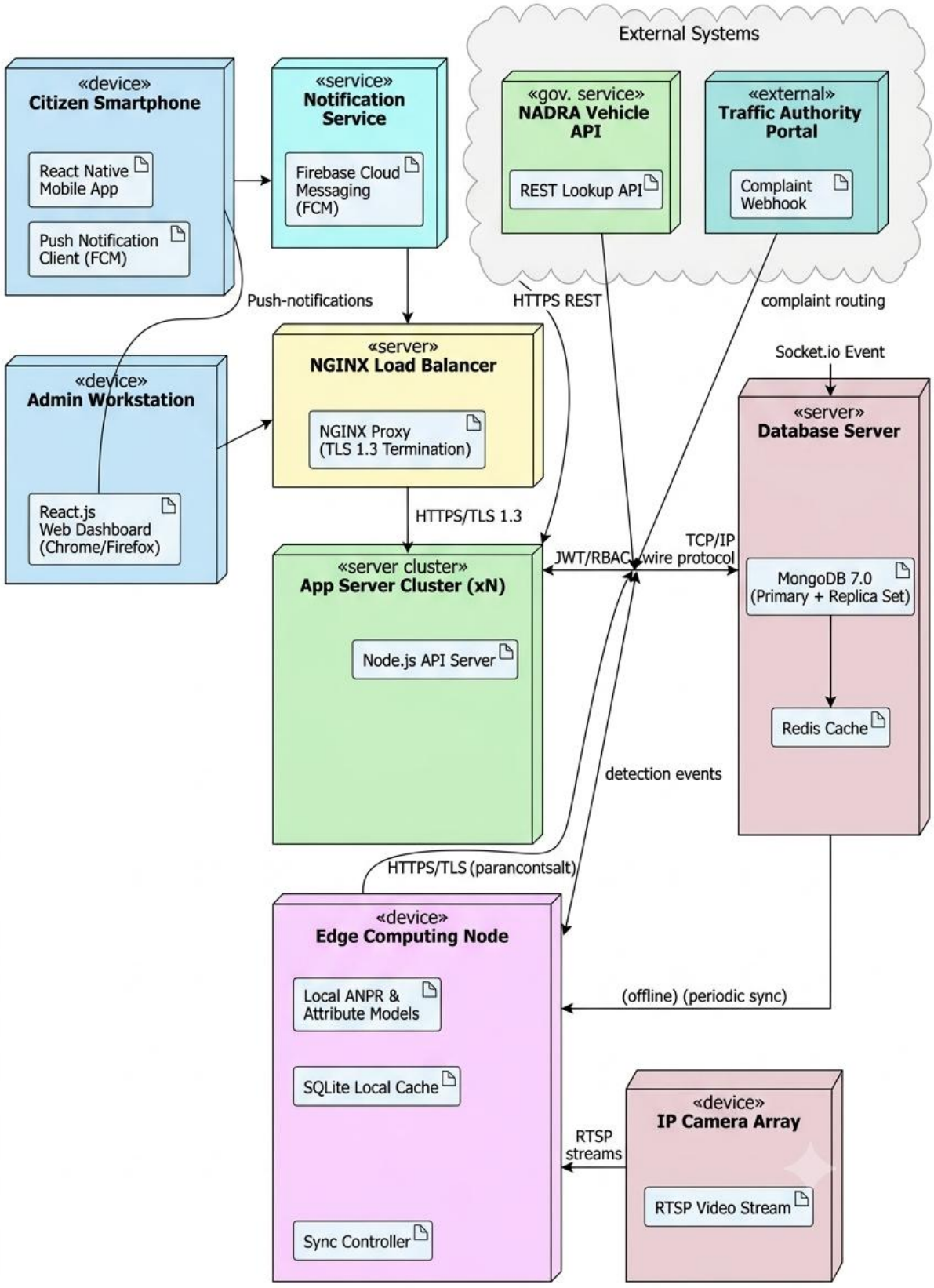


Figure 31: Deployment Diagram — IVVSP Physical Infrastructure

In the Client Network segment, Citizen Smartphones (Android 10+ / iOS 14+) host the React Native mobile application and a push notification client. Admin Workstations (Windows 10/11 or macOS) run the React.js Web Dashboard within a standard web browser (Chrome 120+ / Firefox 120+). Both device types communicate exclusively through a NGINX Load Balancer node, which performs TLS 1.3 termination and distributes traffic across the App Server Cluster.

In the Application Network segment, the App Server Cluster comprises one or more Ubuntu 22.04 Docker nodes, each running the Node.js API Server, the Socket.io Event Bus, and the JWT/RBAC Auth Module as containerized services orchestrated by Docker Compose. The cluster communicates with the Database Server (Ubuntu 22.04) which runs MongoDB 7.0 in a Primary + Replica Set topology, Redis Cache, and S3-compatible Media Storage via internal TCP connections. A separate Notification Service node hosts the Firebase Cloud Messaging (FCM) integration, and dispatches push notifications to citizen smartphones via the FCM HTTPS API.

In the AI Processing segment, the AI Processing Server (Ubuntu 22.04 with GPU acceleration) runs the ANPR Engine, Attribute CNN, Re-ID Engine, and Rule Engine in Docker containers. This server receives RTSP video streams directly from IP Camera nodes co-located on the surveillance network and communicates detection events to the App Server Cluster via internal REST over HTTPS. Edge Computing Nodes (NVIDIA Jetson Nano devices) are co-located with camera groups: they run local copies of the ANPR and Attribute models, buffer detection events to SQLite during connectivity outages, and synchronize buffered events to the App Server Cluster Edge Sync API via HTTPS upon reconnection.

Two external system nodes are shown outside the organizational network boundary: the NADRA Vehicle API (a government REST interface used for real-time vehicle registration lookups) and the Traffic Authority Portal (a webhook endpoint that receives routed citizen complaint notifications). Both external connections traverse the public internet over HTTPS with TLS 1.3.

4.12 Chapter Summary

This chapter presents the complete system design for the IVVSP platform across eleven distinct design perspectives. The System Architecture Diagram established the five-tier layered structure spanning Presentation, Backend Services, AI/ML Processing, Data, and Edge layers. The Entity Relationship Diagram, Data Dictionary (Tables 4.1–4.6), and Logical Schema

(Table 4.7) collectively defined a comprehensive data model across 12 MongoDB collections with explicit field constraints, index strategies, and inter-collection relationships satisfying NFR-17. The DFD Level 0 and Level 1 diagrams formalize all data flows across six primary processes and seven data stores. The Class Diagram established 12 domain classes with behavioral operations and UML associations.

The Activity Diagram traced the complete vehicle detection and case management workflow using four-lane swim lane notation. The Sequence Diagram specified the chronological message exchange for the suspicious vehicle reporting scenario across six lifelines. The Collaboration Diagram confirmed correct object decoupling through numbered message links across nine participating objects. The State Transition Diagram formalized the finite state machines for both the Case and Alert entities. The Component Diagram identified all deployable software units and their provided/required interface contracts. The Deployment Diagram mapped all software artefacts to their physical infrastructure nodes across clients, applications, AI processing, and external network segments, annotating all inter-node communication protocols.